Layer Cake: Language Representation and Compute Characteristics of ML, DL & LLM Text Classifiers

by Peter J Worth & Dr. Ionut Cardei
Florida Atlantic University
Dept of Electrical Engineering and Computer Science

Spring 2025

1 Table of Contents

1 TABLE OF CONTENTS 1-2

2 FIGURE INDEX 2-3

3 TABLE INDEX 3-3

4 ABSTRACT 4-4

5 PREAMBLE & RESEARCH CONTEXT 5-5

5.1 Language Representation (Learning) 5-5
5.2 Text Classification, Deep Learning and Word-Class Embeddings 5-11
5.3 Unique Contributions of this Work 5-14
5.4 Structure of this Paper 5-16

6 EXPERIMENTAL DESIGN: TEST SCOPE & DATA PIPELINES 6-17

6.1 Supported Language Models 6-20
6.1.1 Static Embedding Models 6-21
6.1.2 Context-Aware Language Models (Transformers or LLMs) 6-22
6.2 Supported Datasets 6-25
6.2.1 Reuters21578: multi-label, 115 classes 6-26
6.2.2 BBC News: single-label, 5 classes 6-26
6.2.3 RCV1-V2: multi-label, 103 classes 6-26
6.2.4 20 Newsgroups: single-label, 20 classes 6-26
6.2.5 arXiv: multi-label, 58 classes 6-27
6.2.6 arXiv protoformer: single-label, 10 classes 6-27
6.2.7 OHSUMED: multi-label, 23 classes 6-27
6.2.8 IMDB: single-label, 2 classes 6-27
6.2.9 Dataset Summary Table 6-28
6.3 Supported Metrics 6-28
6.4 Language Representation and Test Workflow 6-29
6.4.1 Machine Learning Approaches (Baseline Testing) 6-29
6.4.2 Neural Models (Deep Learning) 6-33

7 RESULTS & ANALYSIS 7-42

7.1 ML Classifiers and Representation Forms 7-42
7.1.1 By ML Classifier 7-42
7.1.2 By Representation Form 7-45
7.2 Neural Classifiers (Deep Learning) 7-51
7.2.1 By Language Model 7-52
7.2.2 By Classifier 7-54
7.2.3 By Dataset 7-55
7.3 Global Analysis across all Classifiers 7-57

8 SUMMARY & CONCLUDING REMARKS 8-64

8.1 Areas of Promising Research 8-65

9 BIBLIOGRAPHY 9-67

2 Figure Index

FIGURE 1: DOCUMENT SIMILARITY QUERY IN VECTOR SPACE 5-6
FIGURE 2: WORD SIMILARITY IN VECTOR SPACE USING DISTRIBUTIONAL HYPOTHESIS 5-7
FIGURE 3: TOKEN EMBEDDINGS + INPUT EMBEDDINGS WITH TRANSFORMERS 5-10
FIGURE 4: BERT INPUT REPRESENTATION WHICH INCLUDES SEGMENT EMBEDDINGS (DEVLIN, ET AL., 2019) 5-11
FIGURE 5: LANGUAGE REPRESENTATION DEVELOPMENT TIMELINE & MILESTONES 5-13
FIGURE 6: LAYER CAKE CLASSIFIER TESTING ARCHITECTURE 6-17
FIGURE 7: LANGUAGE MODEL SUMMARY TABLE 6-25
FIGURE 8: STATIC LANGUAGE MODEL CLASSIFIER (CNN ARCHITECTURE) 6-36
FIGURE 9: LCCNNBERTCLASSIFIER ARCHITECTURE (TRANSFORMERS) 6-39
FIGURE 10: MACHINE LEARNING CLASSIFIER PERFORMANCE SUMMARY: MACRO-F1 AND MICRO-F1 SCORES 7-43
FIGURE 11: ML CLASSIFIER REPRESENTATION FORM RESULTS: MACRO-F1 7-46
FIGURE 12: ML CLASSIFIER REPRESENTATION FORM RESULTS: MICRO-F1 7-47
FIGURE 13: NEURAL CLASSIFIER LANGUAGE MODEL TYPE PERFORMANCE SUMMARY 7-53
FIGURE 14: NEURAL MODEL CLASSIFIER PERFORMANCE SUMMARY 7-54
FIGURE 15: GLOBAL CLASSIFIER PERFORMANCE - MACRO-F1 SCORES 7-58
FIGURE 16: GLOBAL CLASSIFIER PERFORMANCE - MICRO-F1 SCORES 7-58

3 Table Index

TABLE 1: ML CLASSIFIER SUMMARY DATA: MACRO-F1 (TOP 5) 7-44
TABLE 2: ML CLASSIFIER SUMMARY DATA: MICRO-F1 (TOP 5) 7-45
TABLE 3: ML CLASSIFIERS, MULTI-LABEL DATASETS MACRO-F1 VALUES BY REPRESENTATION FORM 7-48
TABLE 4: ML CLASSIFIERS, SINGLE-LABEL DATASETS MACRO-F1 VALUES BY REPRESENTATION FORM 7-49
TABLE 5: ML CLASSIFIERS, MULTI-LABEL DATASETS MICRO-F1 VALUES BY REPRESENTATION FORM 7-50
TABLE 6: ML CLASSIFIERS, SINGLE-LABEL DATASETS MICRO-F1 VALUES BY REPRESENTATION FORM 7-51
TABLE 7: DL CLASSIFIER PERFORMANCE BY DATASET - MACRO-F1 VALUES 7-56
TABLE 8: DL CLASSIFIER PERFORMANCE BY DATASET - MICRO-F1 VALUES 7-57
TABLE 9: SUMMARY BY ALL CLASSIFIERS: MACRO-F1 AND MICRO-F1 SCORES 7-59
TABLE 10: GLOBAL SUMMARY BY DATASET (MACRO-F1 SCORES) 7-60
TABLE 11: GLOBAL SUMMARY BY DATASET (MICRO-F1 SCORES) 7-61
TABLE 12: SUMMARY GLOBAL MACRO-F1 GAP SCORES & TIMES 7-62
TABLE 13: SUMMARY GLOBAL MICRO-F1 GAP SCORES & TIMES 7-62

4 Abstract

The introduction and widespread adoption of LLMs over the last few years has changed the entire technological landscape, ushering in a wave of transformation (literally and figuratively) that rivals the changes brought about by the advent of the Internet in tandem with broadband networking at the turn of the century. These LLMs have been shown to be effective not just in language generation capabilities but also excelling in tasks such as machine (language) translation, question & answering, text summarization and also text classification, the subject of this work. However there are significant computational requirements associated with working with LLMs and despite the introduction of a variety of optimization techniques such as quantization, prefix tuning, low rank adaptation (LoRA) and a variety of prompting strategies, an analysis of LLM performance versus more classical approaches from machine learning (ML) and even augmented embedding approaches in tandem with static language models such as GloVe, Word2Vec or FastText is missing from the research literature. This research addresses this gap by analyzing the performance and compute requirements of text classifiers from across the ML and DL landscape, and across the language model landscape, leveraging word and subword based language models such as Word2Vec, GloVe and FastText, to the more powerful and computationally expensive tokenized, transformer based approaches to language representation which underpin modern LLMs, including models from the BERT, RoBERTa, DistilBERT, XLNet, GPT-2, Llama and DeepSeek families. We look at the performance as well as compute requirements of the different classifiers and language model configurations finding that LLMs, when fine-tuned properly and fully, given their breadth and size, do in fact perform the best relative to the other classifiers we test - 8.87% better Macro-F1 score than static word embedding DL approaches and 12.73% better than classical ML approaches - this comes at a significant computational cost for fine-tuning the underlying model, some 4398.07% increase over the static embedding DL approaches and 4126.02% more computational time for the classical ML classifier training. The analysis we provide here should give pause to researchers and practitioners alike to ensure that their model choices and related optimization techniques are properly right sized for the task, and resources, of the problem at hand.

Keywords:

Language Representation Learning, Text Classification, NLP, Word Embeddings, Vector Space Models, LLMs, Language Models, Semantic Spaces, Language Generation, Computational Linguistics

5 Preamble & Research Context

The current state of Artificial Intelligence (AI), and more specifically the *natural language generation* (NLG) tools that make up the foundations of modern AI, are not just indistinguishable from a moderately intelligent human being at this point, but in many respects closer to some of the most intelligent individuals on the planet. Many milestones along these lines have been achieved over the last few years along these lines, most notably the introduction of ChatGPT in November 2022, closely followed by GPT-4 passing the bar exam in early 2023 (Katz, et al., 2024), and then the Turing Test being officially passed by both GPT-4.5 and LLaMa-3.1 in March of 2025 (Jones & Bergen, 2025). The current innovation that holds much promise for these language generation tools is the introduction of advanced reasoning capabilities, sometimes referred to as *chain-of-thought*, which was first introduced in January of 2022 (Wei, et al., 2022), facilitating the introduction of more advanced, step-wise analytical problem solving capabilities (e.g. mathematical problems and proofs) into many LLMs. For a good survey on the current state of advanced reasoning capabilities, particularly as it relates to Reinforcement Learning (RL), see (Xu, et al., 2025).

The technological advancements necessary to facilitate this rapid advancement in language generation capabilities which are now being integrated into applications across the technological landscape, reflect decades of research and development across linguistics, computer science, NLP and AI, as well as of course advancements in computing technology which are fundamental to the training and support of these large language models (LLMs) given their size and the computational power necessary to both construct LLMs and also fine-tune them for specific tasks or datasets. All of these technological advancements have led to radical changes not only with respect to how humankind conducts and transacts business throughout the globe, but also – among other things - with respect to how we interact with each other, and even how nation states and corporations govern their constituents.

5.1 Language Representation (Learning)

While there are many innovations that have supported the development of LLMs which are radically changing not just the AI landscape but the technology landscape writ large, one of the research areas that receives less attention is the impact of various forms of *language representation* on the effectiveness of various NLP solves, hence part of the motivation for this work. Different types of transformers, and now different types of reasoning models, have been the focus of much R&D in the last few years, and for good reason as they have proven to be very effective at solving a variety of NLP problems, with text classification and language generation being two prime examples.

But these different forms of language representation are not just technical data input requirements to the different types of classifiers (ML and DL) but also literally provide the underlying geometric configuration, i.e. *semantic space* following (Worth, 2023), within which the various classifiers optimize within in order to classify the underlying data. This connection between NLP and linguistics is long standing and rests at the very heart of language models in all their forms, resting on principles of semantics and morphology that were established primarily by Noam Chomsky in the middle of the last century[1]. One of these foundational

[1] His most influential works with respect to computational linguistics perhaps being "Three Models for the description of Language", an article published in 1956 where he introduces *formal grammars* which underpin computer language theory (Chomsky, 1956), *Syntactic Structures* published in 1957 where he introduces the notion of *generative grammars* and the now famous hierarchy of languages which is foundational to

principles is known as the *distributional hypothesis*, a fundamental tenet of semantic theory introduced in the middle of the last century by Zellig Harris (Harris, 1954), and J.R. Firth (Firth, 1957), the latter author being perhaps best known for his description of the distributional semantics as "*you shall know a word by the company it keeps*". This quite simple yet extraordinarily powerful principle, that the meaning of words can be inferred by the context within which they are found in a textual corpus, remains a key insight which provides much of the theoretical foundations of many forms of language representation which have been introduced in the last few decades.

It is these advancements in language representation in fact, culminating in the introduction of the *attention mechanism* which was introduced in 2017 (Vaswani, et al., 2017), combined with rapid advancement in the underlying compute capabilities necessary to train and tune these LLMs (GPUs), that has provided the technological foundations within which this rapid advancement of AI capabilities is possible. These forms of language representation range from the simple, such as one hot encoding, to more complex forms of *text vectorization* such as term-frequency, inverse-document frequency (TF-IDF) which was introduced as far back as 1972 (Sparck Jones, 1972) and, to the more advanced methods such as *vector space models* (Salton, 1971) which were introduced a means by which document search and information retrieval could be solved for.

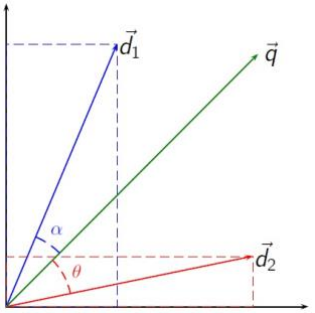


Figure 1: Document Similarity Query in Vector Space

The next major milestone in language representation came a few decades later with the discovery that these text vectorization approaches could be combined with core linear algebraic operations such as eigendecomposition to identify latent contextual, or semantic, relationships between words or terms in a document corpus, a technique that is still widely used today and remains a very effective means of feature reduction of text called Latent Semantic Analysis, or Latent Semantic Indexing (LSA/LSI) (Deerwester, et al., 1990). We use LSA as one of our baseline feature reduction techniques in this work and our findings show that it remains an effective form of language representation, at least within the context of text classification.

Vector space models ultimately evolve into one of the most fundamental forms of language representation that are still widely used today, namely *word embeddings*. This powerful form of language representation method was first introduced in 2013 with Word2Vec (Mikolov, et al., 2013), and uses deep learning and word co-occurrence metrics to *learn* embeddings by predicting words based on (local) word contexts[2] using either a continuous bag of words (CBOW) approach, where the model is optimized to predict the target word given a specific context, or a Skip-Gram predictive approach where the context of a word is predicted

Computation Theory (Chomsky, 1957), and the book *Aspects of the Theory of Syntax* first published din 1965 where he further elaborates on the concept of generational grammars and describes the distinction between deep and surface structures which inform our modern notions of syntax and semantics which underpin NLP (Chomsky, 1965).

[2] Window context being defined as for example 5 words before or after the current target word where 5 is configurable training parameter for the model.

given the target word. Ultimately, Word2Vec language models produce embeddings that reflect the distributional hypothesis, where words that appear in similar contexts are represented by embeddings that are geometrically close to each other as defined by the underlying embedding / vector space. In other words, the model is trained to find the coordinates each word in a latent feature space where distances between vectors reflect their semantic similarity.

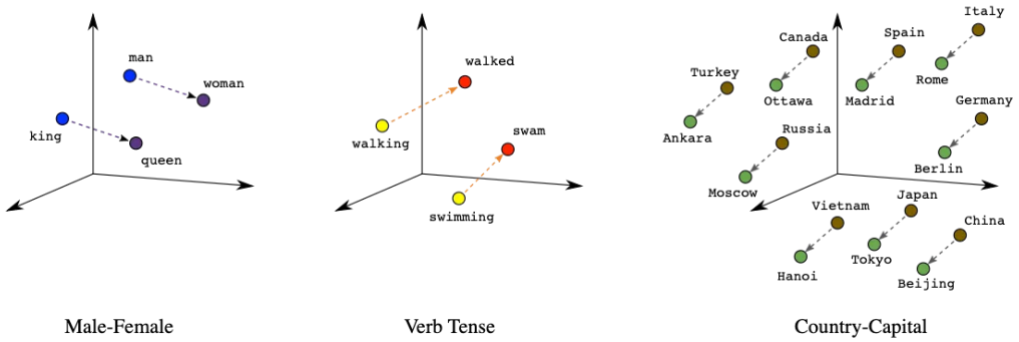


Figure 2: Word Similarity in Vector Space using distributional hypothesis

GloVe (Global Vectors for Word Representation) embeddings on the other hand, introduced in 2014 (Pennington, et al., 2014), leverages both local and global word context to generate word embeddings, explicitly constructing a *word co-occurrence matrix* during training that records how often two words appear together within a context window across a corpus. It then factorizes this matrix via log-bilinear regression, optimizing a cost function which minimizes the difference between the dot product of two-word vectors and the logarithm of their co-occurrence count, effectively balancing both local and global co-occurrence statistics for the words in question.

FastText on the other hand, first introduced in 2017 by Meta's AI team (Bojanowski, et al., 2017), while very much related to Word2Vec in the sense that it is also a local co-occurrence computed (static) word embedding model, it is unique in that it introduces the concept of *subwords*, or n-grams, to provide better support for out of vocabulary terms (OOV) that were not including in the training dataset, very much a precursor to the token based parsing approach used by LLMs. With fastText embeddings, each word is represented not only by its word-level embedding but also by the embeddings of its subwords, enabling the language representation form to compute OOV words by composing them from the embeddings of their subwords. During training, FastText learns embeddings not just for entire words but also for their subword n-grams, and when creating the final vector for a word, FastText aggregates the word's subword embeddings (and its own embedding if available) into a single vector, resulting in embeddings that can handle morphologically rich languages better than Word2Vec which cannot break words into smaller units.

Just like Word2Vec, FastText operates in a *continuous Euclidean space* (or more generally, a vector space) where words with similar meanings have vectors that are close together and relationships between words can be modeled as linear transformations (e.g., "king - man + woman ≈ queen"). Since FastText breaks words into subword units (n-grams), this means that similar words (e.g., "running" and "runner") will have more similar vectors than they would in Word2Vec, due to shared subword components ("run"). FastText also constructs vectors geometrically, learning word and subword representations from their co-occurrence patterns in a latent space. This underlying geometrical configuration allows FastText to capture not just word-level relationships, but also similarities across words with shared subwords, such as *morphological* variations of the verb 'run', such that the words "run," "running," "runner" will have closer embeddings because of shared subwords. Because of its inherent subword design, FastText can produce reasonable

embeddings for out-of-vocabulary words based on their subwords than their word-based counterparts such as GloVe or Word2Vec.

TF-IDF / Count Vectorization is explicit, algebraic construction based on word frequencies. Each vector dimension represents a distinct, interpretable feature (e.g., the presence or absence of a word), representing a direct frequency-based transformation of the raw data. Word2Vec and GloVe on the other hand, are geometric based, implicit constructions of word vectors in a continuous *latent space*. Vectors are learned through training on large corpora to capture semantic and syntactic relationships. These are dense**,** distributed representations where each dimension captures abstract relationships rather than direct counts. In essence, while TF-IDF and Count Vectorization build vectors through explicit algebraic transformations of text data, Word2Vec, FastText and GloVe embedding models use deep learning to geometrically place words into a latent vector space based on their usage (i.e. distribution) patterns in large corpora – with Word2Vec and FastText using local co-occurrence information to construct the vector space and GloVe using global context to construct the vector space. The text vectorization techniques yield clear, interpretable vectors (e.g., word counts), while the latter produces more abstract, dense representations.

It is fair to say that the introduction of word embeddings, an evolution and offshoot of vector space models, revolutionized language representation, allowing words to be represented as low-dimensional continuous vectors, but because they are static representations, i.e. are not context aware, they are relatively easy to work with from both a computational standpoint as well as from a language representation architecture standpoint. That is to say, given the clean mapping of a word to an embedding, one can enhance this structure relatively easily within the context of a neural network classifier by simply concatenating one embedding form with another, a technique we use in the ML baseline testing part of this study for example.[3] The static nature of these early language models, despite their ease of use in ML and DL contexts, nonetheless were constrained in terms of granularity by their static nature, i.e. they did not solve the *polysemy* problem[4]. This is a significant intellectual boundary with respect to the utility of these models for driving more advanced, more intelligent, NLP applications as much of the meaning of language is driven by context, and therefore a language representation form which encapsulates and encodes the contextual meaning of language is necessary for NLP applications to have a true complete "understanding", or semantic representation, of the underlying text.

This is precisely the motivation of the *attention mechanism*, which was introduced in in 2017 in the seminal paper "Attention is all you need" (Vaswani, et al., 2017), where what have become to be known as *transformers* were introduced that use more sophisticated, and more granular, tokenization strategies than their predecessors, as well take into account positional information of tokens, so that contextual information (token positional information effectively), can be used to generate *context-aware*, i.e. what we call *dynamic*, embeddings. It's important to recognize that the process of converting raw language text into a form that transformers can understand is itself a sophisticated process, but it is fundamental to the power inherent to transformer architecture language models (LLMs) and comes in different variants, depending upon the language model in question. Notably this process of transformation of text, how the language is *represented* in this case, is distinct from its predecessors in many respects, in particular in the encoding of positional information which facilitates the attention mechanism which is fundamental to the transformer architecture.

[3] For example, we concatenate and use other forms of linear algebraic transformations with basic text representation forms, LSA forms of data, and Word-Class Embedding structures as well, leveraging the code and test methodology introduced in (Moreo, et al., 2021).

[4] Polysemy in linguistics is a term that means that a given word could have multiple meanings depending upon how it is used – like "blue" for example which could be the color blue or represent the mood, with the specific meaning depending upon context

Transformers begin processing text by tokenizing raw textual input into token IDs using a tokenizer. Transformer tokenizers are sophisticated and fundamental to language representation in LLMs so important to understand within the context of this research.

The most prevalent types of tokenizers used with transformers are WordPiece, Byte-Pair Encoding, and SentencePiece tokenization, each of which is used by one or more language models that is tested with Layer Cake and each of which, like FastText's tokenization strategy, is a form of subword tokenization. WordPiece is a subword tokenization algorithm originally developed for Google's neural machine translation system (Schuster & Nakajima, 2012) (Wu, et al., 2016), and then used with BERT (Devlin, et al., 2019). This tokenization strategy begins with a base vocabulary of individual characters and then iteratively builds a vocabulary of subwords by merging the most frequent pairs of characters and/or subwords. It is designed to keep common words intact, while splitting rare or unknown words into smaller, meaningful subword units.

Byte Pair Encoding on the other hand, introduced in 2016 (Sennrich, et al., 2016) and used in the GPT class of language models, is another subword tokenization method which was initially used for data compression and was then later adapted for NLP by the SentencePiece and GPT/GPT-2 models. Byte Pair Encoding begins with characters as tokens and then repeatedly merges the most frequent pair of tokens (characters or subwords) into a new tokens, continuing until reaching a desired vocabulary size. SentencePiece tokenization, used by T5 (Raffel, et al., 2019), Albert (Lan, et al., 2019) and XLNet (Yang, et al., 2019) models, is a language-independent, unsupervised tokenization method introduced by Google in 2018 for neural text processing (Kudo & Richardson, 2018). Unlike traditional tokenizers, it does not rely on whitespace for tokenization, facilitating support for languages that do not have clear word boundaries (e.g. a whitespace), as is normal for all western European language, like for example Japanese or Chinese where a single symbol can denote a phrase and/or multiple words. The output tokens include a special symbol (typically '▁') to denote spaces, allowing it to reconstruct the original text without external tokenizers.

Regardless of the tokenization strategy however, token IDs are generated for each specific language model based upon its underlying tokenization strategy and training corpus (and other algorithmic or parametric factors) and serve to map individual tokens to *token embeddings*, which are word embeddings for all intents and purposes just mapped to tokens rather than words. These token embeddings, like their word embedding counterparts in fact, are high-dimensional vector (e.g., 768-dim) that encodes the tokens semantic information within the semantic space that was constructed during the language model's development, or pretraining. The overall language model then, the fixed embedding layer that sits at the base of the transformer neural architecture, is a large embedding matrix of shape [vocab_size, hidden_size] that is again learned during pretraining and then typically fine-tuned during downstream tasks as we do in our research here with Layer Cake. However, this is not sufficient to achieve context-aware embeddings, which are *the* fundamental characteristic of transformers.

In order to be truly context aware, the token embeddings must be coupled with additional information about the context within which the token is found, so that the transformer can use this information to generate *context-aware* embeddings. This is where *positional embeddings* come in, one of the most revolutionary parts of the original Attention paper (Vaswani, et al., 2017). There are two main types of positional embeddings that are used by transformers (1) *learned positional embeddings*, where each position (e.g., 0–511) has a trainable vector stored in a matrix of shape [max_position_embeddings, hidden_size], and (2) *sinusoidal encodings*, which use a deterministic function of sine and cosine waves to encode positions

with different frequencies (the approach used in the original 2017 paper). In both cases, the output is a vector of the same shape as the token embedding, allowing element-wise addition.

These positional embeddings then are added to the token embeddings to yield the final embeddings that are fed into the transformer, allowing the model to take into account positional information, i.e. context, when computing embeddings and facilitating the construction a semantic space which is much more nuanced, and much more granular, than the static word embedding language model predecessors that had no way to track or understand the context within which a given word, or token, is found in a given text or corpus. The final input embedding for each token is the sum of its token embedding and its corresponding positional embedding. This results in a matrix of shape [sequence_length, hidden_size], which is optionally passed through LayerNorm and Dropout before entering the first Transformer layer. This combined representation encodes both the *semantic meaning* of tokens and their relative positions, enabling the self-attention mechanism to reason about context and order.

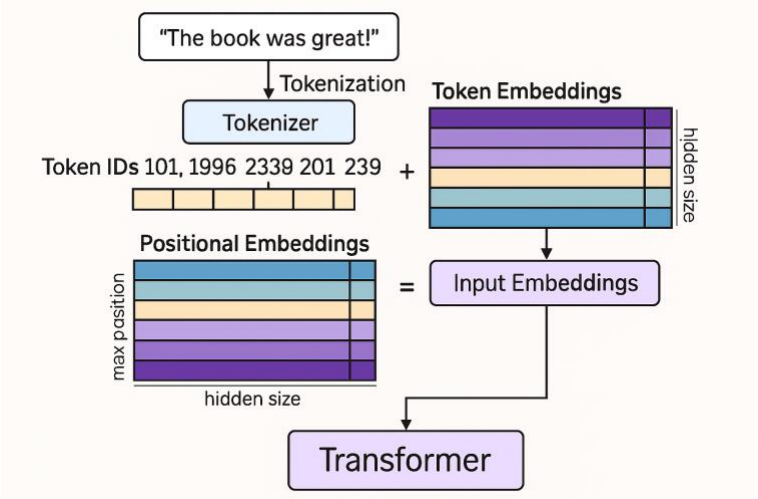


Figure 3: Token Embeddings + Input Embeddings with Transformers

Furthermore, some language models also include what are known as *segment embeddings*, which provide information about which sentence, and which part of which sentence, each token belongs to necessary for training tasks like next-sentence prediction which require such information. Models such as BERT (Devlin, et al., 2019), ALBERT (Lan, et al., 2019), XLNet (Yang, et al., 2019), and ELECTRA (Clark, et al., 2020) all leverage some form of segment embeddings, learnable vectors that differentiate tokens belonging to different input sentences within a single input sequence. In these models, each token in the input sequence receives an additional segment embedding corresponding to the sentence or segment it belongs to, along with its token embedding (encoding its semantic meaning), and positional embedding (encoding its position). These three embeddings then—token, positional, and segment—are combined (element-wise summation) to form the final input embeddings that are provided to the transformer layers of the model, enabling the encoding of multi-sentence inputs which facilitates training on tasks like Next Sentence Prediction (NSP), question-answering, and sentence-pair classification.

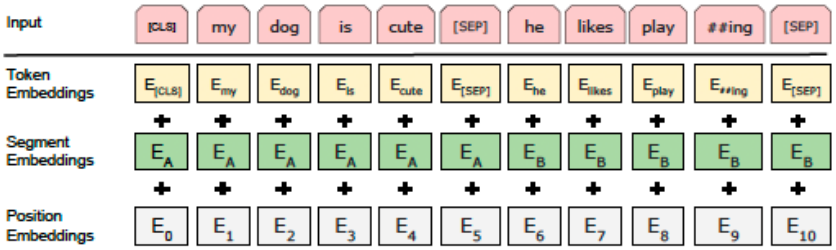


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

Figure 4: BERT input representation which includes segment embeddings (Devlin, et al., 2019)

It is the tokenization strategy, as well as the attention-based masking that transformers are typically known for, that combine together to produce the fine-grained powerful language models that have been the engine behind language generation tools and AI more broadly over the last 7 years. Some of the early influential deep learning approaches to NLP which leveraged transformer architectures and attention mechanisms to compute contextualized (dynamic) text representation forms of natural language were *ELMo* (Embeddings from Language Models), which was introduced in 2018 (Peters, et al., 2018) which used bidirectional LSTM layers to generate contextualized embeddings, and then of course *BERT* (Bidirectional Encoder Representations from Transformers), which was introduced in 2019 (Devlin, et al., 2019) which uses a unique, bidirectional approach to generating contextualized embeddings which, in both its original form and in many branches of derivation, is still a common tool used for a variety of NLP solves such as sentiment analysis, question answering, and named entity recognition.

Almost all the most popular and influential LLMs in use right now leverage this attention-based architecture which has replaced virtually all of the prior deep learning architectures that came before it (e.g. RNNs with self-attention mechanisms), underpinning many state-of-the-art results in machine translation, text classification (Devlin, et al., 2019), natural language understanding, and (natural) language generation tools (Radford, et al., 2019) and applications. These models, and their associated platforms and operational infrastructures, have pushed the boundaries of language generation and NLP more broadly, greatly enhancing AI capabilities and enabling the development of powerful NLP applications such as conversational agents, document summarization, and ultimately LLMs which show surprising versatility.

5.2 Text Classification, Deep Learning and Word-Class Embeddings

Text classification, the core NL problem that is analyzed and tested in this research is one of the foundational problems in Natural Language Processing (NLP) that has applications across the commercial and application landscape ranging from topic classification for news and other related research sites, to spam detection and email filtering, to content moderation and sentiment analysis, to fraud and malware detection in the cyber security space. The problem, as we frame it here, involves assigning predefined labels to textual data and then building and training models to predict how new text (documents) are to be classified based upon historical data that has been fed into the model, i.e. a form of *supervised training* which is a very common, and very effective, means of model training and construction.

Research related to Text Classification, one of the major themes of this work, follows a similar trajectory as language representation described above, with early solutions leveraging various forms of text vectorization like TF-IDF (Sparck Jones, 1972) and feature reduction techniques such as LSA/LSI (Deerwester, et al., 1990) used in tandem with classical Machine Learning models such as Support Vector Machines (Joachims, 1998), or simpler methods such as Logistic Regression or Naïve Bayes (McCallum & Nigam, 1998). A good summary of the various ML approaches to text classification can be found in the 2002 work by Fabrizio Sebastiani (Sebastiani, 2002), a paper which surveys different Machine Learning approaches to the problem of text classification before the proliferation of neural networks and deep learning techniques in NLP, also focusing on document (language) representation, classifier construction and evaluation as we do here.

With the advent of deep learning came the introduction of (static) word embeddings, as well as the introduction of various neural network architectures that were proven to be effective with text classification problems specifically, in particular work related to Convolutional Neural Networks (Yoon, 2014) (Zhang, et al., 2015), Long Short Term Memory networks (Zang, 2021), and Attention based neural networks (Liu & Guo, 2019), each of which is used as benchmark models to test static word embeddings in a deep learning setting following (Moreo, et al., 2021). Each of these neural models is tested with each of the static embedding language models in a variety of hyperparameter settings in the layer cake framework that is the subject of this work, and the models generally hold up well relative to the transformer and ML classifiers that they are compared against, taking considerably less computational resources than the transformer-based classifiers we test in this work in fact.

What closely followed this research, related to text classification in particular and which provided some of the (initial) motivation for this work in fact, was what is known as Word-Class Embeddings (WCE), which are effectively a pre-computed class vector (sized based on the number of classes) that maps each specific term, or word, of the dictionary of terms or words in scope for the given language model to all of the predefined class labels, calculated using a variety of techniques that range from the straight probabilistic to the use of informational theoretical primitives like pointwise mutual information (Moreo, et al., 2021). Confining the WCEs to their own vector space allows for them to be added to pretrained (static) word embeddings quite elegantly, and their work shows that WCEs do in fact significantly improve the performance of CNN, ATTN and LSTM based neural classifiers.[5]

The next stage of evolution for deep learning, embedding based classifiers is of course the transformer based models, which use multiple attention heads along with a more sophisticated representation of the underlying text which captures the specific context within which the token is found in order to generate context-aware, dynamic word embeddings that solve the polysemy as well as the out-of-vocabulary (OOV) problem in NLP and generally provide much greater granularity and specificity when navigating through the underlying semantic space which is bound by the dictionary of tokens for the given language model and is defined by the training algorithm and pre-trained data which the language model was trained on. Some of the earliest transformer models that were shown to be effective for text classification problems were BERT, which stands for *Bidirectional Encoder Representations from Transformers* which was released in 2019 (Devlin, et al., 2019), as well as derivations thereof like RoBERTa, or *Robustly Optimized BERT Pretraining Approach* which was released shortly after BERT also in 2019 (Liu, et al., 2019) as well as DistilBERT, a distilled

[5] We found however that these WCEs, which again are computed in their own vector space, did not meaningfully impact the performance of transformer based classifiers, and in fact appeared to decrease the effectiveness of these models regardless of how the WCEs (really token-class embeddings within the context of transformers, or TCEs) were added to the dynamically produced, context aware embeddings which ara a product of the various transformer based classifiers used in this work.

(Hinton, et al., 2015) variant of BERT that is computationally much faster than its predecessor but was shown to be, and confirmed in this study, just as effective (Sanh, et al., 2020).

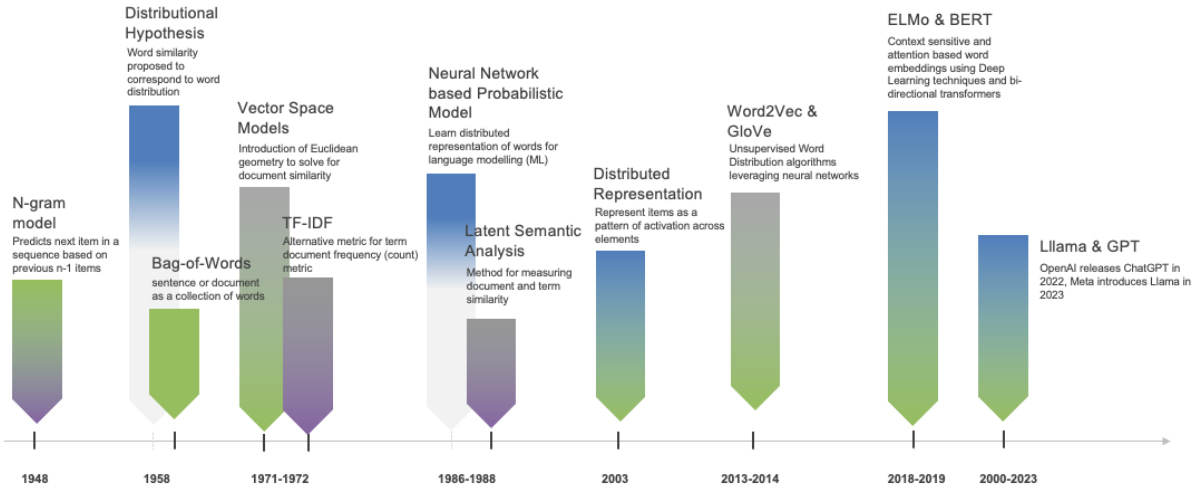


Figure 5: Language Representation Development Timeline & Milestones

These transformer models, each with its own nuanced training approach and transformer architecture, along with the GPT-3 model (Brown, et al., 2020), as well as an XLNet model (Yang, et al., 2019), along with smaller language models from both the Llama (Touvron, et al., 2023) and DeepSeek family (Bi, et al., 2024), were included in the benchmarking that was performed in this research, each of which was tested across a variety of classifiers and each of which was shown to be effective for text classification, although the computational requirements of the transformer models significantly surpassed their static neural network counterparts, metrics and insights that are unique to this study.

There have been a variety of good research surveys written that look at the problem of text classification within the context of different DL architectures and/or LLMs, for example relatively early studies looking at pre-trained language models for NLP more broadly in 2020, (Qiu, et al., 2020), another focused on deep learning approaches for text classification specifically (Minaee, et al., 2021), and another very thorough study that looks at the effectiveness of various optimization techniques and hyperparameter tuning applied to the text classification problem focused exclusively on LLMs published in late 2024 (Trust & Minghim, 2024). These studies do not include classical ML approaches to text classification in their work however. While there do exist some notable surveys related to text classification that include both ML and DL approaches, like for example (Li, et al., 2022), these studies are dated and do not account for the increased size and compute requirements for the more modern LLMs, properties that are becoming of increasing importance given the need for, and expense and scarcity of, GPU clusters that are necessary to both build, train and fine-tune such models.

With Layer Cake we look to address this gap in the research by comparing the classical ML approaches, which are CPU bound, alongside various DL approaches which include LLMs as well as less computationally expensive language models which are not transformer based such as Word2Vec, FastText and GloVe within the context of text classification, directly on the same system with the same number of GPUs, shedding light on the comparative computational resource requirements for each of the types of classifiers and language models directly.

5.3 Unique Contributions of this Work

This work focuses on the underlying forms of language representation and associated techniques, or classifiers, which have proven to be effective for text classification problems, covering both ML and DL methods, showing that static word embeddings, when combined with WCEs, can be just as effective as transformer based approaches to text classification in many cases, with a fraction of the computational resource requirements (time to build the model, given same model and number of GPUs). We also show that many classical ML techniques, also when combined with WCEs, can also be just as effective as many transformer and DL methods of classification, both results (and analysis) that distinguish this work from prior research. While these results are consistent with (Moreo, et al., 2021), we have expanded the footprint of the research to include additional language models which include a variety of transformers which were not included in the original study.

Furthermore, while ML classifiers typically depend on text vectorization techniques for language representation, we show that LSA as well as various forms of embedding representations also perform competitively in an ML classifier setting, both reproducing and expanding (baseline) results from (Moreo, et al., 2021), adding both fastText and Word2Vec language models into the analysis for comparison purposes. The Deep Learning classifiers we test with on the other hand, which require fundamentally different forms of language representation than their ML counterparts, and come in word, subword, and token based variants (embeddings), we show both the power and effectiveness of these models, as well as some of their constraints, for example with respect to the use of TCEs, illustrating both some of the architectural drawbacks of modern transformer based classifiers as well as their inherent effectiveness, without TCEs, as it relates to the problem at hand. Also with respect to the ML testing and analysis part of this research, we include a broad spectrum of language representation forms into the models to test their relative efficacy, an aspect of model design that is oft overlooked in the text classification and NLP research literature, especially given that LLMs presume a fixed structure of language representation that is necessary for the models to work whereas the ML models have more flexibility with how the model is prepared with data, i.e. how the text is represented.[6]

We look at these results not only through the lens of the different classifiers and language models, but also track, report and analyze the requisite computational resource requirements necessary (GPUs, CPUs) for the different types of models to be constructed, creating perhaps for the first time a sort of level playing field through which ML and DL approaches, as well as different forms of language representation, can be evaluated in comparison with one another both in terms of computational resource requirements and model effectiveness. It is with this unique multi-variant perspective that we provide a framework, i.e. Layer Cake, through which the true performance, and inherent cost, of a given classifier for a given language model can be evaluated, a perspective that is lacking in vast majority of NLP and LLM research that has been published as of late which leaves out the cost-benefit analysis factor for different models, an aspect of analysis which is critical for properly evaluating possible approaches given the size of many of the most

[6] With LLMs, and in fact with almost all neural models in NLP, the underling (natural) language is converted into a fixed size (max_width) sequence of token indices where each token is represented by a vector of real numbers of a size fixed by the language model (aka hidden_size) that is the result of the pretraining of that language model against a fixed pretraining dataset – i.e. what are known as *embeddings* but which are extensions of Vector Space Models which have been around since the 1970s. Each LLM creates these embeddings by defining the underling vector space dimensions (as a form of feature reduction from the total dictionary size of the pretrained corpus) and then learning the embeddings for each token using the language model type and variant specific (neural) architecture and algorithm (loss function, etc.).

performant LLMs on the market as well as the soaring costs of GPUs as they get bigger and faster to accommodate the computational requirements for training, and fine-tuning, modern LLM variants.

While there is much published research related to state of art results in NLP associated with a veritable potpourri of LLM architectures, with this work we introduce a much broader analytical and comparative perspective to the problem, focusing on different classifiers from both the ML and DL research domains and therefore different computing architectures, tested in with a broad range of language model types, each of which are constructed with their own specific training methodology and tokenization strategies. That broader perspective allows for a much more comprehensive and holistic analysis of the different approaches to text classification through which the performance of LLMs themselves can be viewed and by which the most important levers in the entire classification problem domain can be viewed.

Given the focus on LLM performance across the problem landscape in most research in NLP the last few years, this recentering of the analysis around different language representation forms as both inputs to the classifier models (embedding layers) and as outputs of different DL architectures (i.e. embeddings), allows for an analysis of the results from the perspective of an optimization problem in high dimensional semantic space, a semantic space that is defined by the dataset representation form and the language model representation form – a combination of the two really – which is precisely what is going on mathematically within the context of the text classification use case that is the subject of this work. Some of these results therefore, due to their abstract mathematical and geometrical foundations, should be applicable to other NLP problems that leverage text vectorization, embedding or transformer-based language representation modes – which essentially describes the entire NLP landscape.

Furthermore, in the testing of different neural classifier variants to support both static language models as well as transformer, context aware models, we discovered a significant difference in the flexibility of the different neural architectural models, with the transformer embedding layer being tightly coupled with its computational logic (as one might expect perhaps), which significantly limits the ability to optimize the model – like for example using feature, or dictionary size, reduction techniques as we do for the other classifiers. This constraint, limits how and where in model training one can add pretrained embedding structures that are extra-model or are trained outside the transformer language model context that we are using for the task at hand. In our testing for example, we could not find an approach where the integration of Token-Class Embeddings, or TCEs (the transformer variant of WCEs), actually increased the effectiveness of the model, in fact we found the opposite, that adding these TCEs with the context-aware embeddings during training - no matter how we added them or how we normalized the underlying embedding dimensions - actually made the model less effective.

Also, in addition to this unique perspective and unique set of findings, we also produce within the context of this work not only the source code for the Layer Cake benchmarking platform itself, but also a rich set of results from Layer Cake, covering a broad spectrum of metrics[7] that is readily available for further analysis, contributing to the research community a cross-platform, extensible, benchmarking platform that supports a variety of language models, classifiers and datasets that can be extended for further insights into this field

[7] We report macro-f1, micro-f1, precision, recall, accuracy, Jacard-Index and Hamming loss metrics for each test run.

of research, i.e. text classification, or even extended to support other NLP problems like sentiment classification, topic modeling or even Natural Language Understanding (NLU) benchmarking if need be.[8]

5.4 Structure of this Paper

The rest of this paper follows the following basic outline:

- description of test setup and scope, followed by
- detailed description of the test workflow, forms of language representation and classifiers used in testing, followed by
- detailed analysis of the data which looks at the ML and DL approached both independently and then together globally, and then
- a summary analysis which includes recommended areas of further research, lastly
- a citation list (Bibliography).

The analysis parts of this paper are dominated by charts and graphs that visualize the results according to the key dimensions that represent the major themes of this work – namely classifier architecture, language representation form, and training time and computational complexity.

[8] Layer Cake supports Mac MPS GPU architectures as well as Ubuntu Linux GPU architectures with the exception being the DeepSeek models which are not supported (yet) on MacOS. Source available on GitHub at Source code for Layer Cake available at https://github.com/pworth1971/layer-cake

6 Experimental Design: Test Scope & Data Pipelines

Layer Cake can be viewed as an expanded NLP pipeline that includes support for multiple datasets, multiple forms of language representation, along with support for multiple classifiers from both classical Machine Learning as well as more advanced Deep Learning architectures. The nature of the different classifiers, given how they optimize and solve for the text classification problem in question, determines the forms of language representation that can be used for that given classifier as well as the data preprocessing steps necessary to configure such language representation forms. The overall dataflow and modular architecture of Layer Cake is depicted in the figure below. While our test methodology generally follows (Moreo, et al., 2021), we extend the footprint significantly in terms of the ML classifiers and language models we test, the forms of language representation that we use for the ML testing, and the extension of the testing framework into transformers. All of the code is available on GitHub for further research or the reproduction of results, with both MacOS (MPS) and Ubuntu Linux (CUDA) systems supported.[9]

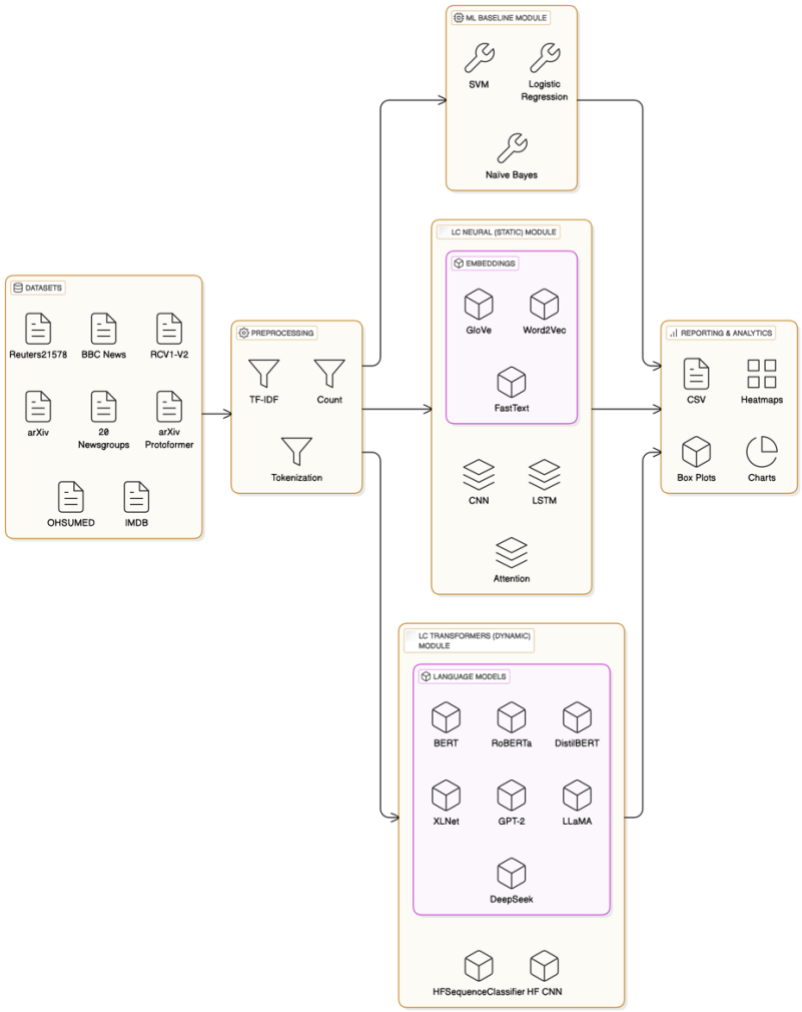


Figure 6: Layer Cake Classifier Testing Architecture

[9] https://github.com/pworth1971/layer-cake

Given the scope of Layer Cake, and the NLP pipeline dependencies that are characteristic of the different types of classifiers it supports as mentioned above, Layer Cake is split into three different modules for testing:

i. *ML Testing*: A script for ML testing which supports Support Vector Machines, Logistic Regression and Naïve Bayes classifiers[10],

ii. *Static DL Testing*: A script DL testing that is designed to incorporate and test *static* word embeddings (language models) which include Word2Vec, fastText and GloVe embeddings, and test them in a CNN, ATTN and LSTM neural architectures, and

iii. *Dynamic DL Testing (transformers)*: a second DL test script which covers transformers models (aka *dynamic*, or *context-aware* embeddings) testing script which uses the HuggingFace libraries to test BERT, RoBERTa, DistilBERT, GPT-3, XLNet, Llama and DeepSeek language model variants in both a custom CNN (following the CNN architecture in ii) as well built in Sequence Classifier[11]

Each of the test modules are run with various hyperparameter settings and configurations, and each of these modules include support for multiple classifiers, furthermore each classifier is tested with different language models and datasets and hyperparameters, and in addition the ML classifiers are all tested with a a variety of language representation forms along with the different language models.

All metrics and analytics for every test run are written to a log database (a tab delimited text file), which supports a variety of metrics (see below) and is used for generating the summary data, analysis, charts and plots that are included in this work. The log data includes information about classifier construction or build times, the various hyperparameter settings that are used, the underlying CPU and GPU architecture, the underlying OS and memory characteristics, as well as the version of the code that is executed.

The ML baseline testing module (ml_classification_test_version.py) module covers the baseline machine learning classifiers that we test with, namely Support Vector Machines and Logistic Regression and Naïve Bayes classifiers. We use the *scikit-learn* libraries[12] to build and run the respective classifiers, namely the *LinearSVC*, *LogisticRegression*, and *MultinomialNB*, using a *OneVsRestClassifier* approach for the multi-label datasets. This code is CPU bound for the most part, although the transformer embedding computations of the different dataset and language model combinations are GPU bound. This code ran on an Ubuntu Linux system with a single GPU system with the same system specification as described above.

The final set of test runs for the Machine Learning classifiers were run on a 2023 MacBook Pro with an Apple M3 chip and 128 GB of RAM.[13] These machines have their own GPU, which Apple calls a Metal Performance Shader, or simply MPS, which is optimized for both graphics and compute (i.e. Deep Learning). PyTorch has support for this environment (MPS), and we leverage this extensively for the language representation construction code for the different language models that we feed into the ML classifiers for example. All

[10] The Naïve Bayes classifiers have some restrictions given the dependency on positive numbers, some of the language model embeddings do not satisfy this requirement and so are not run through this classifier.
[11] HuggingFace's transformers api includes pre-built sequence classifiers as part of its library.
[12] https://scikit-learn.org/stable/
[13] The Deep Seek model we tested does not support MacOS so that language model is not included in our ML testing.

code is designed to work on both the MacOS as well as Ubuntu Linux (22.04 x86) and has been tested extensively on both platforms.[14]

The neural model classifiers that are designed for the static word embedding language models - Word2Vec, fastText and GloVe - are GPU bound (CUDA), although they also run on MPS / Apple Silicon. The neural classifiers used for this testing are crafted using a very thin and optimized PyTorch design, and we retain support for CNN, ATTN and LSTM based neural classifiers, following (Moreo, et al., 2021). These classifiers do not support parallelism and are designed to run on a single GPU.

The transformer language models on the other hand are integrated and tested with a separate python script that leverages the Hugging Face APIs and supports a CNN classifier that is modelled after the CNN architecture we use to test the static word embedding language models, as well as a Hugging Face built in Sequence Classifier that supports the various transformer language models that are in scope for Layer Cake. The transformer classifier test script does support a simple form of parallelism (for PyTorch) known as *Data Parallelism*, where the model is replicated on all of the in-scope GPUs, data is doled out equally across the GPUs during training, in parallel, and then model updates are synchronized after each training step.[15]

Both of the Deep Learning test modules were run on a Linux Ubuntu system using CUDA libraries, with the exception of the RCV1 dataset which, given its size, was tested on the MacOS test system outlined above. The Ubuntu variant that we ran on had the following system specifications:

- OS: Ubuntu Linux version 22.04, x86 architecture
- CPUs: 64 physical, 128 logical; 512 GB RAM (memory)
- GPUs: 4 NVIDIA RTX 6000 Ada Generation, 48GB of Memory each
- CUDA 12.4
- Python 3.10 (DeepSeek model dependencies)
    - PyTorch 1.4.1
    - Transformers 1.45.2
    - Sklearn 1.6.1

One of the important insights that this research sheds light on is the impact of, and relationship between, the requisite form of language representation for each of the classifiers we test with. For example, when working with the transformer classifiers, the first set of layers in the model (Embedding layers) are automatically set by the underlying transformer model itself and cannot be modified or extended in any way without breaking the model, whereas the DL classifiers we use for the static language models (which come in CNN, ATTN and LSTM variants) have more flexibility with how this Embedding layer is set up given that the embeddings are static and are not modified during model training. Furthermore, the ML classifiers, given their more straightforward mathematical design, have significantly more flexibility with respect to how the underlying data can be represented, hence the ability to test various forms of language representation in this context.

[14] While not as performant as the CUDA libraries on the Ubuntu Linux environment, it's not clear whether or not this is a hardware, or a software constraint given that CUDA support and capabilities remain the industry standard and clearly outperform MPS capabilities both in terms of function as well as performance.

[15] For a more detailed treatment of parallelism using PyTorch and/or Hugging Face we suggest https://huggingface.co/docs/transformers/v4.13.0/en/parallelism.

We see these model constraints born out for example in our (failed) attempts at integrating WCEs (really TCEs, or Token Class Embeddings) into the transformer models. This was in fact the original motivation for this work was to test the efficacy of integrating varying types of embeddings that were generated outside of the model within the context of text classification problem, work which is the very crux of the (Moreo, et al., 2021) study. Given the constraints of the underlying Hugging Face transformer models however, as well as the tight coupling between the initial embedding layer and the run-time token embedding computation i.e. the context-sensitive, attention mechanism which is arguably the fundamental characteristic of transformers, the integration of TCEs into the model actually *detracted* from the model performance, a result that was both unexpected and also telling in terms of the integration capabilities of transformers as they are currently architected as a self-contained system.

Furthermore, these classifier architectural differences not only drive different approaches to language representation, but also have significant performance implications, a fact that is well known but not so well quantified in terms of cost-benefit, one of the unique contributions of this work. Quantifying the computational cost, both in terms of types and quantities of GPUs required to train the underlying model (none in the case of ML classifiers) as well as overall classifier model training times, is one of the unique contributions of this work and has implications well beyond just problems of text classification.

What is important to understand with respect to the underlying computational requirements of the different LC modules, especially within the context of the model construction time analysis below, is that:

i. the ML testing is CPU bound,

ii. the neural model testing is all GPU bound, single GPU, and

iii. the transformer model testing supports GPU parallelism (4 GPUs)

While the DL classifiers for the static word embeddings does not support parallelism, i.e. it runs on a single GPU, the transformer based pretrained language model testing on the other hand, does in fact support (the most basic form of PyTorch) parallelism.[16] In other words, our transformer models used a 4 GPU environment whereas our ML baseline script, and our static pretrained neural model classifiers, only depend on a single GPU, an important distinction we will call out in the analysis section below when we look at model training and testing times.

6.1 Supported Language Models

Layer Cake supports a wide array of language models variety of models which gives us an opportunity to compare different forms of language representation in different contexts and look at how, and what, a language model was trained on, and to what extent those factors bear on classifier effectiveness as defined by the various measures that Layer Cake supports, although we primarily focus on the Macro-F1 and Micro-F1 measures, following (Moreo, et al., 2021), which provide good holistic representations of classifier performance.

[16] We made several attempts at getting Word-Class Embeddings, really their tokenized counterparts, integrated with various HF Classifiers, these efforts were met with results that indicated quite clearly – regardless of the method of integration – that at least class level embeddings could not be effectively integrated into the transformer architected model.

6.1.1 Static Embedding Models

For the static language models that we tested, whose embeddings are word based as opposed to token based as the transformer models are, we used a GloVe model, a Word2Vec model and a FastText model, each of which was trained on the Common Crawl dataset which consists of a massive, open source web archive that consists of petabytes of data collected by crawling the web. The Common Crawl dataset contains 600+ billion tokens, making it one of the largest public corpora available for training NLP models. The dataset is multilingual, with content spanning hundreds of languages and while English is dominant, significant amounts of content are curated for other major languages like Spanish, French, and Chinese.[17]

6.1.1.1 GloVe: glove.840B.300d.txt

glove.840B.300d.txt is the GloVe pretrained word embeddings set that was trained based on global co-occurrence statistics against the common (web) crawl dataset which includes 840B tokens and 2.2 million vocabulary words, is of dimension (vector size) 300, and around 2GB all in. The GloVe specific training method focuses on co-occurrence statistics from a word-context matrix across the entire corpus, capturing both syntactic and semantic relationships and was introduced in 2014 (Pennington, et al., 2014).[18]

6.1.1.2 Word2Vec: GoogleNews-vectors-negative300.bin

GoogleNews-vectors-negative300.bin is the Word2Vec trained static word embeddings, originally trained by Google, which also uses a vector space of 300 dimensions, and contains over 3 million words and phrases and is all in about 1.5 GB. The embeddings were trained from the Google News dataset, hence the name, and was built using the Word2Vec model, introduced in 2013 by (Mikolov, et al., 2013). Specifically, the model is trained using the Continuous Bag of Words (CBOW) and skip-gram methods, where the skip-gram model predicts surrounding context words given a center word, and negative sampling is used to optimize training by selecting a few "noise" words during prediction instead of updating the entire vocabulary.

6.1.1.3 FastText: crawl-300d-2M.vec.bin

crawl-300d-2M.vec.bin is the binary format of the FastText static embedding variant we test with that is also of size 300 (dimensions of embedding vector), contains around 2M unique words, about 6GB all in and was also trained on the Common Crawl dataset, preprocessed to remove noisy content, duplicates, and non-linguistic data (e.g., scripts, images, ads). This fastText language model was trained using a skip-gram method with negative sampling, trained on word context, and incorporating subword modeling (n-grams). FastText splits each word into character n-grams and builds word representations by summing subword

[17] The data is sourced from a wide range of websites, covering topics such as news articles, blogs, social media, scientific articles, forums and community sites and technical documentation, representation a broad range of data that supports the training of language models that cover the broad scope of human digital) language. The main goal of the Common Crawl project is to provide an open-access, large-scale web dataset for academic, industrial, and research use. Common Crawl data is publicly available and constantly updated through periodic crawls. Source: https://commoncrawl.org/

[18] https://nlp.stanford.edu/projects/glove/

vectors which enables it to handle out-of-vocabulary (OOV) words by generating approximate embeddings based on their subword components. The model we use lacks the subword information but is trained to that effect. The fastText embeddings were introduced in 2017 by the Facebook (now Meta) AI team (Bojanowski, et al., 2017)

6.1.2 Context-Aware Language Models (Transformers or LLMs)

For the transformer tests, we use three different BERT variants, XLNet and GPT2, as well as a small LLaMA model and a small, distilled DeepSeek model, each of these models is briefly described below.

6.1.2.1 BERT, RoBERTa and DistilBERT

BERT (Devlin, et al., 2019) and RoBERTa (Liu, et al., 2019) are pretrained on massive amounts of text data using a masked language modeling (MLM) approach where some tokens in a sentence are randomly masked out, and the model is trained to predict the original tokens at those masked positions using the appropriate context (i.e., surrounding words). BERT also leverages next sentence prediction (NSP), which amounts to masking of parts of sentences during pretraining, and the model is trained to predict the missing words. MLM introduces probabilistic predictions into the model's learning process. The embeddings that result from MLM are optimized for downstream tasks, such as classification, named entity recognition, or question answering.

BERT considers the entire context of a word from both directions (left-to-right and right-to-left) using the self-attention mechanism. This means that the embedding for a word in BERT is influenced by both the preceding and succeeding words in a sentence, making the embeddings contextual and *bidirectional*. RoBERTa builds on the BERT model but removes the next sentence prediction (NSP) task and introduces changes to pretraining hyperparameters, such as training with larger batch sizes and more data. This leads to better generalization performance and our tests bear that out.

In the embedding space, BERT and RoBERTa learn to position words in a way that encodes both syntactic and semantic relationships, but these relationships are deeply tied to the specific context. In BERT/RoBERTa's embedding space, the distance between words is not just based on their isolated meaning but on how they interact with other words in a given sentence or passage. Thus, the same word can have multiple geometric positions depending on the sentence in which it appears. BERT and RoBERTa embeddings are contextual, probabilistic, and dynamic.

DistilBERT is another BERT language model variant that we use in Layer Cake that was pretrained on the same corpus as BERT but, using a technique called *knowledge distillation* (Hinton, et al., 2015) which is when a model uses a source model as a target (aka teacher model) and learns to mimic its outputs while not requiring the full training computational resources, yielding a smaller model that is comparable in performance. More specifically from a distillation perspective, DistilBERT uses a masking and cosine embedding loss algorithm to generate hidden states as close as possible as the BERT base model while again not requiring the full ensemble of training and parameters as its teacher model, yielding a model that is around 40% smaller than BERT and yet still retains some 97% of its performance across a variety of NLP and NLU benchmarks making it an ideal model to use on edge network devices who do not have access to a lot of computing power for example (Sanh, et al., 2020).

6.1.2.2 XLNet

XLNet is another transformer-based model that we use in Layer Cake that incorporates aspects of both encoder and decoder architectures but functions more like a generalized autoregressive model designed for bidirectional context understanding (Yang, et al., 2019). XLNet incorporates ideas from Transformer-XL (Dai, et al., 2019), the state-of-the-art autoregressive model, into the pretraining objective of BERT by maximizing the expected likelihood over all permutations of the input sequence tokens.

XLNet takes a hybrid approach to training, leveraging advantages of both BERT (bidirectional encoding) and GPT (autoregressive generation). XLNet is neither strictly encoder-only nor decoder-only, instead, it is built on the Transformer-XL framework, which uses a permutation-based autoregressive training objective that enables it to model bidirectional context without the need for masking (as in BERT). While it primarily focuses on understanding text (like encoder-only models), its architecture and training objectives are inspired by decoder-based autoregressive models.

XLNet predicts tokens based on previous tokens but does so in a permuted order. This allows the model to capture bidirectional context in a flexible way without relying on masking like BERT. Unlike BERT's Masked Language Modeling (MLM), XLNet models all positions of the sequence in random permutations during training, making it a generalized autoregressive model. XLNet extends Transformer-XL, which introduces a memory mechanism to learn dependencies over long sequences. This allows XLNet to handle longer contexts than standard Transformers like BERT.

6.1.2.3 GPT-2 (Generative Pretrained Transformer)

GPT-2 (Generative Pretrained Transformer 2) is an unsupervised, *auto-regressive*, language model from OpenAI introduced in 2020 in the paper "*Language Models are Few-Shot Learners.*" (Brown, et al., 2020). GPT2 is an English language model also uses the Transformer architecture, decoder only, leveraging a *causal*, or *auto-regressive* approach to training where tokens are predicted sequentially and are conditioned on previously conditioned tokens.

More precisely, inputs are sequences of continuous text of a certain length and the targets are the same sequence, shifted one token (word or piece of word) to the right. The model uses internally a mask-mechanism to make sure the predictions for the token *i* only uses the inputs from 1 to *i* but not the future tokens, allowing the model to construct an inner representation of the English language that can then be used to extract features useful for downstream tasks.[19]

GPT-2 is designed to generate coherent, human-like text when given a prompt, i.e. *language generation*, but it can be fine-tuned for a wide range of downstream NLP tasks such as text completion, summarization, translation, and question answering and provides a good contrast model to the bidirectional, masked BERT models we test with, as well as a comparison to the other language generation models that we use in in Layer Cake (Llama and DeepSeek, more on these models below).

[19] https://huggingface.co/openai-community/gpt2

6.1.2.4 LLaMA (Large Language Model Meta AI)

Llama, or *Large Language Model Meta AI*, is the name for a family of open-source language models created by Meta (formerly Facebook), that was introduced in 2023 in the paper "LLaMA: Open and Efficient Foundation Language Models" (Touvron, et al., 2023). These are multi-lingual models whose tuned versions use supervised fine-tuning techniques (SFT) and reinforcement learning with human feedback (RLHF) to align utility goals with safety and security objectives.[20]

LLaMA models are - similar to GPT2 - transformer based, decoder only, and trained auto-regressively. Like GPT2 they are also designed for language generation tasks primarily, i.e. what the market has come to understand and call LLMs. Its transformer-based architecture with decoder layers is optimized for scaling and it uses sparse attention mechanisms to handle longer documents effectively, focusing on dense token prediction during its auto-regressive token generation scheme. Given its underlying architecture and training design, the model is best suited for General-purpose tasks, including long-form generation, summarization, and document retrieval.

6.1.2.5 DeepSeek

DeepSeek is a Chinese company that has recently released a few families of language models that have made a significant impact on the overall LLM landscape given the efficiency gains in training that they have gained along with the effectiveness of their models according to a variety of NLU and NLP benchmarks. DeepSeek LLMs were originally introduced in 2024 (Bi, et al., 2024) but have had a remarkable influence on the LLM landscape with their introduction of reasoning models in early 2025 (Guo, et al., 2025). The latest DeepSeek models use large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) (Ouyang, et al., 2022) as a preliminary step and have demonstrated remarkable performance on reasoning that are comparable to LLaMA and OpenAI models.

These DeepSeek models are transformer based with both encoder and decoder aspects, and the R1 family in particular is designed for *reasoning* capabilities, a relatively new capability that is being actively worked on in the LLM community which allows the model to break tasks into components and solve problems sequentially and with dependencies, what is known as *chain-of-thought* (COT), a revolutionary capability that was introduced in 2022 (Wei, et al., 2022) and has now become widely adopted to support advanced reasoning capabilities in LLMs.

DeepSeek has also produced several LLaMA and Qwen[21] based distilled models (Hinton, et al., 2015), the latter of which (smallest version) we incorporate into this this study, which remain effective for a variety of NLP and NLU tasks (as we see here) and are computationally efficient as well given the distillation-based design.

6.1.2.6 Summary Table

[20] https://huggingface.co/meta-llama/Llama-3.2-1B
[21] Qwen is an open-source family of language models developed by Alibaba Cloud in 2023 in the paper (Bai, 2023)

The specific language models we use for Layer Cake testing along with details on their underlying architecture, what they are (pre) trained on, and other salient information, are summarized in the figure / table below.

Arch	LM Type	Model	Architecture	Dimensions	Transformer Layers (Blocks)	Attention Heads	Case Sensitivity	Training Dataset	Training Objective	Best Use Cases
Transformer-Encoder	BERT	bert-base-cased	Transformer-Encoder	768	12	12	Case-sensitive	BooksCorpus + Wikipedia	Masked Language Model (MLM) + Next Sentence Prediction (NSP)	Text classification, sentiment analysis, NER
Transformer-Encoder	RoBERTa	roberta-base	Transformer-Encoder	768	12	12	Case-insensitive	Common Crawl (160GB)	Masked Language Model (no NSP)	Long-text understanding, summarization
Transformer-Encoder	DistilBERT	distilbert-base-cased	Transformer-Encoder	768	6	12	Case-sensitive	Distilled from BERT base	MLM (with distillation)	Lightweight Q&A, mobile applications
Permutation Transformer	XLNet	xlnet-base-cased	Permutation Transformer	768	12	12	Case-sensitive	BooksCorpus + Wikipedia	Permutation-based Training (Autoregressive, but sees bidirectional context)	Text classification, document ranking
Transformer-Decoder (Causal Autoregressive)	GPT-2	gpt2	Transformer-Decoder	768	12	12	Case-insensitive	WebText (45GB web corpus)	Causal Language Modeling (strict left-to-right)	Text generation, conversational AI
Transformer-Decoder (Causal Autoregressive)	LLaMA	LLaMA 3.2-1B	Transformer-Decoder	768	12	12	Case-sensitive	Common Crawl + academic corpora	Causal Language Modeling	Conversational AI, multilingual tasks
Transformer-Decoder (Causal Autoregressive)	DeepSeek-Qwen	DeepSeek-R1-Distill-Qwen-1.5B	Transformer-Decoder	1,536	24	16	Unknown	Fine-tuned Qwen base models	Causal Language Modeling	Multi-modal AI, summarization, retrieval
Static Word Embeddings	GloVe	840B.300d	None (Word Vectors)	300	N/A	N/A	Case-insensitive	Common Crawl (840B tokens)	Co-occurrence Matrix Factorization	Word similarity, information retrieval
Static Word Embeddings	FastText	common-crawl 300d	None (Word Vectors)	300	N/A	N/A	Case-insensitive	Common Crawl (600B tokens)	Subword-based embeddings	OOV handling, text classification
Static Word Embeddings	Word2Vec	GoogleNews 300d	None (Word Vectors)	300	N/A	N/A	Case-insensitive	Google News (100B words)	Continuous Bag of Words (CBOW) / Skip-gram	Word similarity, clustering

Figure 7: Language Model Summary Table

6.2 Supported Datasets

For each of the models, we prepare the underlying dataset as needed, create the representation form of the dataset - which is represented as lists or arrays of documents in said representation form – and then we prepare the model (classifier) with the underlying dataset representation, build the model using training and validation data, and then test the model and report results back based upon the model performance

against the test data that we set aside – following classical ML and DL model building and testing procedures effectively.

The datasets we use for testing come from a variety of sources, covering news and academic content as well as medical data and reviews.[22] We look at both single-label classification datasets, the simpler form where a document in a dataset is classified to a single category, and then we look at multi-label classification datasets where a document can belong to multiple categories. We also have one binary classification dataset (IMDB) which is affectively a sentiment analysis labelling scenario.

A brief description of each dataset we used in our different test scripts is provided below.

6.2.1 Reuters21578: multi-label, 115 classes

The Reuters-21578 dataset (Lewis, 1987) is a cornerstone in text categorization research, comprising 21,578 news documents from the Reuters newswire in 1987 originally collected and labeled by Carnegie Group, Inc. and Reuters, Ltd. The dataset is multi-label and each document is labeled across 115 different categories, and although not very well balanced across categories, nonetheless it is one of the standard benchmarks for multi-label text classification tasks. Researchers have extensively utilized this dataset to evaluate various machine learning and natural language processing algorithms.

6.2.2 BBC News: single-label, 5 classes

The BBC News dataset (Greene & Cunningham, 2006) contains 2,225 news articles from the BBC website, categorized into five distinct topics: business, entertainment, politics, sport, and tech. This single-label, balanced dataset is particularly useful for testing text classification models given that it is relatively small in size, single-label and well balanced across labels.

6.2.3 RCV1-V2: multi-label, 103 classes

The Reuters Corpus Volume I, aka RCV1-v2 (Lewis, et al., 2004), is an extensive dataset comprising over 800,000 newswire stories from Reuters, covering the period between August 20, 1996, and August 19, 1997. This multi-label dataset spans 103 categories, providing a rich resource for large-scale text classification and clustering research. Its size and complexity make it ideal for evaluating algorithms that need to handle large, real-world datasets with overlapping categories. It is news data and there is a lot of overlap between classes as one might expect from a news site.

6.2.4 20 Newsgroups: single-label, 20 classes

[22] Details on the different in scope datasets can be found in the Appendix

The 20 Newsgroups dataset (Lang, 1995) is a collection of approximately 20,000 newsgroup documents, partitioned across 20 different newsgroups. Each document is labeled with a single category, and the dataset is well-balanced, making it ideal for testing out different classifiers and as such has been a popular benchmark dataset for evaluating text classification algorithms.

6.2.5 arXiv: multi-label, 58 classes

The arXiv dataset, introduced with (Sinha, et al., 2015), consists of research papers from the arXiv repository, categorized into various scientific disciplines. The dataset is multi-label, articles can belong to more than one class (but at least one), and there are 58 class labels. Given the nature of the content, there are many scientific words or phrases (and some special characters) that must be managed to yield good model performance results and as such makes it a good test dataset to evaluate how well models handle domain-specific terminology and semantics.

6.2.6 arXiv protoformer: single-label, 10 classes

The arXiv protoformer dataset introduced in 2022 (Farhangi, et al., 2022) for testing transformer performance for text classification, is a curated version of the arXiv dataset, often used in experiments with prototype-based models (e.g., Protoformer). The set we use is filtered for the most common classes (just 10) and is single label. This is a simplified variant of the full arXiv dataset, which is focused on distinct topics, avoiding overlapping categories.

6.2.7 OHSUMED: multi-label, 23 classes

The OHSUMED dataset (Sanderson, 2010) is a classic benchmark in medical text classification and information retrieval. It has been widely used for evaluating models on multi-label and hierarchical classification tasks. The dataset consists of medical abstracts from the Medline database, labeled with MeSH (Medical Subject Headings) terms. It is a multi-label test set, consisting of 23 distinct classes. It is focused on medical topics, making it highly domain-specific, meaning that many of the words and terms would not be found in a typical language model training set. The specialized vocabulary presents challenges for text classification models, particularly in handling out-of-vocabulary terms and understanding complex medical jargon.

6.2.8 IMDB: single-label, 2 classes

The IMDB dataset (Maas, et al., 2011) is one of the most common datasets used in Machine Learning and Deep learning research used for sentiment analysis and (binary) classification problems. It is the only binary classification dataset we use in this study. The IMDB dataset consists of movie reviews from IMDB, each labeled with the sentiment (positive or negative) expressed in the review, making it ideal for supervised training methods. The dataset is perfectly balanced, with 25,000 training samples and 25,000 test samples, but the text length is significantly variable, with some reviews being very long and detailed.

6.2.9 Dataset Summary Table

A summary table of the different datasets we use with Layer Cake and their basic characteristics is provided below.

Dataset	Type	Classes	Size	Characteristics
Reuters-21578	multi-label	115	64 MB	News articles, imbalanced class distribution.
BBC News	single-label	5	6.7 MB	Balanced topics from BBC news.
arXiv	multi-label	58	5.5 GB	Research papers, overlapping categories.
arXiv Protoformer	single-label	10	147 MB	Simplified version of arXiv for single-label tasks.
IMDB	single-label	2	694 MB	Movie reviews, sentiment analysis.
OHSUMED	multi-label	23	387 MB	Medical abstracts, domain-specific.
RCV1-v2	multi-label	101	7.4 GB	Large-scale news corpus, hierarchical topics.
20 Newsgroups	single-label	20	15 MB	Newsgroup articles, balanced and diverse topics.

6.3 Supported Metrics

Primarily we look at *macro-f1* and *micro-f1* scores for the different classifier, language model, and representation form test runs, but we also track *precision*, *recall*, *accuracy*, *Hamming Loss* and *Jaccard Index* numbers for each model run as well, although these latter metrics are not used in our analysis.

Note that we use the *macro-f1* score to determine whether or not our neural models are improving and to determine whether or not we have met our early-stop condition. We briefly cover how these metrics are calculated below. As a refresher, we outline how we compute these metrics within the context of Layer Cake benchmarking.

Precision measures the accuracy of positive predictions. It is the ratio of correctly predicted positive observations to the total predicted positives, or

$$\text{Precision} = \frac{TP}{TP + FP}$$

where *TP* is true positives and *FP* is false positives.

Recall (sensitivity) on the other hand, measures the ability of a model to find all the relevant cases (i.e., True Positives) within a dataset. It is the ratio of correctly predicted positive observations to all observations in the actual class, or mathematically:

$$\text{Recall} = \frac{TP}{TP + FN}$$

The *F1 score* is a more widely used measure of predictive performance given that incorporates precision and recall into one measure using the *harmonic mean*, defined mathematically as:

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The macro variant of the F1 score, i.e. *macro-f1*, uses the *arithmetic mean* of all the per-class F1 scores, treating all classes equally regardless of their *support* - i.,e. the number of times the class is found in the training data set. This unweighted average F1 Score across classes is defined mathematically as:

$$\text{Macro-F1} = \frac{1}{N} \sum_{i=1}^{N} 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i}$$

Where Precision and Recall are calculated as outlined above and *N* is the number of classes.

The *micro-F1 Score* on the other hand uses the class specific contribution to compute the average F1 score, measuring the aggregate contributions of all classes with respect to the average score, or mathematically:

$$\text{Micro-F1} = 2 \times \frac{\text{Micro-Precision} \times \text{Micro-Recall}}{\text{Micro-Precision} + \text{Micro-Recall}}$$

Where Micro-Precision and Micro-Recall are the precision and recall calculated globally.

6.4 Language Representation and Test Workflow

The baseline ML classifiers we use within the context of this work are Support Vector Machines, Logistic Regression and Naïve Bayes models, the first two of which effectively work by optimally separated the underlying dataset representation, in high dimensional space defined by the space of words in the underlying dataset. The Naïve Bayes model is a pure probabilistic approach and while its use is limited, due to the constraint that it must work in the positive real number space, it nonetheless provides a nice probabilistic baseline for our analysis.

The neural models come in two forms - the neural models that are designed to work with static word embedding language models such as GloVe and Word2Vec (and FastText), and then the transformer architecture models which are much more sophisticated and computationally intensive than their static model counterparts. For these latter models we leverage the Hugging Face APIs which not only provide the ability to work with the various models that are included in this study but also come with predesigned Sequence Classifier models that are designed specifically for sequence, i.e. text or natural language, classification.

6.4.1 Machine Learning Approaches (Baseline Testing)

With our baseline testing, we experiment on a variety of forms of language representation given that the model design is straightforward enough, and flexible enough, to support a variety of different language representation methods. These models are constructed using the base python sklearn libraries and as such they are CPU bound, as opposed to the GPU bound neural models that we describe below. While we primarily look at these baseline ML classifiers as baseline test cases to be viewed against the backdrop of

their more sophisticated – and computationally intensive – counterparts, these models are nonetheless effective in and of themselves as standalone text classification solutions and are very easy to work with and perform very well on small and medium size datasets.

Another insight that is revealed within the context of this research is that with the static embedding (language) models, one has a great deal of flexibility with respect to how the embedding layer, which is the storehouse of the word-embedding matrix basically, is setup and initialized. The fundamental method that we use for our tests, again following (Moreo, et al., 2021), is to set up the embedding layer with only the necessary and requisite embeddings in the dataset that we are solving for – effectively reducing the semantic embedding space by the difference between the language model dictionary size and the dataset dictionary size (as measured by the language model in question). This computational optimization is significant, as our tests show, and these hand-rolled PyTorch models are very efficient, albeit not quite as effective as their transformer cousins, even with the additional assistance from WCEs.

The transformer models however, given that they aren't as much embedding storehouses as much as they are systems of embedding generators, have a much more tightly coupled and rigid structure that does not lend itself to customization at all really. This of course not only has performance implications given that invariably the language model supports tokens that are not relevant for a given dataset we may be fine-tuning the model on, but nonetheless are loaded into the model memory during training and testing adding dimensional complexity to each and every operation, and greatly increasing the amount of data that needs to be synchronized across GPUs for large parallel model training which in fact is the norm when it comes to LLMs given their size. This undoubtedly is a factor in the results around model construction times where the transformers, as expected, way exceed the training times for both the ML and static LM neural models.

We have two basic forms of language representation that we test when working with the baseline ML in models that we support with Layer Cake, each with its own computational architecture essentially and each coupled with its own forms of feature reduction, a critical aspect of model optimization in the ML world given the dreaded *curse of dimensionality*. The two basic forms of language representation are:

- **i.** *vectorization forms*: Layer Cake supports both Count and TF-IDF Vectorization with built in feature reduction using the sklearn min_df feature which only includes words, or tokens, that appear in more than min_df docs, in our case 5. We call these language representation forms *vmode* in our analysis and they come in two basic flavors, Count and TF-IDF, [23]
- **ii.** *embeddings*: word, or again token, level representations of real-number vectors of fixed dimensions (set by the underlying language model during training), which also come in two basic flavors, static and dynamic.

Each of these representation form types is tested with and without Word-Class Embeddings[24], given that we have the ability to perform various forms of linear algebraic transformation on the primary language

[23] The TFIDFVectorization arguments that we use, following the 2019 study, are *min_df=5* which indicates that a word, or token, is only included in the doc vector if it exists in at least 5 documents across the dataset, and *sublinear=True* which applies sublinear scaling to the tf-idf values that are used in the underlying doc vector representation, i.e. the tf value is replaced with 1+log(tf), see https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html for details. Note also that while the Layer Cake platform supports Count vectorization as well, these results are not reported in this study as they are what one might expect, that the tf-idf representation performs better generally than the straight up count vectorization, i.e. bag of words model.

[24] As defined in the study of the same name published in 2021 (Moreo, et al., 2021) which forms the basis of the foundational code for Layer Cake.

representation form and as long as we end up with a feature matrix (a dictionary size by feature size matrix essentially) that aligns with the label matrix, the model will classify the data fairly consistently and effectively. The setup for the neural model is much more rigid, with a language model token indexing strategy for the embedding layer effectively – optimized in the static case against the dataset we want to tune the model to, and fully loaded in the case of the transformer models.

We also include a few variants of Latent Semantic Analysis (LSA), sometimes called Latent Semantic Indexing (LSI), (Deerwester, et al., 1990) into the ML representation form mix as well, a method that continues to show its value as an extremely effective model optimization technique. We even show that LSA can be used in conjunction with WCEs / TCEs to boost model effectiveness, consistent with the results found by (Moreo, et al., 2021) with respect to WCEs boosting the performance of DL approaches with static embeddings. LSA uses a matrix factorization technique called *singular value decomposition* (SVD) which generalizes the eigendecomposition of square matrices to non-square matrices.

One of the advantages of looking at these different representation forms within the context of these classic machine learning classifiers is that the underlying mathematics, and the underlying forms of language representation are relatively straightforward and are coupled together directly, as a form of linear separation in vector space, in semantic space really following (Worth, 2023). Once we get to the transformer classifiers, we become constrained by the underlying language model itself in terms of how the text can be represented. We discuss this more below in the section on dynamic, contextualized deep learning classifiers, aka *transformers*.

The ML framework in fact, is flexible enough to not just support different text vectorization techniques and different embedding structures along with LSA as a form of feature reduction, but it also can be used to test a wide variety of different combinations thereof, including Word/Token Class embedding integrations into the feature mix as well, allowing for insights into the performance of the various forms of language representation both in terms of effectiveness (*macro-f1* and *micro-f1* statistics) as well as model training times, i.e. *timelapse analysis*. These various forms of (linear algebraic) transformation amount to, in the case of feature reduction, semantic spatial space compression where a higher dimensional language representation form is projected into a lower dimensional form using matrix dot product operations. This of course in many respects what language models do with their embedding structures, reduce the feature space for the word or token representation from the dictionary size down to the embedding vector dimensional size (the *solo* and *doc* representation forms described below).

The table below summarizes the various forms of language representation that are used within the context of the ML baseline testing of Layer Cake.

Representation	Description	Feature Sources	Key Use Case
solo	Embedding representation alone	Embeddings (weighted/avg/summary)	When embeddings alone are sufficient
solo-wce	Embeddings + Weighted Class Embeddings	Embeddings + WCEs/TCEs	When class interaction features improve results
cat-doc	Vectorized text + Embedding representation	TF-IDF/Count + Embeddings	When combining raw text and embeddings enhances performance
cat-wce	Vectorized text + Weighted Class Embeddings	TF-IDF/Count + WCEs/TCEs	When integrating raw text with class interaction features is needed

Representation	Description	Feature Sources	Key Use Case
cat-doc-wce	Vectorized text + Embedding representation + Weighted Class Embeddings	TF-IDF/Count + Embeddings + WCEs/TCEs	Comprehensive representation combining raw, embedding, and class features
dot	Projection into embedding space via dot product	TF-IDF/Count + Embedding Space	When projecting text into embedding space is crucial
dot-wce	Projection into embedding space with Weighted Class Embeddings	TF-IDF/Count + Embeddings + WCEs/TCEs	Projection combined with class-sensitive embedding
lsa	Latent Semantic Analysis for dimensionality reduction	TF-IDF/Count + LSA	For dimensionality reduction while keeping semantic content
lsa-wce	LSA + Weighted Class Embeddings	TF-IDF/Count + LSA + WCEs/TCEs	When reduced features and class sensitivity are both needed
vmode	Raw vectorized representation (TF-IDF or Count)	TF-IDF/Count	Baseline approach using only vectorized data

The basic workflow of the ML classification test module follows the basic steps of any standard NLP workflow – data preparation, model training, model testing. We simply scale the problem using all of the datasets, language models and other configuration parameters (hyperparameters) to drive each of the test runs, with the output logged that captures all of necessary and poignant information about the test run – including but not limited to: classifier type, model hyperparameters, system info, language model, metric, value, timelapse, etc., the basis for our analysis. Layer Cake includes a reporting and analytics module which generates charts, heatmaps and summary data related to each and every test run, logging, macro-f1, mico-f1, accuracy, precision, recall, hamming loss and jacard-index values for every model configuration.

Our test script loops through all the different in scope Classifiers, along with the in scope hyperparameters and representation forms, prepare the data as needed for the model in question, build and train the model with the training and validation data from the dataset (supervised), and then test the model against the test data we set aside when we initialized the dataset docs. For the multi-class datasets, we use a one-vs-all approach, encapsulated in the OneVsRestClassifier API from sklearn which involves fitting one classifier per class and then for each classifier the class is fitted against all other classes.[25] For single-label datasets, Layer Cake uses the relevant classifier LinearSVC, LogisticRegression, or MultinomialNB as the case may be, again we leverage the sklearn APIs for these classifiers as well. The test data we look at below uses the default model parameters.

In preparing the data, we preprocess it by retrieving the data from the relevant Layer Cakes datasets subdirectory, downloading it if necessary, and then for the word and sub-word based language models (i.e. GloVe, Word2Vec and fastText) we lowercase the text, remove punctuation, and remove stopwords whereas for the token based, transformer models (GPT2, BERT, DistilBERT, DeepSeek, RoBERTa and XLNet) we leave the underlying text intact as the transformer based models are designed to handle full sentences and given their dynamic, contextualized computational nature the stopwords and case sensitivity allows the models to generate better embeddings. We then construct the label matrices from the training and validation data in the right form depending upon whether or not the underlying dataset is single-label classification or multi-label classification, and then proceed to compute, and cache, the various base representation forms of the text which include vectorization forms as well as the embedding dataset dictionary and document

[25] See https://scikit-learn.org/0.24/modules/generated/sklearn.multiclass.OneVsRestClassifier.html

encoding structures to be used by the underlying classifiers. These operations can be computationally expensive, so we do them once for each dataset, language model and representation form and then use these cached files, pickle files, throughout.

The vectorized text is constructed using TFIDFVectorizer API from sklearn, which is CPU bound (as opposed to GPU bound as the deep learning classifiers are, requiring special hardware effectively), which is called a) with the min_df=5 flag which operates as a form of feature reduction by limiting the words, subwords or tokens that are included in the output tf-idf vectors to those tokens that are found in at least 5 documents (called 'cutoff' in the literature) and b) we use the sublinear argument to log normalize the tf-idf values, converting the term frequency (*tf*) value to 1 + log(*tf*).[26] The other representation forms include the *embedding_vocab_matrix* structure which is a static representation of the dictionary of words or tokens that are in scope for the given dataset computed from the relevant underlying language model, and then the dataset representation in the underlying language model itself which is done via the *encode_docs* method that we describe above.

Note that for the transformer models, the document encoding is computed dynamically, as the documents are passed through the classifier, via the transformer model itself which sits at the forefront of that computational edifice so to speak. For the static language models the representation is well, static, a simple mapping exercise for each word, or n-gram, in the dataset is represented by a single static embedding representation. The computational component here is how it is that these transformer models get more accurate in the construction of their semantic space, and also where a good deal of the computational overhead is.[27]

6.4.2 Neural Models (Deep Learning)

The neural classifiers we use in Layer Cake are split into two groups, one for the static word embedding language models (Word2Vec, GloVe and FastText) and another for the transformer models, each tested under various hyperparameter conditions and each with its own distinct, although closely related, form of language representation. In fact, technically, each language model has its own form of language representation, albeit all architecturally analogous, all nonetheless distinct with respect to how the underlying feature space – semantic space – is defined. Part of what we are looking to understand with Layer Cake in fact is the difference this type of dimensional attribute has on classifier performance, if any.

The first neural model test script is inherited from (Moreo, et al., 2021) but extended to support Word2Vec and FastText in addition to GloVe. The script, the dataset preparation, and the classifier design in this module are custom tailored, and optimized, to fit the static language models. The classifiers are hand crafted PyTorch models that are set up in very simple CNN, LSTM and ATTN (attention) based neural models, basically sequence classifiers. The second test script we use is for the transformer language models, which require slightly different text preprocessing (we leave the text in its raw form basically) and is built using the

[26] This approach follows (Moreo, et al., 2021). For details on TFIDFVectorizer API from sklearn see https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[27] In the optimized form, denoted as '[opt]' in the representation shorthand we use throughout, we use a random sampling of grid search options approach via the sklearn RandomizedGridSearchCV API, using 9 as the n_iter argument (9 options are tried), the technique being a form of optimized way to get close to the optimal set of values for a given model configuration. It's worth noting however that the variance in terms of model performance between these optimized models and the default parameter ([def] representation form) models is not as significant as one might think and the performance hit for building and testing the optimized classifiers is, at least for the ML test cases, significant.

HuggingFace (HF) libraries which come with a built basic form of parallelism which is convenient for testing.[28]

Just as with the ML classifiers, the neural testing covers both single label classification datasets, where each document gets classified to one and only one class label, as well as multi-label datasets where a document can be classified to one or more labels or classes. Special handling must be done to ensure that our neural classifiers support both cases, just as in the ML case, and this is accomplished by ensuring that the format of the labels that is fed into the classifier is correct and reflects the dataset type, and the classifier itself is set up with the proper loss function depending on the dataset type as well – CrossEntropyLoss for single-label datasets and BCEWithLogitsLoss for multi-label classification.

The static word embedding (neural) classifiers come in three neural variants, simple PyTorch based CNN, ATTN and LSTM architectures, tested in various hyperparameter configurations with and without the WCEs, and our transformer language model classifiers come in two variants, a HF Sequence Classifier variant (hf.sc) which encapsulates the classifier logic for you for each language model, as well as simple CNN classifier which is modeled after its neural counterpart for comparison (hf.cnn). What follows below is a brief description of the test workflows for each of the static neural classifier testing as well as the transformer model testing, with preliminary analysis of each set of data individually, and then combined together so that we get a clear picture of classifier and language model performance in each setting, with again a focus on macro-f1 and micro-f1 metrics, along with training times.

6.4.2.1 Static Neural Classifiers

We generally follow the same test, data preparation, model training and testing workflow that we used when we tested the baseline machine learning classifiers, except that now given that we are working with a neural model, we have very specific requirements with respect to how we are to represent the sequential text data such that the neural model can train effectively. Essentially this involves, as discussed earlier, building a dictionary for the underlying dataset that is a function of the language model (and tokenization strategy) we are working with, and then indexing the dataset against this dictionary and then using these static embeddings at runtime (in the forward method of the neural model training) to represent the text (sequence) which is fed into the classifier head.

For the static neural models, we first initialize and prepare the underlying dataset, and, from a text preprocessing perspective, consistent with best practices for static language representations, we remove stopwords and punctuation from the text before we index it for input into the neural model. [29] The text is then converted into an index representation form which is typical for NLP deep learning architectures, where each word is indexed into a dictionary that maps back to the associated index in the underlying language model, facilitating a lookup of the embedding for the word in question during the forward pass of training. For the transformer based classifiers however, we leave the text as is for the most part, allowing the transformer model tokenizers to do their thing and capture as much information about the underlying text as possible, something that distinguishes the transformer models from their static embedding counterparts

[28] As of this writing Hugging Face supports over 300,000 language models.

[29] This is in contrast to the data preparation for the transformer based models where we keep the text case sensitive, and leave the stopwords and punctuation in place given that the transformer based models are designed to handle the full complement of language syntax, one of its prominent features that distinguishes it from its static language model counterparts which are not, by design, 'context aware'.

which use word, or subword (n-gram) tokenization strategies and generally are not designed to handle punctuation. Computation of the different language representation forms we use for the various tests we perform, is expensive, so we cache this information using pickle files for later use.

The language representation form that is used is to construct the neural model classifier embedding layer for the dataset in question using a) the dataset vocabulary words (post vectorization (i.e. vectorized using *TFIDFVectorizer* with 'cutoff' = 5 so that this dictionary is a subset of the entire dataset dictionary) combined with the static embeddings for all *known* words (i.e. words or terms that have defined static embedding representation as defined by the underlying language model, or in other words all words that are not in the vectorized dataset representation but exist in the underlying dataset doc set and have a corresponding definition in the underlying language model), together forming what you might call the static embedding layer dictionary.

The input text data is tokenized into individual words using the dataset's vectorization strategy (e.g., TF-IDF or count vectorization) and each token is mapped to its corresponding embedding using the custom embedding layer, which is designed to support both pretrained embeddings, learnable embeddings, as well as supervised embeddings, i.e. WCEs, which serve to extend the pretrained language model representation of a given word using class, or label, co-occurrence statistics, serving to add additional information to the pretrained language model representation of the word and extending the dimensions of the underlying word vector by the number of classes in the dataset. The indexed document representation form, which maps to the pretrained underlying word embeddings, is standard (best) practice for neural text classifiers as it allows for efficient lookup of token embeddings from the embedding matrix. Note that in the static neural model classification design, we also add special tokens for unknown words (UNKTOKEN), and padding (PADTOKEN), which are leveraged by the model to ensure the neural model handles the text representation form correctly for classification, meaning that all OOV terms are treated by the model in the same way as a special token, with its own embedding representation.

For example, if we look at the static embedding CNN neural architecture for example in the figure below, it is the EmbeddingCustom layer which encodes the (static) embeddings for the entire vocabulary that is in scope for the dataset in question. Given that the neural model is 'hand-crafted' so to speak, we populate this embedding layer only with words (or subwords in the case of fastText) that exist in the dataset we are classifying, an optimization that we do not have the flexibility to implement with the dynamic, transformer models in fact and one of the reasons that these static models are so much more efficient computationally than their dynamic counterparts. The custom nature of the neural model allows us to change the dimensions of the underlying EmbeddingCustom layer as well - like for example when we add WCEs into the semantic space which we do by concatenating these embeddings, whose size is the number of class labels, to the language model embedding layer directly, allowing the model to learn the optimal set of weights from the entire embedding set from the start.

```
NeuralClassifier(
  (embed): EmbeddingCustom(
    (pretrained_embeddings): Embedding(22846, 300)
  )
  (projection): CNNprojection(
    (conv1): Conv2d(1, 128, kernel_size=(3, 300), stride=(1, 1))
    (conv2): Conv2d(1, 128, kernel_size=(5, 300), stride=(1, 1))
    (conv3): Conv2d(1, 128, kernel_size=(7, 300), stride=(1, 1))
    (dropout): Dropout(p=0.5, inplace=False)
  )
  (label): Linear(in_features=384, out_features=5, bias=True)
)
```

Figure 8: Static Language Model Classifier (CNN Architecture)

The custom embedding layer combines pretrained embeddings with learnable embeddings, and optionally WCEs, to allow the model to be tailored specifically for the dataset in question that is fed into the model for training. This representation form is what is used to initialize the embedding layer of the neural models, which again is used at runtime, i.e. at training time, to be retrieved by the classifier and processed by the neural model in question, pooled together in batches, and ultimately classified the transformation layers of the different deep learning models we test with.

Once the dataset is indexed, a process which must be aligned with the tokenization strategy of the underlying language model we are working with, we then test each of the static word embedding neural classifiers under different hyperparameter settings and with each of the in scope datasets, both with and without WCEs, resulting in metrics – like the initial set of ML classifier data – that can be directed compared against the 2021 study (Moreo, et al., 2021). During training, these indexed, numerical representations of the documents are batched and passed to the model's embedding layer through the NeuralClassifier so that, via the EmbeddingCustom layer, the appropriate embeddings for each token from the embedding matrix (pretrained + learnable embeddings) can be retrieved and used for downstream processing.

The embeddings are then, in the forward method of the classifier, processed by either the CNN, LSTM, or ATTN projection layers depending upon the type of neural classifier being trained, the end result of which yields a contextual, language model embedding representation of the documents is created for the underlying model. One of the primary differences between the static language models and the dynamic ones in fact, is the tokenization strategy that is used. While static word embeddings are designed to look at, and represent words[30], the transformer-based models break language down into tokens, which while solving for out of vocabulary and polysemy type problems, nonetheless moves the language representation form away from word, term or subword based semantic space, to a more granular semantic space whose dimensions are defined by the dictionary of tokens that is established and used by the language model in question.

6.4.2.2 Dynamic Neural Classifiers (transformers)

The transformer model testing workflow, and data preparation, is slightly different than the neural classifier test script for the static pretrained language models. We use two distinct classifier architectures that are

[30] Or in the case of fastText subwords but ultimately these are words in the sense that the subwords themselves do in fact have a semantic meaning

built off of the Hugging Face libraries - namely the SequenceClassifier and a simple CNN, neural network architecture that is modelled after the same neural architecture we use for the static language model testing. The basic test workflow is designed to generate a Classifier of a given type, with a given pretrained language model, a given underlying dataset, and a set of program and model hyperparameters that define the test run. Each run is logged with details around system parameters (OS, GPUs, CPUs, memory, etc.), language model type and attributes, the dataset and classifier type, and other hyperparameters to be used by Layer Cake Reporting & Analytics module for analysis.

The power of this transformer-based approach, which dominates the language model research and development landscape right now for good reason, is predicated on the construction of an embedding space that is defined by the underlying tokenization dictionary used by the language model in question, as well as these models' ability to generate context-aware embeddings via the attention mechanism. This granularity with respect to the underlying semantic space yields very accurate results across a variety of NLP problems but nonetheless comes at a cost in terms of the system resource and computational requirements which are necessary to generate the kind of granularity necessary to drive the performance improvements.

Much of the computational complexity around working with transformer based language models is from this runtime complexity that is driven from the computational requirements of the underlying language model itself, which clearly does a lot of work to generate the context aware embeddings, which are of course the main distinguishing characteristic of the transformer-based models versus their static counterparts. In other words, given the dynamic nature of the transformer based embeddings, the embeddings – the model really – has to sit in a different type of neural classifier entirely that accounts for, and incorporates, the underlying language model directly into the classifier so that the embeddings can be computed, dynamically, and then they can be pooled and fed to the classifier head portion of the model. One of the implications of this technical dependency is that the static based neural models are significantly less computationally intensive than their transformer-based counterparts, one of the key measurement findings of this study in fact. What's also less intuitive but nonetheless another important finding of this study is that the neural classifiers we use with the static language are also more flexible, more configurable and optimizable, and also less computationally intensive than their more sophisticated, context-aware embedding generating counterparts (i.e. transformer language models).

We see this for example in the way the Embedding Layer is structured and how it can be configured in the static neural classifiers versus the dynamic ones - the embedding layer being the foundational structure of all of the neural model classifiers that is used to generate the embeddings that are used to represent the text of the underlying dataset, either as a straight index lookup in the forward pass of the model as is the case with the static neural models, or as inputs into the runtime computational dynamic embeddings that the transformer models generate and that can be consumed, really *pooled*, for classification in the forward pass of model training. Specifically, with the static (word) embedding designed neural classifiers, we have the ability to both filter as well as add on to this initialized Embedding layer, concatenating WCEs in some cases, as well as filtering out the unneeded embeddings in order to optimize the layer by only using the word embeddings that are necessary for the task at hand, rather than the entire language model embedding dictionary which is the case for transformer models. It's this flexibility that allows for WCEs to be integrated into the neural classifiers for the static word embeddings language models, but not for the transformer-based language models which, given their internal dependency on the underlying computational logic that is intrinsic to the language model type itself, cannot be extended horizontally (concatenated) with TCEs (token-class embeddings) without breaking the model.

The transformer-based classifiers are - given the dynamic nature of the underlying language models – integrated directly with the language models which are neural network models in and of themselves, each of which dictates the structure, size and content of its underlying embedding layer - i.e. the specific model data. Outside of the computational logic differences between the static and dynamic language representation forms, the other main difference between the two sets of neural classifiers is of course both the structure, and content, of this embedding layer which provides the computational basis for embedding representation by the language model for the dataset in question – for both the static language models as well as the dynamic ones.

As illustrated in the CNN HF transformer classifier shown below for example, the embedding layer is initialized to include the entire dictionary of token embeddings for the *bert-base-cased* language model we use, a language model which has 28996 pretrained token embeddings, each of which is of size 768, i.e. the embedding dimension size or the *hidden_size* of the (language) model. The HF CNN architecture, as the name implies, leverages a CNN neural architectural approach to classification constructed directly on top of the HF transformer model itself, allowing for the transformer computed embeddings to be first pooled and then passed into the CNN for classification. The Hugging Face Sequence Classifiers are structured the same way, but each of the transformer models has its own unique classifier attached to it, not necessarily a CNN like we use as shown in the figure below, which is analogous to the CNN classifier we use for testing the static word embeddings.

```
LCCNNBERTClassifier(
  (loss_fn): CrossEntropyLoss()
  (pretrained_embeddings): BertModel(
    (embeddings): BertEmbeddings(
      (word_embeddings): Embedding(28996, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (token_type_embeddings): Embedding(2, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (encoder): BertEncoder(
      (layer): ModuleList(
        (0-11): 12 x BertLayer(
          (attention): BertAttention(
            (self): BertSdpaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): BertSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): BertIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): BertOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
    (pooler): BertPooler(
      (dense): Linear(in_features=768, out_features=768, bias=True)
      (activation): Tanh()
    )
  )
  (conv1): Conv2d(1, 128, kernel_size=(3, 768), stride=(1, 1))
  (conv2): Conv2d(1, 128, kernel_size=(5, 768), stride=(1, 1))
  (conv3): Conv2d(1, 128, kernel_size=(7, 768), stride=(1, 1))
  (dropout): Dropout(p=0.5, inplace=False)
  (classifier): Linear(in_features=384, out_features=5, bias=True)
)
```

Figure 9: LCCNNBERTClassifier Architecture (transformers)

We would point out that for both of these (Hugging Face) transformer model classifiers that we use with Layer Cake, the initial embedding layer is fixed, constrained by the underlying language model itself – which in this case consists of 28,996 token embeddings, each of which is a vector of size 768 and each of which maps back to a token in the dictionary space of the specific BERT language model variant being used - *bert-base-cased* in our work. As mentioned previously, given the tight coupling of this embedding layer with the runtime computational requirements necessary to generate context-aware embeddings which is built into the model itself, we were not able to extend, or minimize (optimize) this initial embedding layer at all for any of the transformer models without breaking them (i.e. they become non-functional) so it's fair to say that with the extra granularity and complexity that is built into these transformer language models, comes not just state-of-the-art results, but also mandated minimal computational and resource requirements that

are dictated by the model itself rather than the input data that we are using to feed into the model – in other words no matter what the use case the full model must be used.

This lack of flexibility also shows up when we try and integrate externally trained embeddings into the model, e.g. WCEs or TCEs, which a) cannot be concatenated with the initial model embeddings in the Embedding Layer and so can only be integrated in the forward pass when the context-aware embeddings are computed, and b) are ineffective when pooled together with the run-time computed, context-aware embeddings in the forward pass of the model, no matter how they are integrated (dot product, linear transformation, addition). As we state earlier, the TCEs not only did not help the performance of the models (as measured by Macro-F1 or Micro-F1 or any performance measure for that matter), they seem to confuse the models and have a negative impact on performance, despite the fact that in theory at least, they should add additional information which helps the classification of sequence of tokens that is being pooled. This is not the case however.

As one might expect, this embeddings layer, *BertEmbeddings* in this case, is tightly coupled with the language model, which as one can see from the neural model shown below actually makes up the bulk of the whole neural model classifier itself. It's this model integration which allows for the dynamically generated (transformer) embeddings, for the specific sequence in question, to be passed to the classifier layers of the neural model during training (the forward pass). In other words, one of the distinguishing characteristics of these transformer models is that they themselves are rigidly fixed and heavily dependent on both the size and dimension, and contents, of their embeddings. While this shouldn't come as a surprise necessarily, it should be kept in mind when one considers what kind of flexibility one needs with a language model – like for example if there are other embedding forms that one wants to use in conjunction with pretrained language model embeddings, TCEs for example.

Therefore, given the more rigid nature of the embedding layer for the HF transformer classifiers, they are not well suited for either a) filter optimization methods example to only include token embeddings for those in scope for a given dataset, or b) extend the feature space of the embedding layer to include embeddings trained outside the model, like for example WCEs or TCEs in the case of transformers. Both of these optimizations are available if we are hand crafting the neural classifier using static embeddings however and they are applied for this set of test data (the static embedding test data) and have a significant impact on the runtime performance, aka computational requirements, of these models. This cannot be emphasized enough how important these factors are in any cost-benefit analysis of the relative performance of classifiers and/or language models and/or representation forms, as we demonstrate in this work.

The only way in fact that we could test the relative effect of TCEs in our HF, transformer-based language model driven classifiers was by was by including the TCEs with the dynamically generated embeddings in the forward pass of training and using various combinations of these two types of embeddings into the downstream classifier parts of the neural network architecture. And what we found, quite conclusively after a wide range of approaches to integrating the TCEs with the transformer model generated embeddings was that if anything, the inclusion of TCEs into the transformer model classifiers - whether by concatenation or dot product operations, i.e. linear transformations or projections effectively[31] – made the classifier less

[31] We experimented with dot product operations, addition operations, and concatenation operations of the transformer generated embedding representations of the text with the TCEs that we computed per class (per token). None of these methods worked for any of the transformer models we tested. We hypothesize that, given how well the transformer models perform generally relative to both their static embedding counterparts and the baseline ML classifiers we test with (SVMs, Logistic Regression methods primarily), that this is because the additional information that is encapsulated in these TCEs is already part of the underlying transformer model representation of the text at some level and

effective, despite the additional information that the TCEs contained. This is in fact, another one of the unique contributions of this work, i.e. that not only does adding the TCEs into transformer models prove ineffective, but that the underlying embedding layer of the transformer models does not lend itself to customization – period.

The final results we look at in for the transformer classifiers use a 4 A100 GPU system, distributed using the PyTorch Data Parallel form of model parallelism, which is the default mode used by the HF APIs, whereas the neural models needed just one (dedicated) GPU. For the HF transformer language model script, we have two different classifiers that we use, a built in HF Sequence Classifier which encapsulates the language model specific logic for you in a custom classifier alongside a simple CNN based architecture which is modelled after the CNN neural net that we use with the static embeddings.

therefore adding it to the embeddings only proves to reduce the effectiveness of the classification portion of the classifier. NB The WCEs are combined with the static language model embeddings via concatenation, i.e. we extend the embedding layer dimensions to include the WCEs yielding a richer word representation that is fed into the model during training.

7 Results & Analysis

In this section we analyze the data from the different Layer Cake test scripts, focusing on the primary dimensions of analysis that we are concerned with in this study, namely classifier architecture and language representation forms. We look at the ML data first, followed by the neural models, and then the summary analysis across the broad spectrum of classifiers included in Layer Cake to a get a full picture of the data.

For the purposes of the neural model analysis, we group the static and dynamic classifier test results together, yielding a comparative perspective for this specific dimension, static versus dynamic language models, which gives us a unique perspective on a) how much better the transformers are for this particular use case (classification) and b) what the underlying cost is computationally for this improvement. This puts us in a unique position to be able to ascertain the cost, again computationally, for the step up in model improvement – a measure we have not seen in the literature before.

For the ML classifiers, our baseline testing module effectively, we look at the several different dimensions of the results data, evaluating performance across the classifiers, the representation forms and the language model types.[32]

7.1 ML Classifiers and Representation Forms

We look at the baseline ML results by Classifier, Dataset and Representation Form, with each dimensional analysis supported by visualizations and summary data generated by Layer Cake's Reporting and Analytics module below.

7.1.1 By ML Classifier

To visualize the results by classifier – SVM, LR and NB models - we use a box plot, aka a box-and-whisker plot, with ethe Macro-F1 results on the top and the Mirco-F1 results on the bottom. This box plot shows the data distribution based on a five-number summary: *minimum, first quartile (Q1), median, third quartile (Q3), and maximum* which helps show the central tendency of the underlying data as well as the data spread and outliers.

The "box", which is the shaded area (from Q1 to Q3) represents the interquartile range (IQR), which is the range between the first quartile (Q1) and the third quartile (Q3). The height of the box gives you an idea of the spread of the middle 50% of the data, i.e. a taller box indicates more variability in the data, while a shorter box means the values are closer together.

[32] Note that all of the data we look at for the ML classifiers was generated with the *default* parameters, to get the breadth and scope of the hyperparameter and representation forms for the different variations of classifiers and language types, optimized runs proved very time consuming, particularly the LR classifiers for the larger datasets (RCV), and language models (Llama).

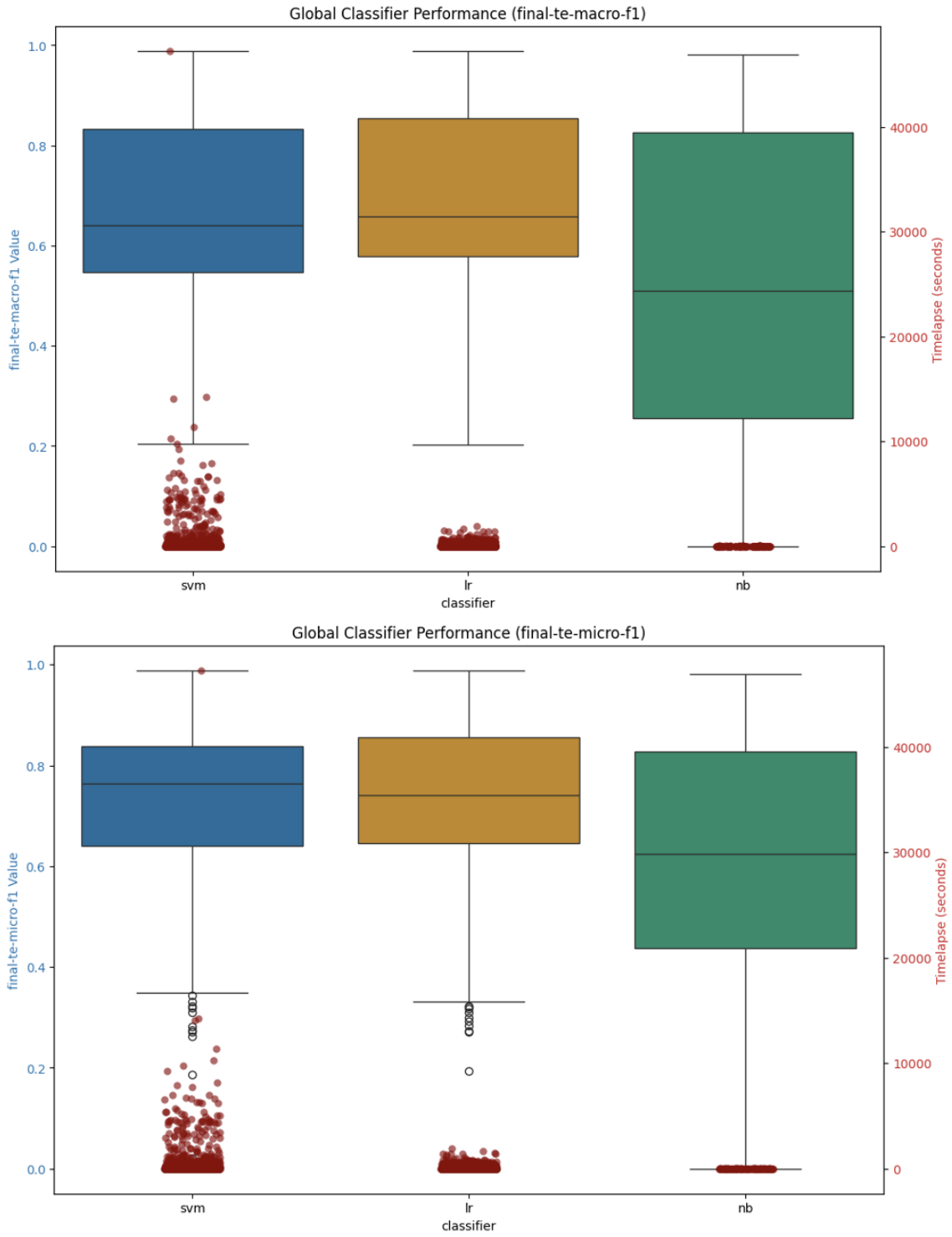


Figure 10: Machine Learning Classifier Performance Summary: Macro-F1 and Micro-F1 Scores

On average across the datasets and language models, the Logistic Regression (LR) models perform slightly better than the SVM classifiers, and both the SVM and LR classifiers perform slightly better than the Naïve

Bayes models.[33] The runtimes for the classifiers are clustered below 10,000 seconds, which equates to just under three hours for model construction, with the bulk of the classifiers, particularly for the Logistic Regression models, at around 1000 seconds which is 16 minutes, very reasonable.

The SVM classifier models take slightly more time to build which is expected given the computational complexity of the underlying mathematics for SVMs versus their LR or NB counterparts. Naïve Bayes is the most efficient of the models but given the nature of the model does not support negative numbers and as such can only be used with text vectorization representation forms, most embeddings spill into negative dimensional space, but generally speaking the metrics hold up for NB as well.

Table 1: ML Classifier Summary Data: Macro-F1 (Top 5)

classifi	dataset	dimension	representation	value	timelapse
svm	20newsgroups	(11314, 22920)	[svm:20newsgroups]:->count+llama(avg):[def]	0.705	64.75
lr	20newsgroups	(11314, 24988)	[lr:20newsgroups]:->tfidf+tfidf.(llama+wce(tfidf))+tfidf.(llama+wce(tfidf)):[def]	0.703	314.08
lr	20newsgroups	(11314, 4116)	[lr:20newsgroups]:->llama(avg)+tfidf.(llama+wce(tfidf)):[def]	0.701	36.34
lr	20newsgroups	(11314, 22920)	[lr:20newsgroups]:->count+llama(avg):[def]	0.700	787.88
svm	20newsgroups	(11314, 4116)	[svm:20newsgroups]:->llama(avg)+tfidf.(llama+wce(tfidf)):[def]	0.689	133.89
lr	arxiv	(11962, 18997)	[lr:arxiv]:->count+llama(avg):[def]	0.784	466.42
svm	arxiv	(11962, 18997)	[svm:arxiv]:->count+llama(avg):[def]	0.776	59.19
lr	arxiv	(11962, 18948)	[lr:arxiv]:->tfidf+llama(avg):[def]	0.769	151.81
lr	arxiv	(11962, 2048)	[lr:arxiv]:->llama(avg):[def]	0.769	22.03
svm	arxiv	(11962, 18948)	[svm:arxiv]:->tfidf+llama(avg):[def]	0.765	23.97
lr	arxiv_protoformer	(82500, 34540)	[lr:arxiv_protoformer]:->tfidf+llama(avg):[def]	0.857	2569.18
lr	arxiv_protoformer	(82500, 2048)	[lr:arxiv_protoformer]:->llama(avg):[def]	0.856	184.94
lr	arxiv_protoformer	(82500, 1536)	[lr:arxiv_protoformer]:->deepseek(avg):[def]	0.855	154.60
svm	arxiv_protoformer	(82500, 1536)	[svm:arxiv_protoformer]:->deepseek(avg):[def]	0.850	238.68
svm	arxiv_protoformer	(82500, 32930)	[svm:arxiv_protoformer]:->tfidf+deepseek(avg):[def]	0.850	1571.72
lr	bbc-news	(1229, 1541)	[lr:bbc-news]:->xlnet(avg)+tfidf.(xlnet+wce(tfidf)):[def]	0.989	2.34
lr	bbc-news	(1229, 8969)	[lr:bbc-news]:->tfidf+tfidf.(xlnet+wce(tfidf))+tfidf.(xlnet+wce(tfidf)):[def]	0.989	3.47
lr	bbc-news	(1229, 1541)	[lr:bbc-news]:->roberta(avg)+tfidf.(roberta+wce(tfidf)):[def]	0.989	1.86
lr	bbc-news	(1229, 8537)	[lr:bbc-news]:->tfidf+tfidf.(roberta+wce(tfidf))+tfidf.(roberta+wce(tfidf)):[def]	0.989	1.72
svm	bbc-news	(1229, 768)	[svm:bbc-news]:->gpt2(avg):[def]	0.988	1.72
lr	imdb	(25000, 30664)	[lr:imdb]:->tfidf+roberta(avg):[def]	0.925	198.31
lr	imdb	(25000, 2048)	[lr:imdb]:->llama(avg):[def]	0.923	27.83
svm	imdb	(25000, 768)	[svm:imdb]:->roberta(avg):[def]	0.922	9.38
lr	imdb	(25000, 32714)	[lr:imdb]:->tfidf+llama(avg):[def]	0.921	489.62
lr	imdb	(25000, 768)	[lr:imdb]:->roberta(avg):[def]	0.921	7.65
svm	ohsumed	(24061, 18940)	[svm:ohsumed]:->tfidf[fasttext]:[def]	0.691	13.22
svm	ohsumed	(24061, 18940)	[svm:ohsumed]:->tfidf[glove]:[def]	0.691	13.44
svm	ohsumed	(24061, 18940)	[svm:ohsumed]:->tfidf[word2vec]:[def]	0.691	14.11
svm	ohsumed	(24061, 19240)	[svm:ohsumed]:->tfidf+fasttext(avg):[def]	0.690	20.48
svm	ohsumed	(24061, 19240)	[svm:ohsumed]:->tfidf+word2vec(avg):[def]	0.687	17.97
lr	rcv1	(23149, 30931)	[lr:rcv1]:->count+llama(avg):[def]	0.648	11126.84
lr	rcv1	(23149, 30931)	[lr:rcv1]:->tfidf+llama(avg):[def]	0.635	8408.93
lr	rcv1	(23149, 2048)	[lr:rcv1]:->llama(avg):[def]	0.633	889.43
svm	rcv1	(23149, 28679)	[svm:rcv1]:->tfidf+roberta(avg):[def]	0.629	5519.30
svm	rcv1	(23149, 2048)	[svm:rcv1]:->llama(avg):[def]	0.624	1749.58
svm	reuters21578	(9603, 9556)	[svm:reuters21578]:->tfidf+glove(avg):[def]	0.606	19.01
svm	reuters21578	(9603, 9556)	[svm:reuters21578]:->tfidf+fasttext(avg):[def]	0.606	16.72
lr	reuters21578	(9603, 9256)	[lr:reuters21578]:->tfidf[fasttext]:[def]	0.600	19.53
lr	reuters21578	(9603, 9256)	[lr:reuters21578]:->tfidf[glove]:[def]	0.600	20.87
lr	reuters21578	(9603, 9256)	[lr:reuters21578]:->tfidf[word2vec]:[def]	0.600	22.29

[33] The Naïve Bayes classifier, MultinomialNB from sklearn, only supports the vectorized representation form given its dependency on positive embeddings only so this is the only representation form that is computational viable.

Table 2: ML Classifier Summary Data: Micro-F1 (Top 5)

classifi	dataset	dimension	representation	value	timelapse
lr	20newsgroups	(11314, 24988)	[lr:20newsgroups]:->tfidf+tfidf.(llama+wce(tfidf))+tfidf.(llama+wce(tfidf)):[def]	0.715	314.08
lr	20newsgroups	(11314, 4116)	[lr:20newsgroups]:->llama(avg)+tfidf.(llama+wce(tfidf)):[def]	0.713	36.34
lr	20newsgroups	(11314, 22920)	[lr:20newsgroups]:->count+llama(avg):[def]	0.711	787.88
svm	20newsgroups	(11314, 22920)	[svm:20newsgroups]:->count+llama(avg):[def]	0.711	64.75
svm	20newsgroups	(11314, 4116)	[svm:20newsgroups]:->llama(avg)+tfidf.(llama+wce(tfidf)):[def]	0.702	133.89
lr	arxiv	(11962, 18997)	[lr:arxiv]:->count+llama(avg):[def]	0.785	466.42
svm	arxiv	(11962, 18997)	[svm:arxiv]:->count+llama(avg):[def]	0.783	59.19
lr	arxiv	(11962, 18948)	[lr:arxiv]:->tfidf+llama(avg):[def]	0.768	151.81
svm	arxiv	(11962, 18948)	[svm:arxiv]:->tfidf+llama(avg):[def]	0.765	23.97
lr	arxiv	(11962, 21054)	[lr:arxiv]:->tfidf+tfidf.(llama+wce(tfidf))+tfidf.(llama+wce(tfidf)):[def]	0.764	366.72
lr	arxiv_protoformer	(82500, 34540)	[lr:arxiv_protoformer]:->tfidf+llama(avg):[def]	0.857	2569.18
lr	arxiv_protoformer	(82500, 2048)	[lr:arxiv_protoformer]:->llama(avg):[def]	0.856	184.94
lr	arxiv_protoformer	(82500, 1536)	[lr:arxiv_protoformer]:->deepseek(avg):[def]	0.855	154.60
svm	arxiv_protoformer	(82500, 1536)	[svm:arxiv_protoformer]:->deepseek(avg):[def]	0.850	238.68
svm	arxiv_protoformer	(82500, 32930)	[svm:arxiv_protoformer]:->tfidf+deepseek(avg):[def]	0.850	1571.72
svm	bbc-news	(1229, 6358)	[svm:bbc-news]:->tfidf[fasttext]:[def]	0.989	0.86
svm	bbc-news	(1229, 6358)	[svm:bbc-news]:->tfidf[glove]:[def]	0.989	0.85
svm	bbc-news	(1229, 6358)	[svm:bbc-news]:->tfidf[word2vec]:[def]	0.989	0.88
svm	bbc-news	(1229, 768)	[svm:bbc-news]:->gpt2(avg):[def]	0.989	1.72
svm	bbc-news	(1229, 768)	[svm:bbc-news]:->tfidf->LSA[bert].(bert):[def]	0.989	3.29
lr	imdb	(25000, 30664)	[lr:imdb]:->tfidf+roberta(avg):[def]	0.925	198.31
lr	imdb	(25000, 2048)	[lr:imdb]:->llama(avg):[def]	0.923	27.83
svm	imdb	(25000, 768)	[svm:imdb]:->roberta(avg):[def]	0.922	9.38
lr	imdb	(25000, 32714)	[lr:imdb]:->tfidf+llama(avg):[def]	0.921	489.62
lr	imdb	(25000, 768)	[lr:imdb]:->roberta(avg):[def]	0.921	7.65
svm	ohsumed	(24061, 18940)	[svm:ohsumed]:->tfidf[fasttext]:[def]	0.706	13.22
svm	ohsumed	(24061, 18940)	[svm:ohsumed]:->tfidf[glove]:[def]	0.706	13.44
svm	ohsumed	(24061, 18940)	[svm:ohsumed]:->tfidf[word2vec]:[def]	0.706	14.11
svm	ohsumed	(24061, 19240)	[svm:ohsumed]:->tfidf+fasttext(avg):[def]	0.704	20.48
svm	ohsumed	(24061, 19240)	[svm:ohsumed]:->tfidf+word2vec(avg):[def]	0.703	17.97
svm	rcv1	(23149, 28679)	[svm:rcv1]:->tfidf+roberta(avg):[def]	0.826	5519.30
lr	rcv1	(23149, 30931)	[lr:rcv1]:->count+llama(avg):[def]	0.819	11126.84
svm	rcv1	(23149, 26758)	[svm:rcv1]:->tfidf+word2vec(avg):[def]	0.816	4421.36
svm	rcv1	(23149, 26758)	[svm:rcv1]:->tfidf+fasttext(avg):[def]	0.815	4174.40
svm	rcv1	(23149, 26758)	[svm:rcv1]:->tfidf+glove(avg):[def]	0.812	4257.52
svm	reuters21578	(9603, 9556)	[svm:reuters21578]:->tfidf+word2vec(avg):[def]	0.878	15.78
svm	reuters21578	(9603, 9556)	[svm:reuters21578]:->tfidf+fasttext(avg):[def]	0.877	16.72
svm	reuters21578	(9603, 9256)	[svm:reuters21578]:->tfidf[fasttext]:[def]	0.877	5.63
svm	reuters21578	(9603, 9256)	[svm:reuters21578]:->tfidf[glove]:[def]	0.877	5.82
svm	reuters21578	(9603, 9256)	[svm:reuters21578]:->tfidf[word2vec]:[def]	0.877	5.77

When we look at the top 5 results by Classifier and Dataset, as illustrated in the tables above (the first for the Macro-F1 results and the second for the Micro-F1 results), we can see that both the SVM and Logistic Regression Classifiers are both effective, and that generally the text vectorization techniques (TF-IDF or Count), dominate the best results, with some limited exceptions.

The model build times (the timelapse column) are very much a function of the size of the feature matrix that is used with the model (dimensions column) which is what we would expect and aligns with the dimensionality complexity challenges that are fundamental to machine learning, hence the emphasis on feature reduction techniques in the literature which can have a significant impact on computational resource requirements, i.e. model training times.

7.1.2 By Representation Form

When we look at the various representation forms, and evaluate their performance across the different classifiers (SVM, LR and NB), we find that, perhaps surprisingly, that the *lsa* and *solo* representation forms,

(solo being the representation of the underlying dataset using the language model in question), both individually and when combined (concatenated) with the WCEs, are the best performing options in terms of model efficacy – again on average across all of the different datasets. The lsa representation form, given that is inherently a form of feature reduction as well, is very efficient in terms of model construction times, i.e. from a timelapse perspective.

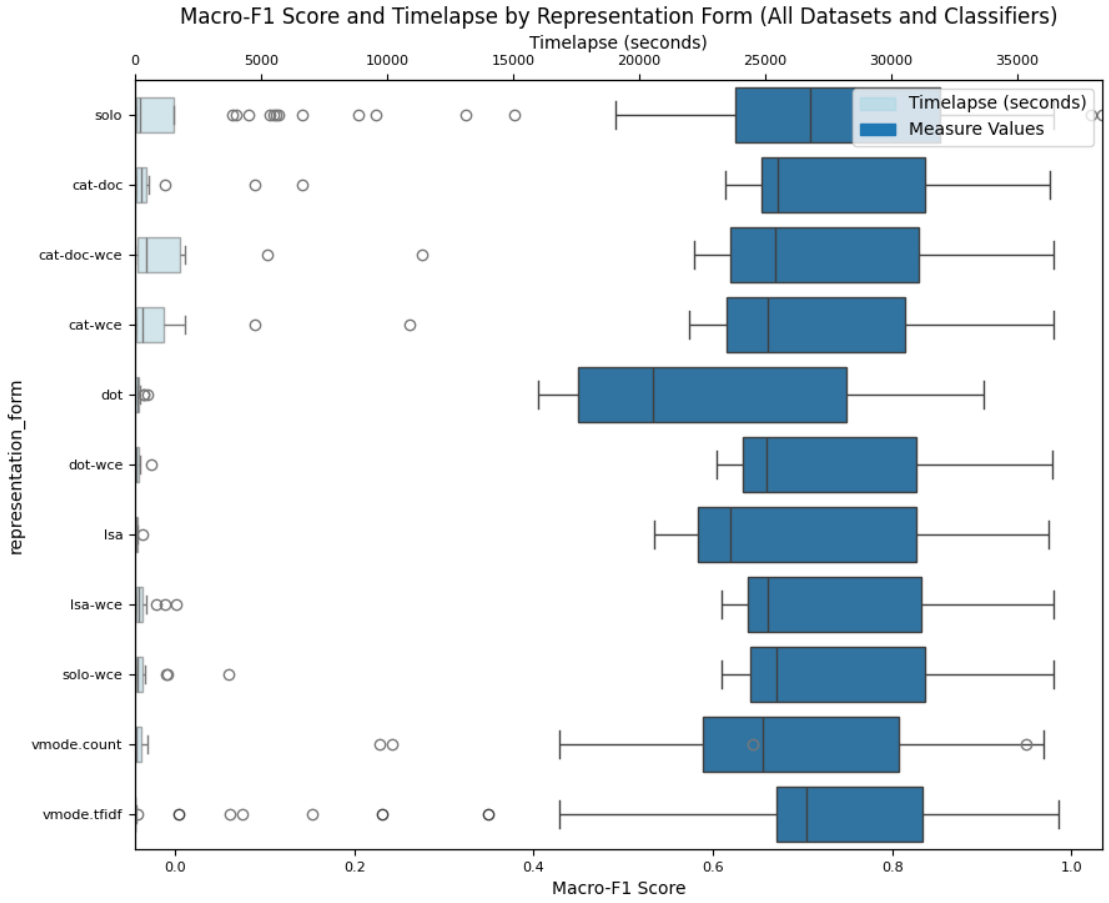


Figure 11: ML Classifier Representation Form Results: Macro-F1

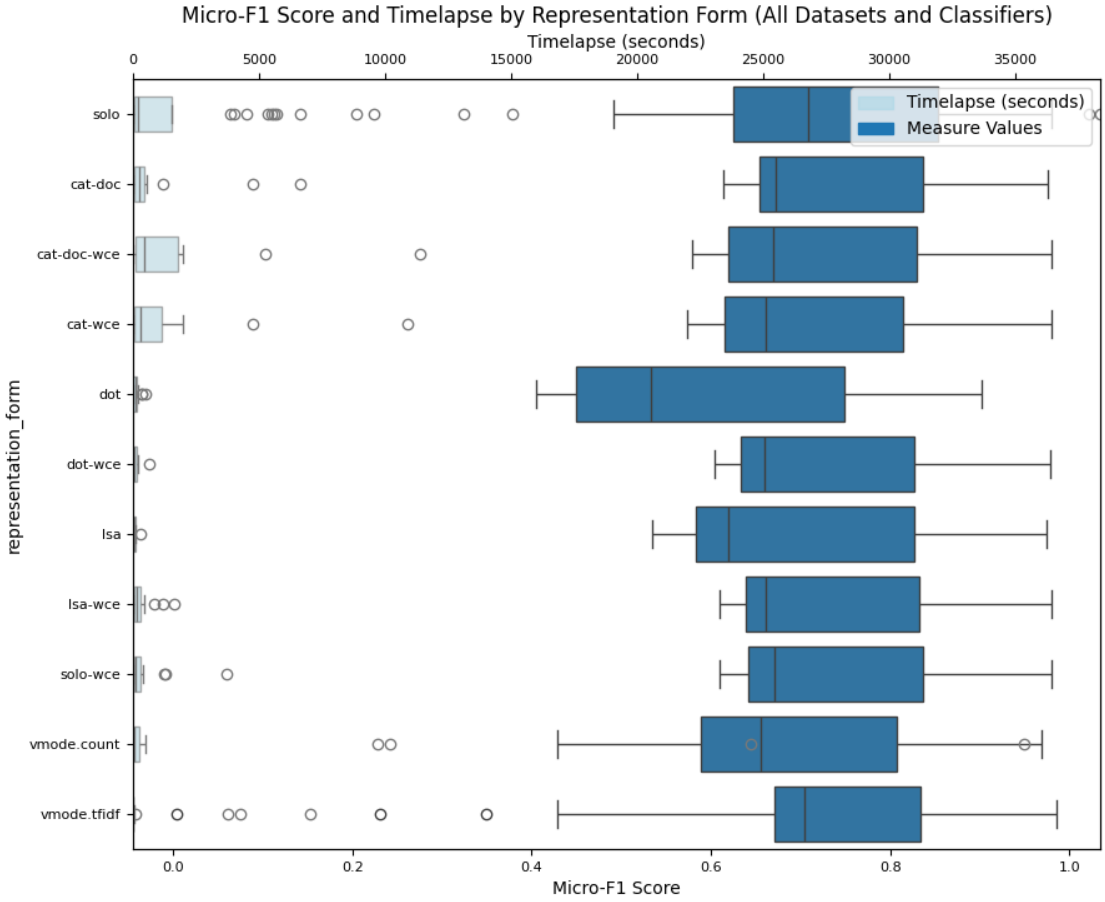


Figure 12: ML Classifier Representation Form Results: Micro-F1

In looking at the breakout of representation form by dataset, as shown in Tables 3 and 4 below, we can see that solo-wce and lsa-wce, along with simply solo and wce, perform quite well relative to the best Macro-F1 and Micro-F1 scores. In particular when we look at the largest datasets, we see solo representation forms, which again are the underlying dataset represented by the language model embeddings (rather than a text vectorization method) are very close approximations of the best values with just a fraction of the model construction times, representing a significant computational cost savings.

For RCV-1 Macro-F1 for *cat-doc*, which is the vectorized text combined (concatenated) with the embedding representation of the dataset (pooled), we have .648 versus the .633 for *solo* whereas the mean model construction time is only 392.63 seconds versus the 5686.60 seconds for the cat-doc model, almost 15x the run time basically – which is 6.5 minutes versus almost an hour and a half. For the arxiv_protoformer dataset, the best value for Macro-F1 is also *cat-doc*, where we get a max value of .857 versus the .856 for *solo*, whereas the mean model construction time is only 77.7 seconds, just over a minute, versus the 887 seconds which is almost 15 minutes, again almost a 15x the run time for the model construction.

Table 3: ML Classifiers, Multi-Label Datasets Macro-F1 Values by Representation Form

dataset	representation	measure	max	mean	media	min	st	coun	timelapse
reuters21578	cat-doc	final-te-macro-f1	0.606	0.530	0.540	0.306	0.063	36	106.87
reuters21578	vmode.tfidf	final-te-macro-f1	0.600	0.468	0.565	0.239	0.160	30	13.09
reuters21578	cat-doc-wce	final-te-macro-f1	0.587	0.500	0.519	0.246	0.069	36	218.73
reuters21578	solo-wce	final-te-macro-f1	0.584	0.528	0.530	0.472	0.022	40	138.81
reuters21578	vmode.count	final-te-macro-f1	0.583	0.461	0.508	0.069	0.121	30	71.16
reuters21578	lsa-wce	final-te-macro-f1	0.568	0.518	0.519	0.379	0.029	40	148.77
reuters21578	lsa	final-te-macro-f1	0.567	0.496	0.495	0.387	0.048	40	50.43
reuters21578	solo	final-te-macro-f1	0.567	0.435	0.462	0.221	0.102	20	52.00
reuters21578	dot	final-te-macro-f1	0.564	0.434	0.436	0.203	0.074	40	74.49
reuters21578	dot-wce	final-te-macro-f1	0.563	0.519	0.521	0.385	0.029	40	75.76
reuters21578	cat-wce	final-te-macro-f1	0.561	0.496	0.517	0.204	0.072	40	141.75
rcv1	cat-doc	final-te-macro-f1	0.648	0.565	0.580	0.380	0.055	36	5686.60
rcv1	solo	final-te-macro-f1	0.633	0.503	0.504	0.382	0.070	18	392.63
rcv1	vmode.tfidf	final-te-macro-f1	0.604	0.431	0.571	0.113	0.226	27	5006.67
rcv1	vmode.count	final-te-macro-f1	0.591	0.463	0.499	0.205	0.113	27	14796.16
rcv1	lsa	final-te-macro-f1	0.586	0.489	0.477	0.421	0.048	36	203.08
rcv1	cat-doc-wce	final-te-macro-f1	0.528	0.464	0.475	0.166	0.062	36	8329.86
rcv1	solo-wce	final-te-macro-f1	0.526	0.486	0.486	0.452	0.019	36	830.58
rcv1	cat-wce	final-te-macro-f1	0.516	0.456	0.470	0.167	0.062	36	7806.45
rcv1	lsa-wce	final-te-macro-f1	0.513	0.481	0.480	0.432	0.020	36	809.75
rcv1	dot-wce	final-te-macro-f1	0.512	0.478	0.479	0.432	0.020	36	431.93
rcv1	dot	final-te-macro-f1	0.500	0.397	0.403	0.218	0.076	36	348.67
ohsumed	vmode.tfidf	final-te-macro-f1	0.691	0.494	0.663	0.123	0.254	30	16.44
ohsumed	cat-doc	final-te-macro-f1	0.690	0.618	0.615	0.528	0.038	40	367.95
ohsumed	cat-doc-wce	final-te-macro-f1	0.671	0.619	0.621	0.563	0.027	40	1166.05
ohsumed	cat-wce	final-te-macro-f1	0.671	0.616	0.622	0.566	0.031	40	806.73
ohsumed	lsa-wce	final-te-macro-f1	0.656	0.602	0.598	0.563	0.026	40	453.72
ohsumed	solo-wce	final-te-macro-f1	0.651	0.601	0.605	0.554	0.026	40	630.59
ohsumed	dot-wce	final-te-macro-f1	0.649	0.594	0.598	0.548	0.026	40	103.40
ohsumed	lsa	final-te-macro-f1	0.629	0.511	0.503	0.402	0.064	40	55.10
ohsumed	vmode.count	final-te-macro-f1	0.628	0.557	0.585	0.405	0.063	30	206.46
ohsumed	solo	final-te-macro-f1	0.616	0.466	0.445	0.410	0.060	20	77.04
ohsumed	dot	final-te-macro-f1	0.484	0.389	0.399	0.269	0.061	40	180.23
arxiv	cat-doc	final-te-macro-f1	0.784	0.660	0.682	0.235	0.098	36	155.20
arxiv	solo	final-te-macro-f1	0.769	0.541	0.507	0.335	0.126	20	37.39
arxiv	solo-wce	final-te-macro-f1	0.759	0.664	0.656	0.608	0.034	40	99.21
arxiv	cat-doc-wce	final-te-macro-f1	0.757	0.636	0.647	0.018	0.115	36	435.75
arxiv	vmode.tfidf	final-te-macro-f1	0.741	0.470	0.676	0.000	0.336	30	12.45
arxiv	lsa	final-te-macro-f1	0.716	0.594	0.590	0.500	0.058	40	39.09
arxiv	cat-wce	final-te-macro-f1	0.707	0.625	0.641	0.008	0.108	40	312.51
arxiv	lsa-wce	final-te-macro-f1	0.705	0.653	0.648	0.617	0.024	40	110.25
arxiv	dot-wce	final-te-macro-f1	0.704	0.650	0.644	0.603	0.025	40	53.71
arxiv	vmode.count	final-te-macro-f1	0.689	0.588	0.635	0.437	0.097	30	104.40
arxiv	dot	final-te-macro-f1	0.556	0.442	0.468	0.208	0.090	40	62.82

Table 4: ML Classifiers, Single-Label Datasets Macro-F1 Values by Representation Form

dataset	representation	measure	max	mean	media	min	st	coun	timelapse
imdb	cat-doc	final-te-macro-f1	0.925	0.884	0.888	0.836	0.024	40	156.40
imdb	solo	final-te-macro-f1	0.923	0.888	0.891	0.852	0.026	20	13.47
imdb	cat-doc-wce	final-te-macro-f1	0.897	0.844	0.852	0.612	0.056	40	247.18
imdb	solo-wce	final-te-macro-f1	0.896	0.862	0.860	0.832	0.016	40	123.66
imdb	lsa	final-te-macro-f1	0.889	0.872	0.874	0.855	0.010	40	33.94
imdb	vmode.tfidf	final-te-macro-f1	0.889	0.864	0.875	0.827	0.024	30	13.73
imdb	vmode.count	final-te-macro-f1	0.876	0.835	0.836	0.763	0.025	30	86.05
imdb	dot-wce	final-te-macro-f1	0.862	0.842	0.842	0.822	0.011	40	28.54
imdb	lsa-wce	final-te-macro-f1	0.862	0.848	0.849	0.827	0.009	40	140.87
imdb	cat-wce	final-te-macro-f1	0.861	0.819	0.845	0.402	0.086	40	250.59
imdb	dot	final-te-macro-f1	0.857	0.787	0.783	0.685	0.048	40	53.64
bbc-news	cat-doc-wce	final-te-macro-f1	0.989	0.958	0.980	0.337	0.107	36	2.47
bbc-news	solo-wce	final-te-macro-f1	0.989	0.960	0.980	0.273	0.112	40	3.06
bbc-news	solo	final-te-macro-f1	0.988	0.975	0.977	0.960	0.007	20	2.23
bbc-news	lsa	final-te-macro-f1	0.988	0.973	0.976	0.945	0.011	40	4.77
bbc-news	vmode.tfidf	final-te-macro-f1	0.988	0.978	0.980	0.962	0.009	30	1.18
bbc-news	cat-wce	final-te-macro-f1	0.985	0.961	0.980	0.380	0.095	40	3.91
bbc-news	dot-wce	final-te-macro-f1	0.985	0.977	0.977	0.961	0.006	40	2.51
bbc-news	lsa-wce	final-te-macro-f1	0.985	0.957	0.977	0.385	0.095	40	5.34
bbc-news	cat-doc	final-te-macro-f1	0.985	0.973	0.977	0.938	0.009	36	2.27
bbc-news	vmode.count	final-te-macro-f1	0.981	0.964	0.965	0.929	0.013	30	1.53
bbc-news	dot	final-te-macro-f1	0.980	0.890	0.890	0.654	0.075	40	2.83
arxiv_protoformer	cat-doc	final-te-macro-f1	0.857	0.817	0.827	0.775	0.024	41	887.01
arxiv_protoformer	solo	final-te-macro-f1	0.856	0.796	0.788	0.748	0.032	20	77.77
arxiv_protoformer	cat-doc-wce	final-te-macro-f1	0.846	0.823	0.826	0.785	0.017	40	1936.78
arxiv_protoformer	solo-wce	final-te-macro-f1	0.845	0.828	0.829	0.817	0.007	49	1600.68
arxiv_protoformer	cat-wce	final-te-macro-f1	0.840	0.819	0.823	0.781	0.016	39	1627.69
arxiv_protoformer	vmode.tfidf	final-te-macro-f1	0.840	0.823	0.822	0.806	0.013	38	28.83
arxiv_protoformer	lsa-wce	final-te-macro-f1	0.838	0.827	0.828	0.804	0.007	58	687.20
arxiv_protoformer	lsa	final-te-macro-f1	0.832	0.809	0.808	0.787	0.013	49	90.05
arxiv_protoformer	dot-wce	final-te-macro-f1	0.831	0.822	0.821	0.813	0.005	49	129.36
arxiv_protoformer	vmode.count	final-te-macro-f1	0.819	0.798	0.805	0.768	0.016	37	8694.00
arxiv_protoformer	dot	final-te-macro-f1	0.783	0.727	0.743	0.601	0.049	49	190.23
20newsgroups	cat-doc	final-te-macro-f1	0.705	0.630	0.643	0.363	0.064	36	130.05
20newsgroups	cat-doc-wce	final-te-macro-f1	0.703	0.619	0.641	0.058	0.105	36	258.76
20newsgroups	solo-wce	final-te-macro-f1	0.701	0.647	0.641	0.618	0.021	40	62.59
20newsgroups	solo	final-te-macro-f1	0.684	0.609	0.617	0.500	0.044	20	33.80
20newsgroups	cat-wce	final-te-macro-f1	0.681	0.618	0.635	0.311	0.074	40	140.84
20newsgroups	lsa-wce	final-te-macro-f1	0.680	0.644	0.642	0.601	0.019	40	73.99
20newsgroups	vmode.tfidf	final-te-macro-f1	0.679	0.649	0.667	0.583	0.034	30	9.37
20newsgroups	dot-wce	final-te-macro-f1	0.678	0.641	0.636	0.606	0.019	40	36.30
20newsgroups	lsa	final-te-macro-f1	0.661	0.589	0.598	0.515	0.048	40	43.36
20newsgroups	vmode.count	final-te-macro-f1	0.630	0.562	0.602	0.229	0.081	30	89.89
20newsgroups	dot	final-te-macro-f1	0.605	0.446	0.424	0.292	0.087	40	57.37

For the Micro-F1 scores, we see a similar pattern. The best performers are again the text vectorized representations, *cat-doc* which for RCV-1 dataset is .826, whereas the solo representation form is .800, only off by a small fraction of a percentage point and the runtime for the model construction is 5686 seconds for cat-doc and only 392 seconds for the solo representation form, again almost a 15x run-time performance hit for the model construction. Similarly, with the arxiv_protoformer dataset, the Micro-F1 best score for cat-doc, which again represents the max score for the model, is .857 as compared to .856 for the solo representation form, almost the same. The runtimes for the model construction are 887 seconds for the cat-doc representation form as compared to just 77 for the solo representation, approximate 11x the build time.

Clearly this points to the language models as being capable of performing quite well as a form of feature reduction, at least with respect to text classification for machine learning classifiers.

Table 5: ML Classifiers, Multi-Label Datasets Micro-F1 Values by Representation Form

dataset	representation	measure	max	mean	media	min	std	coun	timelaps
reuters21578	cat-doc	final-te-micro-f1	0.878	0.814	0.836	0.344	0.088	36	106.87
reuters21578	vmode.tfidf	final-te-micro-f1	0.877	0.774	0.842	0.582	0.119	30	13.09
reuters21578	lsa	final-te-micro-f1	0.861	0.803	0.805	0.680	0.041	40	50.43
reuters21578	cat-doc-wce	final-te-micro-f1	0.859	0.778	0.811	0.376	0.114	36	218.73
reuters21578	solo-wce	final-te-micro-f1	0.859	0.821	0.822	0.802	0.013	40	138.81
reuters21578	solo	final-te-micro-f1	0.857	0.728	0.736	0.487	0.096	20	52.00
reuters21578	vmode.count	final-te-micro-f1	0.852	0.729	0.821	0.275	0.156	30	71.16
reuters21578	lsa-wce	final-te-micro-f1	0.842	0.815	0.816	0.793	0.011	40	148.77
reuters21578	dot-wce	final-te-micro-f1	0.841	0.813	0.812	0.793	0.011	40	75.76
reuters21578	cat-wce	final-te-micro-f1	0.840	0.780	0.812	0.359	0.102	40	141.75
reuters21578	dot	final-te-micro-f1	0.807	0.718	0.733	0.406	0.078	40	74.49
rcv1	cat-doc	final-te-micro-f1	0.826	0.772	0.773	0.643	0.034	36	5686.60
rcv1	vmode.tfidf	final-te-micro-f1	0.812	0.722	0.777	0.570	0.103	27	5006.67
rcv1	solo	final-te-micro-f1	0.800	0.686	0.691	0.583	0.062	18	392.63
rcv1	lsa	final-te-micro-f1	0.792	0.696	0.699	0.634	0.042	36	203.08
rcv1	cat-doc-wce	final-te-micro-f1	0.790	0.744	0.753	0.487	0.051	36	8329.86
rcv1	solo-wce	final-te-micro-f1	0.790	0.752	0.748	0.729	0.016	36	830.58
rcv1	vmode.count	final-te-micro-f1	0.780	0.681	0.742	0.358	0.121	27	14796.16
rcv1	cat-wce	final-te-micro-f1	0.778	0.738	0.746	0.446	0.053	36	7806.45
rcv1	lsa-wce	final-te-micro-f1	0.773	0.747	0.745	0.715	0.014	36	809.75
rcv1	dot-wce	final-te-micro-f1	0.761	0.741	0.741	0.714	0.012	36	431.93
rcv1	dot	final-te-micro-f1	0.671	0.583	0.605	0.362	0.084	36	348.67
ohsumed	vmode.tfidf	final-te-micro-f1	0.706	0.567	0.682	0.298	0.178	30	16.44
ohsumed	cat-doc	final-te-micro-f1	0.704	0.645	0.646	0.587	0.031	40	367.95
ohsumed	cat-wce	final-te-micro-f1	0.691	0.643	0.642	0.597	0.027	40	806.73
ohsumed	cat-doc-wce	final-te-micro-f1	0.690	0.644	0.644	0.602	0.025	40	1166.05
ohsumed	lsa-wce	final-te-micro-f1	0.676	0.635	0.638	0.596	0.023	40	453.72
ohsumed	solo-wce	final-te-micro-f1	0.672	0.634	0.637	0.597	0.021	40	630.59
ohsumed	dot-wce	final-te-micro-f1	0.668	0.630	0.630	0.593	0.021	40	103.40
ohsumed	lsa	final-te-micro-f1	0.659	0.563	0.555	0.448	0.059	40	55.10
ohsumed	vmode.count	final-te-micro-f1	0.652	0.589	0.613	0.426	0.057	30	206.46
ohsumed	solo	final-te-micro-f1	0.638	0.516	0.500	0.465	0.051	20	77.04
ohsumed	dot	final-te-micro-f1	0.534	0.441	0.454	0.310	0.065	40	180.23
arxiv	cat-doc	final-te-micro-f1	0.785	0.659	0.685	0.151	0.110	36	155.20
arxiv	cat-doc-wce	final-te-micro-f1	0.764	0.642	0.656	0.026	0.115	36	435.75
arxiv	solo	final-te-micro-f1	0.763	0.522	0.489	0.318	0.134	20	37.39
arxiv	solo-wce	final-te-micro-f1	0.761	0.669	0.663	0.610	0.034	40	99.21
arxiv	vmode.tfidf	final-te-micro-f1	0.745	0.469	0.667	0.000	0.335	30	12.45
arxiv	lsa	final-te-micro-f1	0.714	0.580	0.574	0.471	0.065	40	39.09
arxiv	cat-wce	final-te-micro-f1	0.709	0.629	0.646	0.020	0.108	40	312.51
arxiv	lsa-wce	final-te-micro-f1	0.707	0.658	0.655	0.619	0.023	40	110.25
arxiv	dot-wce	final-te-micro-f1	0.705	0.655	0.648	0.604	0.024	40	53.71
arxiv	vmode.count	final-te-micro-f1	0.691	0.583	0.642	0.409	0.110	30	104.40
arxiv	dot	final-te-micro-f1	0.548	0.423	0.436	0.195	0.094	40	62.82

Table 6: ML Classifiers, Single-Label Datasets Micro-F1 Values by Representation Form

dataset	representation	measure	max	mean	media	min	std	coun	timelaps
imdb	cat-doc	final-te-micro-f1	0.925	0.884	0.888	0.836	0.024	40	156.40
imdb	solo	final-te-micro-f1	0.923	0.888	0.891	0.852	0.026	20	13.47
imdb	cat-doc-wce	final-te-micro-f1	0.897	0.846	0.853	0.650	0.048	40	247.18
imdb	solo-wce	final-te-micro-f1	0.896	0.863	0.860	0.832	0.016	40	123.66
imdb	lsa	final-te-micro-f1	0.889	0.872	0.874	0.855	0.010	40	33.94
imdb	vmode.tfidf	final-te-micro-f1	0.889	0.866	0.875	0.827	0.023	29	14.16
imdb	vmode.count	final-te-micro-f1	0.876	0.835	0.836	0.766	0.025	30	86.05
imdb	dot-wce	final-te-micro-f1	0.862	0.842	0.842	0.823	0.011	40	28.54
imdb	lsa-wce	final-te-micro-f1	0.862	0.848	0.849	0.827	0.009	40	140.87
imdb	cat-wce	final-te-micro-f1	0.861	0.825	0.845	0.531	0.064	40	250.59
imdb	dot	final-te-micro-f1	0.857	0.787	0.783	0.685	0.048	40	53.64
bbc-news	cat-doc-wce	final-te-micro-f1	0.989	0.959	0.981	0.368	0.103	36	2.47
bbc-news	lsa	final-te-micro-f1	0.989	0.974	0.977	0.946	0.011	40	4.77
bbc-news	solo	final-te-micro-f1	0.989	0.976	0.977	0.962	0.007	20	2.23
bbc-news	solo-wce	final-te-micro-f1	0.989	0.961	0.981	0.322	0.104	40	3.06
bbc-news	vmode.tfidf	final-te-micro-f1	0.989	0.978	0.981	0.962	0.010	30	1.18
bbc-news	cat-doc	final-te-micro-f1	0.985	0.974	0.977	0.935	0.010	36	2.27
bbc-news	cat-wce	final-te-micro-f1	0.985	0.963	0.981	0.479	0.080	40	3.91
bbc-news	dot-wce	final-te-micro-f1	0.985	0.977	0.977	0.962	0.006	40	2.51
bbc-news	lsa-wce	final-te-micro-f1	0.985	0.958	0.977	0.391	0.094	40	5.34
bbc-news	dot	final-te-micro-f1	0.981	0.891	0.893	0.663	0.074	40	2.83
bbc-news	vmode.count	final-te-micro-f1	0.981	0.965	0.966	0.931	0.013	30	1.53
arxiv_protoformer	cat-doc	final-te-micro-f1	0.857	0.817	0.827	0.775	0.024	41	887.01
arxiv_protoformer	solo	final-te-micro-f1	0.856	0.796	0.789	0.749	0.032	20	77.77
arxiv_protoformer	cat-doc-wce	final-te-micro-f1	0.845	0.823	0.826	0.785	0.017	40	1936.78
arxiv_protoformer	solo-wce	final-te-micro-f1	0.844	0.828	0.829	0.818	0.006	49	1600.68
arxiv_protoformer	cat-wce	final-te-micro-f1	0.841	0.819	0.823	0.780	0.016	39	1627.69
arxiv_protoformer	vmode.tfidf	final-te-micro-f1	0.840	0.823	0.822	0.808	0.012	38	28.83
arxiv_protoformer	lsa-wce	final-te-micro-f1	0.838	0.827	0.829	0.804	0.007	58	687.20
arxiv_protoformer	dot-wce	final-te-micro-f1	0.832	0.823	0.822	0.813	0.005	49	129.36
arxiv_protoformer	lsa	final-te-micro-f1	0.831	0.809	0.808	0.787	0.013	49	90.05
arxiv_protoformer	vmode.count	final-te-micro-f1	0.819	0.798	0.805	0.769	0.016	37	8694.00
arxiv_protoformer	dot	final-te-micro-f1	0.785	0.729	0.745	0.607	0.048	49	190.23
20newsgroups	cat-doc-wce	final-te-micro-f1	0.715	0.627	0.650	0.078	0.105	36	258.76
20newsgroups	solo-wce	final-te-micro-f1	0.713	0.656	0.652	0.620	0.023	40	62.59
20newsgroups	cat-doc	final-te-micro-f1	0.711	0.638	0.652	0.310	0.073	36	130.05
20newsgroups	lsa-wce	final-te-micro-f1	0.693	0.653	0.652	0.595	0.021	40	73.99
20newsgroups	solo	final-te-micro-f1	0.693	0.617	0.626	0.504	0.046	20	33.80
20newsgroups	cat-wce	final-te-micro-f1	0.693	0.623	0.646	0.283	0.081	40	140.84
20newsgroups	dot-wce	final-te-micro-f1	0.691	0.650	0.645	0.604	0.021	40	36.30
20newsgroups	vmode.tfidf	final-te-micro-f1	0.688	0.665	0.674	0.614	0.024	30	9.37
20newsgroups	lsa	final-te-micro-f1	0.669	0.594	0.605	0.519	0.050	40	43.36
20newsgroups	vmode.count	final-te-micro-f1	0.647	0.568	0.609	0.186	0.095	30	89.89
20newsgroups	dot	final-te-micro-f1	0.615	0.452	0.430	0.299	0.088	40	57.37

7.2 Neural Classifiers (Deep Learning)

Within the context of our neural classifier model analysis, we look at performance results by classifier and by (pretrained) language models and integrate the results with model training times (timelapse field) into

one graphic, focusing on the macro-f1 and micro-F1 metrics which are good, industry standard metrics to measure the performance of a classifier in both multi-label and single-label scenarios.[34]

The primary questions we are looking at answering here are:

i. *pretrained embedding performance comparison*: which pretrained language models perform the best and by how much? Do certain pretrained embeddings perform better on certain datasets? If so why? If not why not?

ii. *neural architecture performance comparison*: which neural architectures perform the best in terms of model macro-f1 and micro-f1 metrics? Is this the same across all datasets or does it differ?

iii. *system resource performance comparison*: what are the computational resource requirements required to train each classifier, i.e. how many GPUs are required, and for how long, to fine-tune each model with each dataset? What is the most effective model that takes the minimal amount of computing resources to train while still retaining competitive results?

7.2.1 By Language Model

When looking at the on average best performing language models, of the model variants that we have chosen for this work, we find that XLNet and RoBERTa stand out as top performers, with the transformer models outperforming their static counterparts, even when they were assisted with WCEs, again on average. We can see this illustrated in the language model summary macro-f1 and micro-f1 charts (box plots) below. With the box plot graphic, which looks at the left y-axis which shows the measurement value between 0 and 100 (%), with the center line representing the median value, the shaded box area the interquartile range between the 25th and 75th percentiles, and outliers represented as unshaded circles. Overlaid on these graphics is the timelapse value which shows how it took, in seconds, to build (train) the model.

We also see, again on average across all the datasets, relatively the same performance for the three static word embedding language models we test with (GloVe, Word2Vec and fastText models). Given that they are of comparable size and are trained basically on the same underlying data (Web Common Crawl), this is not a surprising result and speaks to how comparable the different training methods are to each other when used in this type of neural model / text classification setting. We see the best performance across all the datasets that we tested from the XLNet and RoBERTa language models, with BERT, GPT2 and DeepSeek coming in very close behind those two frontrunners.

[34] Specific formulas for metric calculation can be found in the Experimental Setup section of this work.

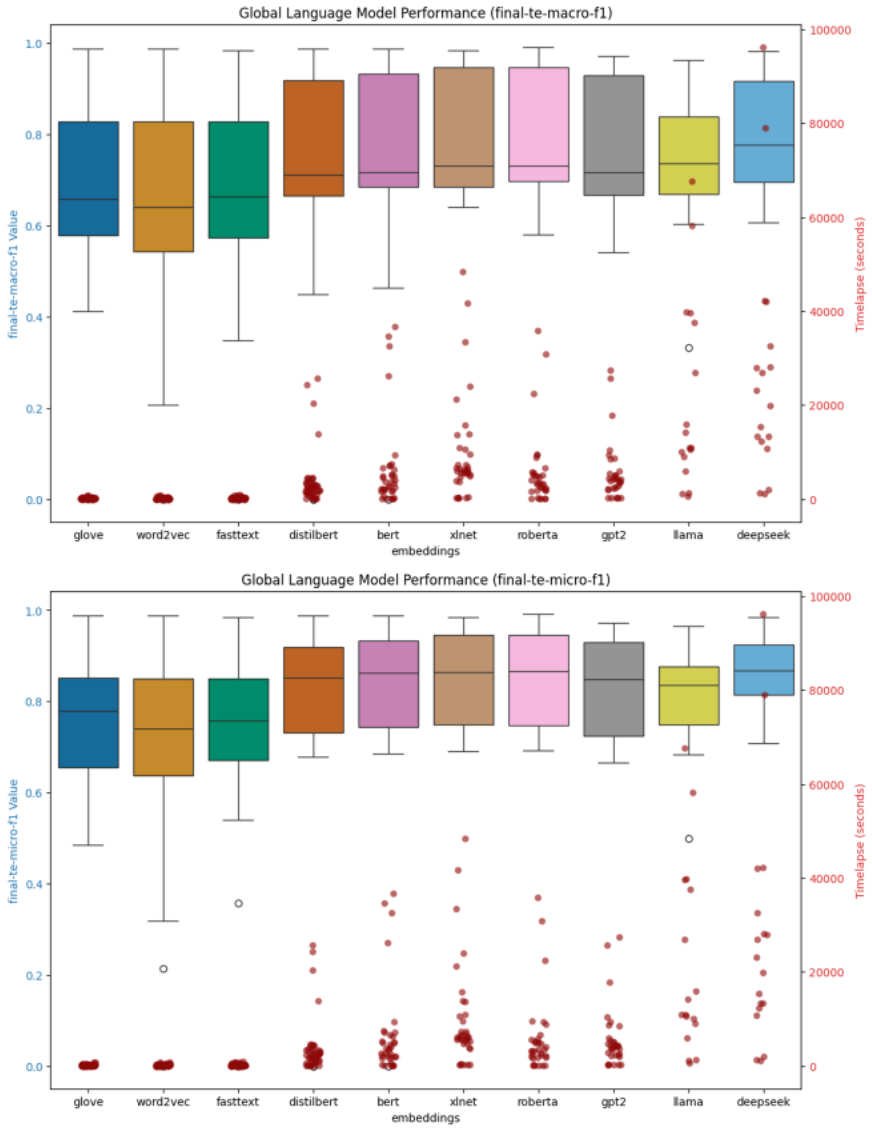


Figure 13: Neural Classifier Language Model Type Performance Summary

Notably however, from a computational resource requirements perspective (the timelapse, secondary y-axis data on the right), the static models are significantly cheaper to build. According to our analysis, the computational resource requirements of the transformer models are 50x those of their static counterparts, and this does not take into account the 4 to 1 GPU ratio that the model training of the transformers used. *This accounts for some 200x more computational resources required by the transformer model training versus the static model in our analysis.*

Also, it's also important to understand that this computational costs is not just a one-time cost, in the event that you want your model to be updated on an ongoing basis, which is the case in any operational setting, that represents a 50x incremental resource cost for each and every model training update on an ongoing basis which, over time, geometrically expands the gap resource wise between what would be required to train a static embedding model versus a transformer, dynamic model.

Model Type	Avg Training Time (seconds)	Average Training Time (minutes)
Static	182.8344083	3.047240138
Transformer	9978.987402	166.3164567
	Training time Factor:	***54.5793732***

7.2.2 By Classifier

When we look at the data in aggregate by Classifier, we see the same trend from another perspective – namely that the HuggingFace transformer classifiers perform better than their static counterparts, but their training cost is significantly higher.

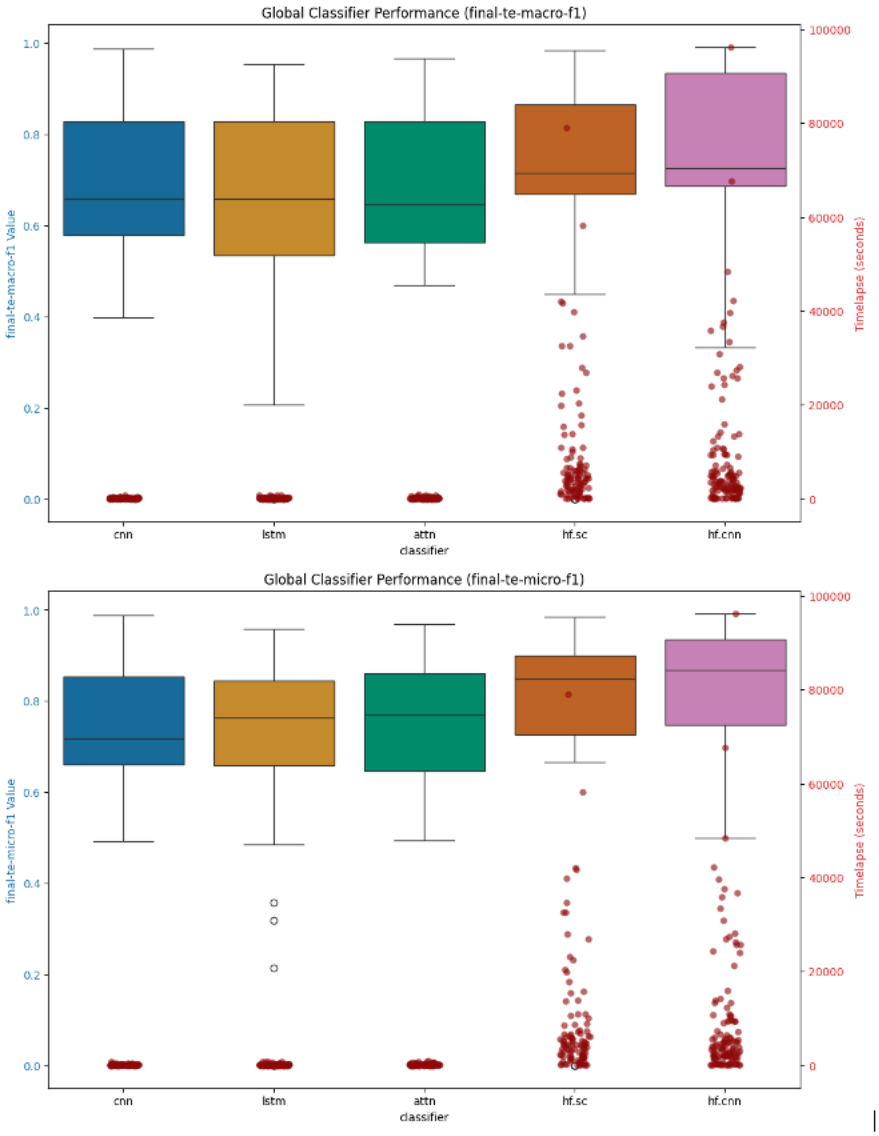


Figure 14: Neural Model Classifier Performance Summary

When we look at the detail, as illustrated in the table below, we can see that while we get a .042 and .077% increase in effectiveness of the transformer models and classifiers for Macro-F1 scores, and a .049 and .091% increase in effectiveness for Micro-F1 scores for the transformer models over the best performing static language model neural classifiers (CNN), *we nonetheless incur a ~200x training cost for model construction for these transformer models* – accounting for both the increased time in training the models as well as the fact that the transformer models were trained on 4 GPUs while the static models on just a single GPU.

classifier	measure	mean	median	count	Timelapse
lstm	final-te-macro-f1	0.664	0.658	99	186.14
attn	final-te-macro-f1	0.691	0.646	104	197.00
cnn	final-te-macro-f1	0.700	0.659	100	164.84
hf.sc	final-te-macro-f1	0.742	0.715	101	8797.42
hf.cnn	final-te-macro-f1	**0.777**	0.726	124	9044.06
lstm	final-te-micro-f1	0.727	0.763	99	186.14
attn	final-te-micro-f1	0.752	0.771	104	197.00
cnn	final-te-micro-f1	0.759	0.716	100	164.84
hf.sc	final-te-micro-f1	0.807	0.848	101	8797.42
hf.cnn	final-te-micro-f1	**0.849**	0.867	124	9044.06

That's the difference between 2-3 minutes, which is how long on average it takes to build the static, hand crafted PyTorch models versus the 2.5 hours it takes to build the Hugging Face transformer models – across all datasets. Now while certainly there is room for improvement with respect to how we optimized the training for our transformer models, this is nonetheless a staggering difference in compute resource requirements to train the different types of classifiers, with a heavy price to pay (literally and figuratively) for the performance improvement gains.

We can also see that the custom CNN classifier head that we added to the HuggingFace transformer models performs slightly better than the built in HuggingFace Sequence Classifiers, .035% better on Macro-F1 scores and .042% better on average for the Micro-F1 scores, illustrating that performance gains can be had by adding customized, tailored layers to the transformer models optimized for specific tasks, with the CNN based approach for classification showing surprising resilience.

7.2.3 By Dataset

This is of course on average, if we drill into the performance of the different DL classifiers, driven by the underlying language model design, with the custom PyTorch CNN, LSTM and ATTN neural models designed to work with the static word embedding models (Word2Vec, FastText and GloVe), and the more sophisticated, and computationally intensive Hugging Face neural classifiers which are designed to support the transformer models (BERT, RoBERTa, DistilBERT, XLNet, GPT-2, Llama and DeepSeek), we can see that the same pattern holds for the different datasets, regardless of whether they are single-label or multi-label type classification problems.

Table 7: DL Classifier Performance by Dataset - Macro-F1 Values

dataset	classif	measure	max	mean	median	min	std	co	avg timelaps
20newsgroups	hf.sc	final-te-macro-f1	0.706	0.683	0.684	0.663	0.012	13	3843.29
20newsgroups	hf.cnn	final-te-macro-f1	0.704	0.690	0.688	0.674	0.009	13	3986.35
20newsgroups	cnn	final-te-macro-f1	0.694	0.672	0.674	0.649	0.014	12	115.84
20newsgroups	lstm	final-te-macro-f1	0.681	0.640	0.642	0.590	0.030	12	56.35
20newsgroups	attn	final-te-macro-f1	0.676	0.646	0.642	0.612	0.021	12	54.03
arxiv	hf.cnn	final-te-macro-f1	0.821	0.731	0.726	0.708	0.029	13	4516.82
arxiv	hf.sc	final-te-macro-f1	0.804	0.547	0.681	0.000	0.314	13	6640.57
arxiv	lstm	final-te-macro-f1	0.696	0.499	0.568	0.000	0.200	23	159.15
arxiv	attn	final-te-macro-f1	0.694	0.579	0.580	0.483	0.060	24	152.10
arxiv	cnn	final-te-macro-f1	0.691	0.595	0.595	0.459	0.052	24	139.24
arxiv_protoformer	hf.sc	final-te-macro-f1	0.865	0.847	0.847	0.830	0.008	13	13097.29
arxiv_protoformer	hf.cnn	final-te-macro-f1	0.864	0.848	0.847	0.839	0.006	13	15057.97
arxiv_protoformer	attn	final-te-macro-f1	0.832	0.823	0.824	0.807	0.007	12	363.89
arxiv_protoformer	cnn	final-te-macro-f1	0.828	0.804	0.807	0.764	0.019	12	276.43
arxiv_protoformer	lstm	final-te-macro-f1	0.828	0.822	0.824	0.804	0.007	11	329.44
bbc-news	hf.cnn	final-te-macro-f1	0.992	0.973	0.972	0.955	0.010	25	372.04
bbc-news	cnn	final-te-macro-f1	0.988	0.979	0.982	0.959	0.009	12	7.68
bbc-news	hf.sc	final-te-macro-f1	0.985	0.972	0.972	0.960	0.008	13	338.51
bbc-news	attn	final-te-macro-f1	0.967	0.947	0.944	0.928	0.011	12	7.91
bbc-news	lstm	final-te-macro-f1	0.955	0.931	0.927	0.909	0.015	12	7.34
imdb	hf.sc	final-te-macro-f1	0.948	0.929	0.934	0.848	0.026	13	5614.67
imdb	hf.cnn	final-te-macro-f1	0.948	0.889	0.935	0.333	0.167	13	5343.42
imdb	attn	final-te-macro-f1	0.891	0.883	0.884	0.864	0.008	12	100.11
imdb	cnn	final-te-macro-f1	0.891	0.870	0.869	0.853	0.012	12	173.42
imdb	lstm	final-te-macro-f1	0.887	0.867	0.870	0.832	0.018	12	126.45
ohsumed	hf.cnn	final-te-macro-f1	0.748	0.726	0.724	0.711	0.011	13	9569.17
ohsumed	hf.sc	final-te-macro-f1	0.745	0.715	0.712	0.676	0.021	13	8848.25
ohsumed	cnn	final-te-macro-f1	0.686	0.634	0.649	0.572	0.036	12	204.21
ohsumed	lstm	final-te-macro-f1	0.683	0.649	0.654	0.570	0.029	12	162.25
ohsumed	attn	final-te-macro-f1	0.675	0.643	0.648	0.563	0.029	12	188.86
rcv1	hf.cnn	final-te-macro-f1	0.718	0.694	0.695	0.664	0.017	13	37901.20
rcv1	hf.sc	final-te-macro-f1	0.699	0.679	0.682	0.654	0.018	8	38358.23
rcv1	attn	final-te-macro-f1	0.550	0.508	0.504	0.468	0.037	8	731.22
rcv1	lstm	final-te-macro-f1	0.535	0.478	0.487	0.403	0.056	5	789.74
rcv1	cnn	final-te-macro-f1	0.498	0.435	0.421	0.398	0.045	4	575.69
reuters21578	hf.sc	final-te-macro-f1	0.665	0.565	0.547	0.450	0.074	15	5513.30
reuters21578	hf.cnn	final-te-macro-f1	0.659	0.596	0.606	0.492	0.044	21	5680.39
reuters21578	attn	final-te-macro-f1	0.592	0.552	0.553	0.489	0.030	12	200.82
reuters21578	cnn	final-te-macro-f1	0.576	0.536	0.533	0.486	0.028	12	125.66
reuters21578	lstm	final-te-macro-f1	0.535	0.481	0.484	0.411	0.039	12	247.16

Table 8: DL Classifier Performance by Dataset - Micro-F1 Values

dataset	classif	measure	max	mean	median	min	std	cou	avg_timelaps
20newsgroups	hf.cnn	final-te-micro-f1	0.713	0.699	0.698	0.683	0.010	13	3986.35
20newsgroups	hf.sc	final-te-micro-f1	0.708	0.693	0.695	0.677	0.010	13	3843.29
20newsgroups	cnn	final-te-micro-f1	0.705	0.684	0.686	0.660	0.014	12	115.84
20newsgroups	lstm	final-te-micro-f1	0.692	0.650	0.655	0.600	0.029	12	56.35
20newsgroups	attn	final-te-micro-f1	0.685	0.655	0.651	0.619	0.020	12	54.03
arxiv	hf.cnn	final-te-micro-f1	0.826	0.739	0.733	0.709	0.028	13	4516.82
arxiv	hf.sc	final-te-micro-f1	0.812	0.553	0.685	0.000	0.318	13	6640.57
arxiv	cnn	final-te-micro-f1	0.706	0.615	0.616	0.492	0.047	24	139.24
arxiv	lstm	final-te-micro-f1	0.706	0.510	0.585	0.000	0.203	23	159.15
arxiv	attn	final-te-micro-f1	0.700	0.591	0.596	0.493	0.059	24	152.10
arxiv_protoformer	hf.sc	final-te-micro-f1	0.865	0.846	0.848	0.828	0.008	13	13097.29
arxiv_protoformer	hf.cnn	final-te-micro-f1	0.865	0.849	0.848	0.837	0.006	13	15057.97
arxiv_protoformer	attn	final-te-micro-f1	0.830	0.822	0.823	0.805	0.007	12	363.89
arxiv_protoformer	lstm	final-te-micro-f1	0.827	0.821	0.824	0.803	0.007	11	329.44
arxiv_protoformer	cnn	final-te-micro-f1	0.826	0.803	0.807	0.764	0.018	12	276.43
bbc-news	hf.cnn	final-te-micro-f1	0.992	0.974	0.973	0.958	0.009	25	372.04
bbc-news	cnn	final-te-micro-f1	0.989	0.981	0.983	0.962	0.009	12	7.68
bbc-news	hf.sc	final-te-micro-f1	0.985	0.973	0.973	0.962	0.007	13	338.51
bbc-news	attn	final-te-micro-f1	0.969	0.951	0.946	0.935	0.010	12	7.91
bbc-news	lstm	final-te-micro-f1	0.958	0.935	0.929	0.916	0.014	12	7.34
imdb	hf.sc	final-te-micro-f1	0.948	0.929	0.934	0.848	0.026	13	5614.67
imdb	hf.cnn	final-te-micro-f1	0.948	0.902	0.935	0.500	0.121	13	5343.42
imdb	attn	final-te-micro-f1	0.891	0.883	0.884	0.864	0.008	12	100.11
imdb	cnn	final-te-micro-f1	0.891	0.870	0.869	0.853	0.012	12	173.42
imdb	lstm	final-te-micro-f1	0.887	0.867	0.870	0.832	0.018	12	126.45
ohsumed	hf.cnn	final-te-micro-f1	0.770	0.746	0.748	0.734	0.009	13	9569.17
ohsumed	hf.sc	final-te-micro-f1	0.764	0.739	0.736	0.720	0.015	13	8848.25
ohsumed	cnn	final-te-micro-f1	0.714	0.673	0.677	0.621	0.028	12	204.21
ohsumed	lstm	final-te-micro-f1	0.711	0.684	0.687	0.633	0.020	12	162.25
ohsumed	attn	final-te-micro-f1	0.699	0.675	0.677	0.624	0.020	12	188.86
rcv1	hf.cnn	final-te-micro-f1	0.871	0.863	0.863	0.854	0.006	13	37901.20
rcv1	hf.sc	final-te-micro-f1	0.866	0.857	0.858	0.846	0.008	8	38358.23
rcv1	attn	final-te-micro-f1	0.803	0.769	0.771	0.728	0.032	8	731.22
rcv1	lstm	final-te-micro-f1	0.802	0.773	0.782	0.733	0.031	5	789.74
rcv1	cnn	final-te-micro-f1	0.800	0.730	0.713	0.692	0.048	4	575.69
reuters21578	hf.sc	final-te-micro-f1	0.899	0.876	0.876	0.850	0.015	15	5513.30
reuters21578	hf.cnn	final-te-micro-f1	0.897	0.886	0.887	0.868	0.008	21	5680.39
reuters21578	attn	final-te-micro-f1	0.867	0.842	0.838	0.822	0.017	12	200.82
reuters21578	cnn	final-te-micro-f1	0.856	0.840	0.838	0.823	0.010	12	125.66
reuters21578	lstm	final-te-micro-f1	0.850	0.813	0.816	0.763	0.029	12	247.16

The transformer models again perform better, with the average difference between the best transformer model and the best static neural models being .067 for Macro-F1 scores and .047 for Micro-F1 scores, and on average across all the different neural classifier models and all the different datasets we test with, the *transformer models take 46.65x the amount of time to train, which, given the 4x compute resources (GPUs) applied to the training, amounts to 186.61 times the compute resources to build the classifiers.*

7.3 Global Analysis across all Classifiers

We now look at the results across the entire range of Layer Cake tests – the machine learning classifiers, the static neural embedding classifiers, and the transformers to see what patterns emerge from the data. When we look at the entire picture, the performance of all of the classifiers against each other, we see what we

might have expected, that the transformer models perform – again on average – better than the static neural classifier models as well as their ML classifier counterparts. What is perhaps somewhat surprising is that the ML classifiers, again on average, performed just as well as the static neural models, and took significantly less training time on significantly less expensive and more available computation resources (CPUs vs. GPUs).

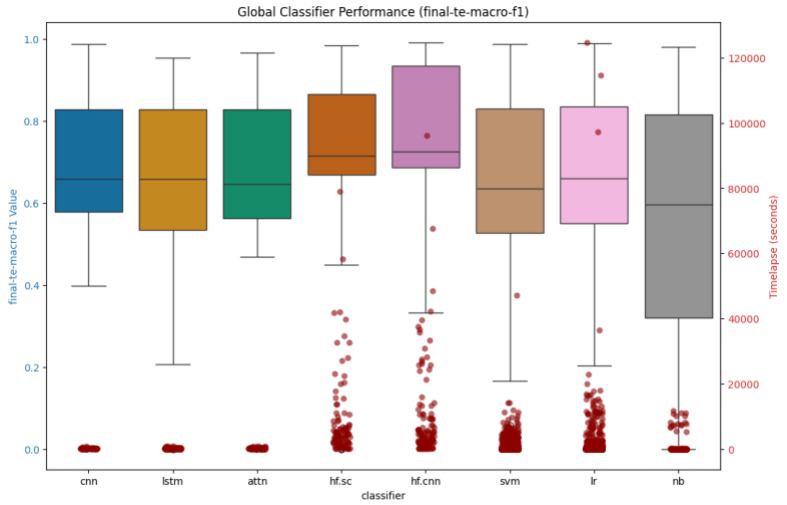


Figure 15: Global Classifier Performance - Macro-F1 Scores

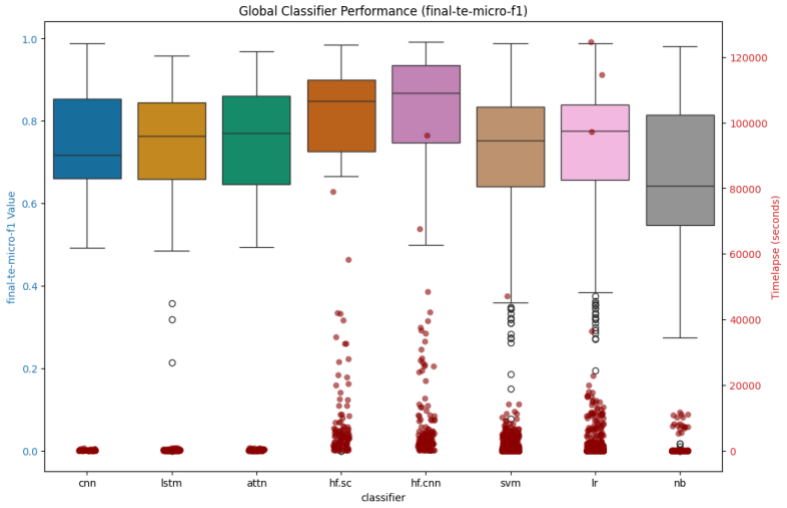


Figure 16: Global Classifier Performance - Micro-F1 Scores

When we look at the detail data in the summary table below, we can see that hf.cnn model is the most performant of all the classifiers, but the static CNN neural model is just .077% off for average Macro-F1 scores and .09% off by Micro-F1 scores, while the model training times are less than 2% of the time it takes to train the hf.cnn models, a 98% savings in compute effectively.

Table 9: Summary by all Classifiers: Macro-F1 and Micro-F1 scores

classifier	measure	mean	median	std	coun	avg timelapse
hf.cnn	final-te-macro-f1	0.777	0.726	0.143	124	9044.06
hf.sc	final-te-macro-f1	0.742	0.715	0.190	101	8797.42
cnn	final-te-macro-f1	0.700	0.659	0.157	100	164.84
attn	final-te-macro-f1	0.691	0.646	0.153	104	197.00
lr	final-te-macro-f1	0.689	0.661	0.174	1539	904.00
svm	final-te-macro-f1	0.670	0.636	0.181	1482	573.13
lstm	final-te-macro-f1	0.664	0.658	0.196	99	186.14
nb	final-te-macro-f1	0.562	0.596	0.303	176	870.42
hf.cnn	final-te-micro-f1	0.849	0.867	0.100	124	9044.06
hf.sc	final-te-micro-f1	0.807	0.848	0.172	101	8797.42
cnn	final-te-micro-f1	0.759	0.716	0.126	100	164.84
lr	final-te-micro-f1	0.755	0.775	0.137	1539	904.00
attn	final-te-micro-f1	0.752	0.771	0.130	104	197.00
svm	final-te-micro-f1	0.740	0.752	0.147	1482	573.13
lstm	final-te-micro-f1	0.727	0.763	0.177	99	186.14
nb	final-te-micro-f1	0.644	0.642	0.241	175	875.38

	macro-f1	micro-f1	timelapse
avg ML model	0.640	0.713	783.34
avg. static neural	0.685	0.746	182.66
avg. transformer	0.759	0.828	8920.74

If we look at the performance data by Dataset, as illustrated in the two tables below, we see the biggest gap between the best transformer models and the next best type of classifier for Macro-F1 values being the largest for the RCV1-V2 dataset, which was the largest dataset we tested with at .185%, followed by the arxiv dataset at .12% and then the ohsumed dataset at .078%. The gap is smaller on the Micro-F1 measure for those datasets at .09%, .119% and .062% respectively. The arxiv gap size being explained possibly by the nature of the underlying data which is not just highly technical but also includes a lot of special symbols and mathematical formulas that transformer models would be better suited at handling than static language models would, or ML classifiers where the data preprocessing would pull much of that noise out of the input data. The glaring exception being the BBC-News dataset, the smallest and most straightforward dataset we tested with, where the CNN classifier performed slightly better than the transformer models, for both Macro-F1 and Micro-F1 measures, but that is due no doubt to its simplicity of format and underlying language being used.

Table 10: Global Summary by Dataset (Macro-F1 Scores)

dataset	classifier	measure	mean_valu	median_val	std_dev	co	mean_timelap
20newsgroups	hf.cnn	final-te-macro-f1	0.690	0.688	0.009	13	3986.35
20newsgroups	hf.sc	final-te-macro-f1	0.683	0.684	0.012	13	3843.29
20newsgroups	cnn	final-te-macro-f1	0.672	0.674	0.014	12	115.84
20newsgroups	attn	final-te-macro-f1	0.646	0.642	0.021	12	54.03
20newsgroups	lstm	final-te-macro-f1	0.640	0.642	0.030	12	56.35
20newsgroups	lr	final-te-macro-f1	0.620	0.641	0.075	186	122.39
20newsgroups	svm	final-te-macro-f1	0.590	0.625	0.093	186	60.39
20newsgroups	nb	final-te-macro-f1	0.586	0.596	0.056	20	11.70
arxiv	hf.cnn	final-te-macro-f1	0.731	0.726	0.029	13	4516.82
arxiv	lr	final-te-macro-f1	0.624	0.652	0.098	186	199.17
arxiv	svm	final-te-macro-f1	0.611	0.638	0.108	186	82.80
arxiv	cnn	final-te-macro-f1	0.595	0.595	0.052	24	139.24
arxiv	attn	final-te-macro-f1	0.579	0.580	0.060	24	152.10
arxiv	hf.sc	final-te-macro-f1	0.547	0.681	0.314	13	6640.57
arxiv	lstm	final-te-macro-f1	0.499	0.568	0.200	23	159.15
arxiv	nb	final-te-macro-f1	0.234	0.226	0.236	20	9.24
arxiv_protoformer	hf.cnn	final-te-macro-f1	0.848	0.847	0.006	13	15057.97
arxiv_protoformer	hf.sc	final-te-macro-f1	0.847	0.847	0.008	13	13097.29
arxiv_protoformer	attn	final-te-macro-f1	0.823	0.824	0.007	12	363.89
arxiv_protoformer	lstm	final-te-macro-f1	0.822	0.824	0.007	11	329.44
arxiv_protoformer	lr	final-te-macro-f1	0.810	0.821	0.040	244	1810.99
arxiv_protoformer	nb	final-te-macro-f1	0.808	0.808	0.008	38	49.06
arxiv_protoformer	svm	final-te-macro-f1	0.806	0.821	0.034	187	1049.55
arxiv_protoformer	cnn	final-te-macro-f1	0.804	0.807	0.019	12	276.43
bbc-news	cnn	final-te-macro-f1	0.979	0.982	0.009	12	7.68
bbc-news	hf.cnn	final-te-macro-f1	0.973	0.972	0.010	25	372.04
bbc-news	hf.sc	final-te-macro-f1	0.972	0.972	0.008	13	338.51
bbc-news	lr	final-te-macro-f1	0.969	0.980	0.033	186	3.23
bbc-news	nb	final-te-macro-f1	0.968	0.975	0.014	20	0.09
bbc-news	svm	final-te-macro-f1	0.948	0.974	0.099	186	3.18
bbc-news	attn	final-te-macro-f1	0.947	0.944	0.011	12	7.91
bbc-news	lstm	final-te-macro-f1	0.931	0.927	0.015	12	7.34
imdb	hf.sc	final-te-macro-f1	0.929	0.934	0.026	13	5614.67
imdb	hf.cnn	final-te-macro-f1	0.889	0.935	0.167	13	5343.42
imdb	attn	final-te-macro-f1	0.883	0.884	0.008	12	100.11
imdb	cnn	final-te-macro-f1	0.870	0.869	0.012	12	173.42
imdb	lstm	final-te-macro-f1	0.867	0.870	0.018	12	126.45
imdb	lr	final-te-macro-f1	0.855	0.857	0.036	190	139.70
imdb	svm	final-te-macro-f1	0.843	0.849	0.058	190	94.81
imdb	nb	final-te-macro-f1	0.821	0.827	0.018	20	4.90
ohsumed	hf.cnn	final-te-macro-f1	0.726	0.724	0.011	13	9569.17
ohsumed	hf.sc	final-te-macro-f1	0.715	0.712	0.021	13	8848.25
ohsumed	lstm	final-te-macro-f1	0.649	0.654	0.029	12	162.25
ohsumed	attn	final-te-macro-f1	0.643	0.648	0.029	12	188.86
ohsumed	cnn	final-te-macro-f1	0.634	0.649	0.036	12	204.21
ohsumed	lr	final-te-macro-f1	0.573	0.603	0.089	190	199.21
ohsumed	svm	final-te-macro-f1	0.567	0.585	0.084	190	635.01
ohsumed	nb	final-te-macro-f1	0.309	0.287	0.175	20	13.83
rcv1	hf.cnn	final-te-macro-f1	0.694	0.695	0.017	13	37901.20
rcv1	hf.sc	final-te-macro-f1	0.679	0.682	0.018	8	38358.23
rcv1	attn	final-te-macro-f1	0.508	0.504	0.037	8	731.22
rcv1	lr	final-te-macro-f1	0.495	0.492	0.066	171	4709.55
rcv1	svm	final-te-macro-f1	0.479	0.479	0.070	171	2726.32
rcv1	lstm	final-te-macro-f1	0.478	0.487	0.056	5	789.74
rcv1	cnn	final-te-macro-f1	0.435	0.421	0.045	4	575.69
rcv1	nb	final-te-macro-f1	0.221	0.165	0.116	18	8349.90
reuters21578	hf.cnn	final-te-macro-f1	0.596	0.606	0.044	21	5680.39
reuters21578	hf.sc	final-te-macro-f1	0.565	0.547	0.074	15	5513.30
reuters21578	attn	final-te-macro-f1	0.552	0.553	0.030	12	200.82
reuters21578	cnn	final-te-macro-f1	0.536	0.533	0.028	12	125.66
reuters21578	lr	final-te-macro-f1	0.510	0.526	0.068	186	103.42
reuters21578	svm	final-te-macro-f1	0.498	0.513	0.064	186	113.00
reuters21578	lstm	final-te-macro-f1	0.481	0.484	0.039	12	247.16
reuters21578	nb	final-te-macro-f1	0.290	0.256	0.094	20	11.78

Table 11: Global Summary by Dataset (Micro-F1 Scores)

dataset	classifier	measure	mean_valu	median_val	std_dev	cou	mean_timelaps
20newsgroups	hf.cnn	final-te-micro-f1	0.699	0.698	0.010	13	3986.35
20newsgroups	hf.sc	final-te-micro-f1	0.693	0.695	0.010	13	3843.29
20newsgroups	cnn	final-te-micro-f1	0.684	0.686	0.014	12	115.84
20newsgroups	attn	final-te-micro-f1	0.655	0.651	0.020	12	54.03
20newsgroups	lstm	final-te-micro-f1	0.650	0.655	0.029	12	56.35
20newsgroups	lr	final-te-micro-f1	0.629	0.651	0.076	186	122.39
20newsgroups	nb	final-te-micro-f1	0.610	0.624	0.065	20	11.70
20newsgroups	svm	final-te-micro-f1	0.596	0.631	0.097	186	60.39
arxiv	hf.cnn	final-te-micro-f1	0.739	0.733	0.028	13	4516.82
arxiv	lr	final-te-micro-f1	0.620	0.656	0.107	186	199.17
arxiv	cnn	final-te-micro-f1	0.615	0.616	0.047	24	139.24
arxiv	svm	final-te-micro-f1	0.611	0.643	0.115	186	82.80
arxiv	attn	final-te-micro-f1	0.591	0.596	0.059	24	152.10
arxiv	hf.sc	final-te-micro-f1	0.553	0.685	0.318	13	6640.57
arxiv	lstm	final-te-micro-f1	0.510	0.585	0.203	23	159.15
arxiv	nb	final-te-micro-f1	0.221	0.213	0.222	20	9.24
arxiv_protoformer	hf.cnn	final-te-micro-f1	0.849	0.848	0.006	13	15057.97
arxiv_protoformer	hf.sc	final-te-micro-f1	0.846	0.848	0.008	13	13097.29
arxiv_protoformer	attn	final-te-micro-f1	0.822	0.823	0.007	12	363.89
arxiv_protoformer	lstm	final-te-micro-f1	0.821	0.824	0.007	11	329.44
arxiv_protoformer	lr	final-te-micro-f1	0.810	0.821	0.039	244	1810.99
arxiv_protoformer	nb	final-te-micro-f1	0.809	0.810	0.009	38	49.06
arxiv_protoformer	svm	final-te-micro-f1	0.807	0.821	0.034	187	1049.55
arxiv_protoformer	cnn	final-te-micro-f1	0.803	0.807	0.018	12	276.43
bbc-news	cnn	final-te-micro-f1	0.981	0.983	0.009	12	7.68
bbc-news	hf.cnn	final-te-micro-f1	0.974	0.973	0.009	25	372.04
bbc-news	hf.sc	final-te-micro-f1	0.973	0.973	0.007	13	338.51
bbc-news	lr	final-te-micro-f1	0.969	0.981	0.033	186	3.23
bbc-news	nb	final-te-micro-f1	0.968	0.975	0.014	20	0.09
bbc-news	attn	final-te-micro-f1	0.951	0.946	0.010	12	7.91
bbc-news	svm	final-te-micro-f1	0.950	0.973	0.093	186	3.18
bbc-news	lstm	final-te-micro-f1	0.935	0.929	0.014	12	7.34
imdb	hf.sc	final-te-micro-f1	0.929	0.934	0.026	13	5614.67
imdb	hf.cnn	final-te-micro-f1	0.902	0.935	0.121	13	5343.42
imdb	attn	final-te-micro-f1	0.883	0.884	0.008	12	100.11
imdb	cnn	final-te-micro-f1	0.870	0.869	0.012	12	173.42
imdb	lstm	final-te-micro-f1	0.867	0.870	0.018	12	126.45
imdb	lr	final-te-micro-f1	0.855	0.857	0.036	190	139.70
imdb	svm	final-te-micro-f1	0.845	0.849	0.049	190	94.81
imdb	nb	final-te-micro-f1	0.820	0.827	0.018	19	5.10
ohsumed	hf.cnn	final-te-micro-f1	0.746	0.748	0.009	13	9569.17
ohsumed	hf.sc	final-te-micro-f1	0.739	0.736	0.015	13	8848.25
ohsumed	lstm	final-te-micro-f1	0.684	0.687	0.020	12	162.25
ohsumed	attn	final-te-micro-f1	0.675	0.677	0.020	12	188.86
ohsumed	cnn	final-te-micro-f1	0.673	0.677	0.028	12	204.21
ohsumed	lr	final-te-micro-f1	0.609	0.633	0.080	190	199.21
ohsumed	svm	final-te-micro-f1	0.602	0.616	0.074	190	635.01
ohsumed	nb	final-te-micro-f1	0.422	0.391	0.110	20	13.83
rcv1	hf.cnn	final-te-micro-f1	0.863	0.863	0.006	13	37901.20
rcv1	hf.sc	final-te-micro-f1	0.857	0.858	0.008	8	38358.23
rcv1	lstm	final-te-micro-f1	0.773	0.782	0.031	5	789.74
rcv1	attn	final-te-micro-f1	0.769	0.771	0.032	8	731.22
rcv1	cnn	final-te-micro-f1	0.730	0.713	0.048	4	575.69
rcv1	lr	final-te-micro-f1	0.727	0.750	0.073	171	4709.55
rcv1	svm	final-te-micro-f1	0.723	0.742	0.067	171	2726.32
rcv1	nb	final-te-micro-f1	0.558	0.575	0.073	18	8349.90
reuters21578	hf.cnn	final-te-micro-f1	0.886	0.887	0.008	21	5680.39
reuters21578	hf.sc	final-te-micro-f1	0.876	0.876	0.015	15	5513.30
reuters21578	attn	final-te-micro-f1	0.842	0.838	0.017	12	200.82
reuters21578	cnn	final-te-micro-f1	0.840	0.838	0.010	12	125.66
reuters21578	lstm	final-te-micro-f1	0.813	0.816	0.029	12	247.16
reuters21578	lr	final-te-micro-f1	0.799	0.820	0.065	186	103.42
reuters21578	svm	final-te-micro-f1	0.787	0.807	0.089	186	113.00
reuters21578	nb	final-te-micro-f1	0.591	0.629	0.108	20	11.78

However, for the 20newsgroups, arxiv_protoformer, bbc-news, imdb, and reuters21578 datasets, there exists a static neural model classifier that comes within .05% on both the Macro-F1 and Micro-F1 measures, indicating that in many cases, these models can compete with the transformer models while greatly reducing computational resource requirements – on average the static neural classifiers took 2.27% of the amount of time to train, and again this is on 1/4 the number of GPUs or .57% of the computational resource requirements.

Table 12: Summary Global Macro-F1 Gap Scores & Times

	Best Transformer Over Best Static LM Value	Best Trasformer Over Best Static LM Value %	Best Transformer Over Best ML Value	Best Transformer Over Best ML Value %	Best Transformer Over Best Static LM Time (seconds)	Best Transformer Over Best Static LM Time %	Best Transformer Over Best ML Time (seconds)	Best Transfomer Over Best ML Time %
	0.018	2.60%	0.070	10.11%	3870.51	3441.28%	3863.96	3257.20%
	0.120	16.35%	0.107	14.67%	4377.58	3243.87%	4317.65	2267.81%
	0.025	3.00%	0.039	4.56%	14694.08	4138.05%	13246.98	831.48%
	-0.007	-0.69%	0.004	0.38%	364.36	4843.29%	368.81	11532.04%
	0.046	4.91%	0.074	7.96%	5514.56	5608.59%	5474.97	4018.99%
	0.078	10.73%	0.153	21.08%	9406.91	5897.69%	9369.95	4803.49%
	0.185	26.71%	0.199	28.63%	37169.97	5183.25%	33191.64	804.77%
	0.044	7.38%	0.086	14.46%	5479.56	2828.55%	5576.96	5492.41%
AVG:	***0.064***	***8.87%***	***0.091***	***12.73%***	***10109.69***	***4398.07%***	***9426.37***	***4126.02%***
				Min:	***168.49***	***Min:***	***157.11***	

When we look at the overall differences, on average across all the datasets, for the different types of classifiers in comparison with each other, using the best results from each classifier type, we find that on average the best transformer model for a given dataset yields an increase of Macro-F1 score of .064 over the best neural static language model classifier for that dataset, an 8.87% improvement, and an improvement over the best ML classifier for that dataset, or a 12.73% improvement. However, to get those better results required - again on average across all the different datasets and using the best model from each classifier type – 10109.69 seconds, or 168.49 minutes, more compute time, a 4398.07% increase relative to the best static neural classifier for that dataset, and 9426.37 seconds, or 157.11 min, more compute time, a 4126.02%, increase relative to the best ML classifier for that dataset.

Table 13: Summary Global Micro-F1 Gap Scores & Times

	Best Transformer Over Best Static LM Value	Best Trasformer Over Best Static LM Value %	Best Transformer Over Best ML Value	Best Transformer Over Best ML Value %	Best Transformer Over Best Static LM Time (seconds)	Best Transformer Over Best Static LM Time %	Best Transformer Over Best ML Time (seconds)	Best Transfomer Over Best ML Time %
	0.015	2.15%	0.070	10.05%	3870.51	3441.28%	3863.96	3257.20%
	0.119	16.05%	0.119	16.05%	4377.58	3243.87%	4317.65	9206.18%
	0.027	3.15%	0.038	4.50%	14694.08	4138.05%	13246.98	831.48%
	-0.007	-0.68%	0.005	0.47%	364.36	4843.29%	368.81	6173.71%
	0.046	4.90%	0.074	7.96%	5514.56	5608.59%	5474.97	4018.99%
	0.062	8.32%	0.137	18.38%	9406.91	5897.69%	9369.95	4803.49%
	0.090	10.45%	0.136	15.76%	37111.45	4799.17%	33191.64	804.77%
	0.044	4.97%	0.087	9.83%	5479.56	2828.55%	5576.96	5492.41%
AVG:	***0.049***	***6.16%***	***0.083***	***10.38%***	***10102.38***	***4350.06%***	***9426.37***	***4323.53%***
				Min:	***168.37***	***Min:***	***157.11***	

For the Micro-F1 scores across the different types of classifiers we test with, we see that the best transformer models for each dataset yield an average increase of .049 over the static neural classifiers, and a .083 average increase over the best ML classifiers for the given dataset, a 6.16% and 10.38% increase respectively. The performance cost for those improvements were - again on average across all the datasets - 10102.38 seconds (168.37 minutes) over the static neural language model classifiers, and 9426.37 seconds (157.11 minutes) over the best ML classifiers for that dataset, which represent a 4350.06% and 4323.53% increase in compute time respectively to achieve said better results.

A few important points of clarification for the timelapse numbers we report however:

i. the ML classifiers were run with their default options which means that a) the Macro-F1 and Micro-F1 scores are not optimized (they could likely be improved slightly), and b) because they are run

without any optimization techniques (like GridSearch Cross Validation for example) their runtimes for the classifier construction are lower that they would be if optimized.

ii. as mentioned previously, the static neural models were run with a single GPU whereas the transformer models ran on 4 GPUs in parallel (same GPU specs for the GPUs), so the timelapse figures for the training times of the transformer models are a factor of (somewhere in the neighborhood of) 4 times their neural counterparts, in terms of computational requirements for classifier training.

iii. The ML classifiers are CPU bound rather than GPU bound so they are not at all an apples-to-apples type of comparison even though we do compare them as such. Having said that however, high end CPUs are much more widely available, and less expensive so it's fair to say that scaling in a CPU bound environment versus a GPU bound environment, at least from a hardware perspective, is, and will remain for the foreseeable future, much less expensive relatively speaking.

8 Summary & Concluding Remarks

Classifying natural language into predefined categories is a fundamental problem in NLP that has applicability across a wide range of industries, the foundations of which inform other important NLP areas of research such as topic modeling and sentiment analysis, threat and anomaly detection in cyber security, and disease diagnosis in biology, genomics and healthcare. While it is relatively straightforward to outline, at least in the supervised approach that we take in this research, it nonetheless serves as a very good litmus test for looking at the relative performance characteristics of different types of language models and different types of classifiers which is the thrust of Layer Cake.

With this research we have created a robust platform for testing both single-label and multi-label datasets, with language models that range from the static, like Word2Vec, GloVe and FastText, to the dynamic transformer based models from the BERT, RoBERTa, DistilBERT, XLNet, GPT-2, Llama and DeepSeek families, testing them under a variety of hyperparameter conditions, with classical ML classifiers using scikit-learn APIs, as well as DL methods using PyTorch and Hugging Face libraries. Layer Cake produces metrics for all these different configurations and datasets and outputs data on model performance in terms of efficacy (Macro-F1, Micro-F1 et al), as well as build and training times (our timelapse measure), and in so doing we are able to yield insights with respect to LLM performance that have hitherto been absent from the research literature and by doing so help both researchers, as well as ML and AI practitioners in the field, to hone in on the right model for the right problem – one of the most important facets of building out any ML or AI based solution.

With the introduction of Layer Cake which drives the results that we analyze with this work, we look to focus the attention specifically on this particular question – namely how much better do these LLMs actually perform on a foundational NLP problem like text classification, relative to their static language model predecessors and their more classical predecessors. Our findings show that yes, transformer models do in fact perform better on average than their static language models counterparts, 8.87% better on average than static embedding DL classifiers from a Macro-F1 score perspective, and better than classical ML classifiers by 12.73%, but these gains come with significant additional computational requirements - 4398.07% and 4126.02% respectively. Similarly, with respect to Micro-F1 score, we see the LLMs outperform the static language model DL methods by 6.16%, and by 10.38% versus the ML classifiers, but those gains coming at a compute cost of 4350.06% over the static neural language model classifiers, and 4323.53% increase in compute time for the training of the ML classifiers. And these numbers do not take into account that we were using 4x the number of GPUs to train the transformer models, albeit using the less performant PyTorch data parallelism method (Data Parallelism or DP).

What we do with this research is to ground the conversation about LLM performance in a computational context so that actual performance gains can be understood relative to the computational cost necessary to achieve such gains. And while most certainly we can expect continued investment in GPU architecture to help solve for this computational gap, if we may coin a phrase here, nonetheless the cost of closing this gap at the present time, given the numbers we report in this work, remains a quite expensive proposition. At present, to procure environments that leverage a single NVIDIA H100 which sits at the forefront of AI and LLM training GPU technology right now, a chip with 80GB of RAM which is optimized for LLM training specifically going for anywhere between $2 and $3 an hour. [35] These chips have 80GB of memory and while

[35] Reflects current list hourly prices available from CoreWeave, AWS, Lambda and JarvisLabs as of 04/12/2025.

list at around $25,000 per chip, can go for more depending upon availability. In contrast, an Apple MacBook with an M3 chip with 128GB of RAM, which is available for both the MPS (GPU) and CPU chips on that system that are available costs anywhere between $5000 and $6000 depending upon configuration.[36] While this is not necessarily a straight apples to apples comparison, it nonetheless sheds light on just how much more computationally expensive these LLMs are to use in the event that there exist more straightforward, and less computationally intensive, options for the same solve.

This kind of research is absolutely essential for the AI and ML research and development community so that practical decisions can be made about how and when LLMs should be used in real-world contexts where cost, and time, are important ingredients for successful outcomes. In short, in this work we look to answer the question, again within the context of text classification specifically but with broader implications on how these LLMs are being used to solve NLP problems more generally, what kind of performance improvement can we expect to gain by using these LLMs and *at what (computational) cost?* Our findings here show that, consistent with past research on LLM performance in NLP, the best performing models are in fact LLMs, but a) they cost significantly more time and money to train and b) in almost all cases there exists a classical ML, CPU bound alternative that performs almost as well as best transformer models for that particular dataset.

What we also find, is that Word-Class Embeddings, which have been shown to improve the performance of text classification models in both ML classifier context as well as static embedding neural models, are not at all effective in a transformer, context-aware embedding environment. This is in all likelihood due to a combination of a) the already granular, context-aware nature of the transformer models which include this type of word/token to class type information, and b) the transformer-based models are designed to be self-contained and are not set up to be augmented with additional embedding information that comes from outside the model. This is of course another important factor that should be considered when determining which type of language model is optimal for a given problem, whether or not it is of value to include embeddings that are computed outside of the model to boost performance.

8.1 Areas of Promising Research

While the testing we've done with Layer Cake covers a broad spectrum of classifiers and language model forms, we nonetheless limit the study to Euclidean spatial geometry, i.e. all of the language representation forms we work with are constructed in Euclidean spaces. However, there is a lot of promising research that has been done with hyperbolic geometry (Tifrea, et al., 2019) (Leimeister & Wilson, 2019) (Valentino, et al., 2024) and it would be very interesting to see how these embedding forms performed relatively speaking, or to see if using these embeddings that are trained in hyperbolic spaces, which lend themselves to a more hierarchical structure, could be used in conjunction with some of the other language representation forms we test with (like we do with WCEs) and what kind of effect performance wise that had on the models.

From a performance standpoint, it would also be interesting to see what the effect might be of using different parallelism approaches to transformer model training to see what kind of performance effects they had on model training times, effectively extending Layer Cake to support such analysis. In our case, we used the default Data Parallel (DP) approach which comes built in to the Hugging Face APIs but the *Distributed Data Parallel* (DDP) PyTorch capabilities are supposed to surpass those of the more straightforward DP form

[36] Source: Amazon

of parallelism so it would be interesting to see what this difference was performance wise in terms of model training. Using DPP would also allow the testing of larger LLMs, LLMs that could be spread across multiple GPUs – models that could not be included in this study due to single GPU memory constraints that prevent the use of larger models in a DP type of setup.

From a language representation perspective, in particular with respect to the Deep Learning approaches we cover with Layer Cake, it would be interesting to see if we could combine vectorization based approaches with the standard token indexing approaches to the model, particularly within the context of classification problems as that would give us an opportunity perhaps – like with tf-idf vectorization – to weight the different tokens alongside the attention based masking that the transformer models are known for. This has been shown to be an effective approach for certain NLP problems, like sentiment analysis, as exemplified in (Liang & Niu, 2022) (Zhao, 2022) (Kamyab, et al., 2021).

Each of these areas will no doubt prove to be fruitful areas of research in the coming years as both the research community and practitioner community of language models looks to both improve and optimize the performance of LLMs across the entire spectrum of NLP, which includes point solutions which involve custom models and language representation approaches as well as field of language generation, a field whose promise continues to drive innovation and investment in various forms of language representation.

9 Bibliography

Bai, J., 2023. Qwen Technical Report. *arxiv,* p. https://doi.org/10.48550/arXiv.2309.16609.

Bi, X. et al., 2024. DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. *arXiv,* 5 January.p. https://doi.org/10.48550/arXiv.2401.02954.

Bojanowski, P., Grave, E., Joulin, A. & Mikolov, T., 2017. Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics,* 01 06, Volume 5, pp. 135-146; https://doi.org/10.1162/tacl_a_00051.

Brown, T. B. et al., 2020. *Language Models are Few-Shot Learners.* Red Hook, NY, USA, Curran Associates Inc., pp. 1877-1901.

Chomsky, N., 1956. Three models for the description of language. *IRE Transactions on Information Theory,* September, 2(3), pp. 113-124. https://doi.org/10.1109/TIT.1956.1056813.

Chomsky, N., 1957. *Syntactic Structures.* Berlin, Boston: De Gruyter Mouton.

Chomsky, N., 1965. *Aspects of the Theory of Syntax.* Cambridge(MA): MIT Press.

Clark, K., Luong, M.-T., Le, Q. V. & Manning, C. D., 2020. ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. *arXiv,* p. https://doi.org/10.48550/arXiv.2003.10555.

Dai, Z. et al., 2019. Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context. *arxiv,* 2 June.p. https://doi.org/10.48550/arXiv.1901.02860.

Deerwester, S. et al., 1990. Indexing by latent semantic analysis. *Journal of the American Society for Information Science,* 41(6), pp. 391-407. https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9.

Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K., 2019. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* Minneapolis, Minnesota, Association for Computational Linguistics, pp. 4171-4186. https://doi.org/10.48550/arXiv.1810.04805.

Farhangi, A. et al., 2022. *Protoformer: Embedding Prototypes for Transformers.* Cham, Springer International Publishing, pp. 447-458.

Firth, J. R., 1957. A synopsis of linguistic theory 1930-55. *Studies in Linguistic Analysis (special volume of the Philological Society),* Volume 1952-59, pp. 1-32.

Greene, D. & Cunningham, P., 2006. *Practical solutions to the problem of diagonal dominance in kernel document clustering.* Pittsburgh, PA, Association for Computing Machinery, pp. 377-384. https://doi.org/10.1145/1143844.1143892.

Guo, D. et al., 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv,* January.p. https://doi.org/10.48550/arXiv.2501.12948.

Harris, Z. S., 1954. Distributional Structure. *WORD,* 08 January, 10(2-3), pp. 146-162. https://doi.org/10.1080/00437956.1954.11659520.

Hinton, J., Vinyals, O. & Dean, J., 2015. Distilling the Knowledge in a Neural Network. *arXiv,* March.p. https://doi.org/10.48550/arXiv.1503.02531.

Joachims, T., 1998. *Text categorization with Support Vector Machines: Learning with many relevant features.* Berlin, Heidelberg, Springer Berlin Heidelberg, pp. 137-142.

Jones, C. R. & Bergen, B. K., 2025. Large Language Models Pass the Turing Test. 31 March.p. https://doi.org/10.48550/arXiv.2503.23674.

Kamyab, M., Liu, G. & Adjeisah, M., 2021. Attention-Based CNN and Bi-LSTM Model Based on TF-IDF and GloVe Word Embedding for Sentiment Analysis. *Applied Sciences,* 11(23), p. https://doi.org/10.3390/app112311255.

Katz, D. M., Bommarito, M. J., Gao, S. & Arredondo, P., 2024. GPT-4 passes the bar exam. *Philosophical Transactions of the Royal Society A,* Volume 382, p. https://doi.org/10.1098/rsta.2023.0254.

Kudo, T. & Richardson, J., 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. *arXiv,* 19 August.p. https://doi.org/10.48550/arXiv.1808.06226.

Lang, K., 1995. *NewsWeeder: Learning to Filter Netnews*. San Francisco, CA, Morgan Kaufmann, pp. 331-339.

Lan, Z. et al., 2019. ALBERT: A Lite BERT for Self-supervised Learning of Language Representations. *arXiv,* p. https://doi.org/10.48550/arXiv.1909.11942.

Leimeister, M. & Wilson, B. J., 2019. Skip-gram word embeddings in hyperbolic space. *arxiv,* 27 May.

Lewis, D., 1987. *Reuters-21578 Text Categorization Collection [Dataset],* s.l.: UCI Machine Learning Repository.

Lewis, D. D., Yang, Y., Rose, T. G. & Li, F., 2004. RCV1: A New Benchmark Collection for Text Categorization Research. *The Journal of Machine Learning Research,* 1 December, Volume 5, pp. 361-397.

Liang, M. & Niu, T., 2022. Research on Text Classification Techniques Based on Improved TF-IDF Algorithm and LSTM Inputs. *Procedia Computer Science,* Volume 208, pp. 460-470; https://doi.org/10.1016/j.procs.2022.10.064.

Li, Q. et al., 2022. A Survey on Text Classification: From Traditional to Deep Learning. *ACM Transactions on Intelligent Systems and Technology (TIST),* 8 April, 13(2), pp. 1-41; https://doi.org/10.1145/3495162.

Liu, G. & Guo, J., 2019. Bidirectional LSTM with attention mechanism and convolutional layer for text classification. *Neurocomputing,* 14 March, Volume 337, pp. 325-338. https://doi.org/10.1016/j.neucom.2019.01.078.

Liu, Y. et al., 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. p. https://doi.org/10.48550/arXiv.1907.11692.

Maas, A. L. et al., 2011. *Learning Word Vectors for Sentiment Analysis*. Portland, Oregon, Association for Computational Linguistics, pp. 142-150.

McCallum, A. & Nigam, K., 1998. *A comparison of event models for naive bayes text classification*. s.l., s.n.

Mikolov, T., Chen, K., Corrado, G. & Dean, J., 2013. *Efficient Estimation of Word Representations in Vector Space*. s.l., arXiv, p. https://doi.org/10.48550/arXiv.1301.3781.

Minaee, S. et al., 2021. Deep Learning--based Text Classification: A Comprehensive Review. *ACM Computing Surveys (CSUR),* April, 54(3), pp. 1-40; https://doi.org/10.1145/3439726.

Moreo, A., Esuli, A. & Sebastiani, F., 2021. Word-class embeddings for multiclass text classification. *Data Mining and Knowledge Discovery,* May, Volume 35, pp. 911-963. https://doi.org/10.1007/s10618-020-00735-3.

Ouyang, L. et al., 2022. Training language models to follow instructions with human feedback. *arXiv,* 4 March.p. https://doi.org/10.48550/arXiv.2203.02155.

Pennington, J., Socher, R. & Manning, C. D., 2014. *GloVe: Global Vectors for Word Representation*. Doha, Qatar, Association for Computational Linguistics, pp. 1532-1543. 10.3115/v1/D14-1162.

Peters, M. E. et al., 2018. *Deep contextualized word representations*. New Orleans, Louisiana, Association for Computational Linguistics, pp. 2227-2237. https://doi.org/10.48550/arXiv.1802.05365.

Qiu, X. et al., 2020. Pre-trained models for natural language processing: A survey. *Science China Technological Sciences,* 15 September, 63(10), pp. 1872-1897; https://doi.org/10.1007/s11431-020-1647-3.

Radford, A. et al., 2019. Language Models are Unsupervised Multitask Learners. pp. https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

Raffel, C. et al., 2019. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *arXiv,* p. https://doi.org/10.48550/arXiv.1910.10683.

Salton, G., 1971. *The SMART Retrieval System-Expermients in Automatic Document Processing*. s.l.:Prentic-Hall, Inc..

Sanderson, M., 2010. Test Collection Based Evaluation of Information Retrieval Systems. *Foundations and Tenets in Information Retrieval,* Volume 4, pp. 247-375. http://dx.doi.org/10.1561/1500000009.

Sanh, V., Debut, L., Chaumond, J. & Wolf, T., 2020. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. p. https://doi.org/10.48550/arXiv.1910.01108.

Schuster, M. & Nakajima, K., 2012. *Japanese and Korean voice search*. Kyoto, Japan, s.n., pp. 5149-5152.

Sebastiani, F., 2002. Machine learning in automated text categorization. March, 34(1), pp. 1-47. https://doi.org/10.1145/505282.505283.

Sennrich, R., Haddow, B. & Birch, A., 2016. *Neural Machine Translation of Rare Words with Subword Units*. Berlin, Germany, Association for Computational Linguistics, pp. 1715-1725.

Sinha, A. et al., 2015. *An Overview of Microsoft Academic Service (MAS) and Applications*. Florence, Italy, Association for Computing Machinery, pp. 243-246.

Sparck Jones, K., 1972. A STATISTICAL INTERPRETATION OF TERM SPECIFICITY AND ITS APPLICATION IN RETRIEVAL. *Journal of Documentation,* 01 January, 28(1), pp. 11-21. https://doi.org/10.1108/eb026526.

Tifrea, A., Bécigneul, G. & Ganea, O.-E., 2019. *Poincaré GloVe: Hyperbolic Word Embeddings*. New Orleans, Louisiana, s.n.

Touvron, H. et al., 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv,* p. https://doi.org/10.48550/arXiv.2302.13971.

Trust, P. & Minghim, R., 2024. A Study on Text Classification in the Age of Large Language Models. *Machine Learning and Knowledge Extraction,* November, 6(4), pp. 2688-2721. https://doi.org/10.3390/make6040129.

Turing, A., 1950. Computing Machinery and Intelligence. *Mind,* 59(236), pp. 433-460; 10.1093/mind/lix.236.433.

Valentino, M., Carvalho, D. S. & Freitas, A., 2024. *Multi-Relational Hyperbolic Word Embeddings from Natural Language Definitions*. St Julian's, Malta, Association for Computational Linguistics, pp. 23-34; https://doi.org/10.48550/arXiv.2305.07303.

Vaswani, A. et al., 2017. *Attention is All You Need*. Long Beach, CA, Curran Associates Inc., p. https://doi.org/10.48550/arXiv.1706.03762.

Wahba, Y., Madhavji, N. & Steinbacher, J., 2023. *A Comparison of SVM Against Pre-trained Language Models (PLMs) for Text Classification Tasks*. Cham, Switzerland, Springer Nature, pp. 304-313; https://doi.org/10.1007/9.

Wang, X. et al., 2022. Self-Consistency Improves Chain of Thought Reasoning in Language Models. *arXiv,* March.

Wei, J. et al., 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv,* 28 January.

Wei, J. et al., 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *arXiv,* p. https://doi.org/10.48550/arXiv.2201.11903.

Worth, P., 2023. Word Embeddings and Semantic Spaces in Natural Language Processing. *International Journal of Intelligence Science,* 13(1), pp. 1-21. https://doi.org/10.4236/ijis.2023.131001.

Wu, Y. et al., 2016. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. *arXiv,* p. https://doi.org/10.48550/arXiv.1609.08144.

Xu, F. et al., 2025. Towards Large Reasoning Models: A Survey of Reinforced Reasoning with Large Language Models. January.p. https://doi.org/10.48550/arXiv.2501.09686.

Yang, Z. et al., 2019. *XLNet: generalized autoregressive pretraining for language understanding*. Red Hook, NY, Curran Associates Inc., p. 5753–5763.

Yoon, K., 2014. *Convolutional Neural Networks for Sentence Classification*. Doha, Qatar, Association for Computational Linguistics, pp. 1746-1751.

Zang, Y., 2021. *Research on Text Classification Method Based on LSTM Neural Network Model*. Dalian, China, IEEE, pp. 1019-1022.

Zhang, X., Zhao, J. & LeCun, Y., 2015. *Character-level convolutional networks for text classification*. Montreal, CA, MIT Press, pp. 649-657.

Zhao, H., 2022. Research of Text Classification Based on TF-IDF and CNN-LSTM. *Journal of Physics: Conference Series,* January, 2171(1), pp. 10.1088/1742-6596/2171/1/012021.